

BNCmatmul User's Guide

Tomonori Kouya

<https://github.com/tkouya/bncmatmul>

Version 0.23: March 16, 2026

Contents

Chapter 1	What's BNCmatmul?	1
1.1	Copyright and condition for distribution	1
1.2	How to compile BNCmatmul	1
1.3	SIMD backends: coverage, ISA requirements and measured performance	2
1.4	Mathematical notation	4
1.5	Why do we require a multiple-precision floating-point arithmetic library?	5
1.6	Algorithms and performance of optimized matrix multiplication	6
1.7	Software layer of BNCmatmul	7
1.8	History of Version and Todo list	9
Chapter 2	Basic datatypes and arithmetic	10
2.1	The <code>c_dd_*</code> arithmetic layer (<code>rdd.h</code>)	10
2.2	Error-free transformation	11
2.3	Double-double precision arithmetic	11
2.4	Quadruple-double precision arithmetic	12
2.5	Triple-double precision arithmetic	13
2.6	Branch-free fused multiply-add (<code>bncfma.h</code>)	14
2.7	FMA-based elementary functions (<code>bncelem.h</code>)	15
2.8	Exported function API for DLL/Python (<code>rdtq_func.h</code>)	17
2.9	<code>rdd.h</code>	17
2.10	AVX2	20
2.11	Miscellaneous functions	22
2.12	Tool functions for benchmarking	23
Chapter 3	Basic linear computation	26
3.1	Datatypes of D, DD, TD, QD and MPFR vector and matrix	26
3.2	Vector arithmetic	27
3.3	Matrix arithmetic	29
3.4	File I/O with matrix and vector	32
3.5	Generating test matrices	33
3.6	Simple, block, and Strassen matrix multiplication and related functions	34
3.7	Getting relative errors of vector and matrix	35
3.8	Ozaki scheme and related functions	36
Chapter 4	LU decomposition	38
4.1	List of functions	38
Chapter 5	Array, polynomial and solvers of algebraic equations	40
5.1	Data types of Array and polynomial	40
5.2	List of functions	41
Chapter 6	Sparse matrix and related functions	51
6.1	Structures of CRS matrix	51
6.2	Krylov subspace iterative solvers	58

Chapter 7	Single-based and complex precision families	67
7.1	Single-based multi-component types (DS, TS, QS)	67
7.2	Complex types (CD and complex multi-component)	67
7.3	Summary of available families	68
Chapter 8	GPU computation with CUDA	70
8.1	Naming convention and data types	70
8.2	Device memory and host-device transfer	71
8.3	Vector and matrix arithmetic	71
8.4	Matrix multiplication and matrix-vector product	72
8.5	LU decomposition and linear solver on GPU	72
8.6	Sparse matrix-vector multiplication (SpMV)	72
8.7	Arbitrary precision matrix multiplication (CUDA MPFR / MPC)	73
8.8	Building and running the GPU suite	73
Chapter 9	Using BNCmatmul from Python	74
9.1	Building the shared library	74
9.2	Loading the library and printing full precision	74
9.3	Arithmetic, FMA and elementary functions	75
9.4	Elementwise array functions with NumPy	75
9.5	Notes and caveats	76
Bibliography		77

Chapter 1

What's BNCmatmul?

BNCmatmul is one of optimized BLAS(Basic Linear Algebra Subprograms) libraries supporting multiple precision arithmetics such as double(binary64), DD(double-double), TD(triple-double), QD(quadruple-double), and arbitrary precision floating point arithmetic of MPFR.

Now that MPLAPACK/MPBLAS, a library for extended multiple precision linear computation, provides the basis for a standard multiple precision numerical computing environment, there is a growing demand for an optimized library that provides faster basic linear computation. Standard optimization methods include the use of SIMD instructions and parallelization with OpenMP, but for multiple precision matrix multiplication, optimization with algorithms such as divide-and-conquer method and Ozaki scheme are also available. We have developed new version of BNCmatmul that can use all of these optimization methods and supports both fixed-precision computation using the multi-component way and arbitrary-precision computation using the multi-digit way. In this paper, we describe its software structure and present the results of performance evaluation of optimized matrix-vector multiplication and matrix multiplication.

We are developing BNCmatmul on the following environment: Xeon and EPYC. On other x86_64 linux environment with Intel OneAPI or GNU Compiler Collection, our library can probably be available.

EPYC AMD EPYC 7402P 24 cores, Ubuntu 18.04.6 LTS, Intel Compiler version 2021.4.0, MPLAPACK 1.0.1, MPFR 4.1.0

Xeon Intel Xeon W-2295 3.0GHz 18 cores, Ubuntu 20.04.3 LTS, Intel Compiler version 2021.5.0, MPLAPACK 1.0.1, MPFR 4.1.0

From BNCmatmul version 0.23, Arm Neon of AArch64 can be used to accelerate our library. We has tested in following Arm CPU environment,

Raspberrypi Raspberry Pi 5 Model B Rev 1.0, Broadcom BCM2712 2.4GHz Coretex-A76 4 cores, Debian GNU/Linux 12 ,GCC 12.2.0, MPLAPACK 2.0.1

Snapdragon Qualcomm Snapdragon X Plus 3.24GHz 8 cores, Ubuntu 24.04.3 LTS on Windows 11, GCC 13.3.0, MPLAPACK 2.0.1

1.1 Copyright and condition for distribution

BNCmatmul is originated by Tomonori Kouya (<https://na-inet.jp/>), and including this document is distributed under [GPL] version 2 or later.

[GPL]: <https://www.gnu.org/copyleft/gpl.html>

1.2 How to compile BNCmatmul

BNCmatmul can be compiled with Intel OneAPI Compiler (ICC) or GNU Compiler Collection (GCC) as follows:

1. Git clone at your home directory from <https://github.com/tkouya/bncmatmul>.

2. Run `./configure` to detect the compilers and the GMP/MPFR/MPC/QD installations and to generate `bncmatmul.inc`; the file is generated, so edit `bncmatmul.inc.in` (or pass configure options) rather than `bncmatmul.inc` itself.
3. Run "make all" to build `libbncmatmul*.a` and `python/libbncmatmul*.so`.
4. Compile some iterative refinement C++ programs in "test" directory with BNCmatmul and MPLAS/MPBLAS.

If GMP, MPFR, MPC and QD are not under the default prefix, point configure at them:

```
./configure --with-gmp=$HOME/local --with-mpfr=$HOME/local \
--with-mpc=$HOME/local --with-qd=$HOME/local \
--enable-omp --enable-avx2 --enable-avx512
```

`-enable-avx2` and `-enable-avx512` default to *auto*, which turns both on when the host is x86-64 (`-enable-neon` and `-enable-sve2` do the same on AArch64); pass `-disable-avx512` to skip that backend. The summary printed at the end of `configure` reports which backends were enabled and with what flags. Note that configure detects the *host* architecture, not whether this particular CPU can execute AVX-512, so on a machine without AVX-512 support the `avx512` target must be disabled explicitly.

`make` then has one target per SIMD backend. Each of them compiles *the whole library again* with the SIMD flags of that backend and archives it under its own name, so the backends coexist and are selected purely at link time:

target	SIMD flags	libraries produced
<code>make</code>	—	<code>libbncmatmul-0.24.a</code> , ... <code>_winograd.a</code>
<code>make avx2</code>	<code>-mavx2 -mfma</code>	<code>libbncmatmul-0.24_avx2.a</code> , <code>-omp_avx2.a</code> , <code>_winograd_a</code>
<code>make avx512</code>	<code>-mavx512f -mfma</code>	<code>libbncmatmul-0.24_avx512.a</code> and the same three
<code>make neon</code>	<code>-mcpu=cortex-a76 -DBNC_ENABLE_NEON</code>	<code>libbncmatmul-0.24_neon.a</code> , <code>-omp_neon.a</code>
<code>make sve2</code>	<code>-mcpu=neoverse-v2 -DBNC_ENABLE_SVE2</code>	<code>libbncmatmul-0.24_sve2.a</code> , <code>-omp_sve2.a</code>

A program is moved from one backend to another by changing the `-l` flag only; compile the caller with the matching `-mavx2` `-mfma` or `-mavx512f` `-mfma` so that the inline SIMD code in the headers takes the same branch:

```
g++ -O3 -ffp-contract=off -mavx512f -mfma -Iinclude prog.cc \
-L. -lbncmatmul-0.24_avx512 -lmpc -lmpfr -lgmp -lqd -lstdc++ -lm
```

Two cautions on include paths. First, `-Iinclude` (the source tree) must come *before* the GMP/MPFR/QD prefix: a previous `make install` leaves BNCmatmul headers under that same prefix, and the stale copies would otherwise shadow the ones the libraries were built from. Second, `-ffp-contract=off` is required — the error-free transformations rely on the compiler *not* contracting a multiply and an add into an FMA on its own.

1.3 SIMD backends: coverage, ISA requirements and measured performance

The four SIMD backends are selected by the preprocessor inside the same sources, in the order `#if __AVX512F__ ... #elif __AVX2__ ... #elif __ARM_SVE2__ ... #elif __ARM_NEON__ ... #else`, so the widest available backend wins and the final `#else` is the portable scalar path. Note that GCC defines `__AVX2__` in an AVX-512 build as well, which is why the AVX2 guard is written `#if defined(__AVX2__) && !defined(__AVX512F__)`; a construct with no `#elif __AVX512F__` arm therefore takes the *scalar* path under `-mavx512f`, not the AVX2 one. In 0.24 every source that carries AVX2 blocks also carries AVX-512 blocks, and of the 426 AVX2 constructs in the 154 sources the library is built from, only 15 lack an AVX-512 arm: `norm2_{ds,ts,qs}vector`, the `ceval_diff` polynomial

evaluations of `poly.c`, `dd_poly.c` and `cdd_poly.c`, the seven `_bncavx2*_estrin` kernels (whose `__m256d ret[]` argument signature is AVX2-specific — the AVX-512 build calls the scalar dispatcher instead), and the OpenMP LU rank-1 updates `_bncomp_{TD,QD}LUdecompPM`. Those fifteen run scalar in an `avx512` build.

ISA requirements. The AVX2 build needs AVX2 and FMA (`-mavx2 -mfma`, Haswell and later). The AVX-512 build needs only the foundation subset AVX-512F plus FMA (`-mavx512f -mfma`); none of the kernels uses AVX-512DQ/BW/VL, so the resulting library runs on any AVX-512F CPU — Skylake-SP, Ice Lake, Sapphire/Emerald Rapids and also Knights Landing. Both builds are compiled with `-ffp-contract=off`: the error-free transformations depend on the compiler not fusing a multiply and an add on its own.

Reproducibility across backends. Every SIMD kernel is a lane-wise rewrite of the scalar error-free transformation, so element-wise operations — AXPY (`add_cmul`), scaling, addition, and the sparse SpMV and SpMV^T — are *bit-identical* in the serial, AVX2 and AVX-512 builds, for all six EFT types. `bench/mds/mds_verify.cc` checks this and the three builds print identical checksums for $n = 96, 257, 512$ and 1000 .

The reduction-based kernels need one qualification. A dot product — hence GEMV and GEMM — accumulates W partial sums in the SIMD lanes and combines them at the end, which is a different summation order from the scalar loop, and multi-component addition is not associative. On random operands this is visible for `dd` and `ds`, whose GEMV and GEMM results differ between the serial, AVX2 and AVX-512 builds; the differences sit in the last limb (below 10^{-16} relative for `dd`, 10^{-14} for the ≈ 48 -bit `ds`), i.e. at the rounding level of the type itself. The `td`, `qd`, `ts` and `qs` GEMV and GEMM happened to agree bit-for-bit across all three backends in the same test (`bench/mds/mds_verify_rand.cc` reproduces it). If run-to-run reproducibility across backends matters more than speed, link every run against the same library.

Measured performance. The numbers below were taken with `bench/mds/run_x86.sh`, which drives the same `bench/mds/mds_bench` binary linked in turn against the serial, AVX2 and AVX-512 libraries, on one Intel Xeon Gold 6526Y core (Emerald Rapids, 2.8/3.9 GHz, GCC 13.3, GMP 6.3.0, MPFR 4.2.2), pinned with `taskset` and averaged over $n = 128, 256, 512, 1024$. Each entry is the geometric mean over those four sizes of the MFLOPS ratio against the serial build; the operations are AXPY (`add_cmul`), GEMV, GEMM and a banded SpMV with half-bandwidth 3.

type	AXPY		GEMV		GEMM		SpMV	
	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512
float	1.32	2.13	6.37	7.83	1.21	1.22	0.27	0.14
double	1.23	1.31	3.14	4.57	1.11	1.09	1.02	0.96
dd	5.80	8.81	6.25	6.52	3.67	3.74	1.68	0.76
td	4.68	6.83	4.63	5.21	3.17	3.62	1.47	0.64
qd	4.55	6.97	3.93	5.62	2.96	3.81	1.71	0.95
ds	9.31	13.36	10.45	7.07	4.36	3.33	0.45	0.17
ts	3.43	27.83	2.18	8.60	1.91	5.43	0.69	0.41
qs	4.72	29.69	4.39	15.56	3.92	9.07	0.99	0.59

Three things are worth reading out of the table. First, SIMD pays off far more for the multi-component types than for plain `float/double`: a DD addition is a chain of a dozen dependent `two_sum` steps, and vectorizing it over W elements removes that dependency chain, whereas plain `double` GEMM is already memory-bound. Second, AVX-512 is a further gain over AVX2 almost everywhere in the dense kernels; the exception is `ds`, whose GEMV and GEMM are slightly faster under AVX2. Third — and this is the one to keep in mind when choosing a build — SpMV *regresses* under SIMD for narrow bands, badly so under AVX-512.

The SpMV regression is structural, not a bug: `drsmatrix` and its multi-precision relatives pad every row to a multiple of the SIMD width (`_BNC_D_WIDTH` for double-based types, `_BNC_S_WIDTH` for float-based ones) so that the row loop needs no remainder handling. A band with half-bandwidth 3 has 7 non-zeros per row, which pads to 8 under AVX2 doubles and AVX-512 doubles but to 16 under AVX-512

floats — more than twice the useful work — and the irregular gather of x has no reuse to amortize it. For such matrices the serial or AVX2 library is the better choice; the wider backend starts to pay once rows hold appreciably more non-zeros than the vector width, as `bench/mds/sp_perf.cc` shows for `dd` ($n = 8192$, speedup against the serial build):

half-bandwidth	non-zeros/row	AVX2	AVX-512
1	3	0.92	0.29
3	7	1.70	0.77
7	15	2.82	1.62
15	31	3.60	2.28
31	63	4.62	3.47
63	127	4.98	4.12

AVX-512 only breaks even somewhere between 7 and 15 non-zeros per row, and even at 127 it has not caught up with AVX2 — for sparse work the AVX2 library is the safer default. The complex sparse types behave differently: `cdd`, `ctd` and `cqd` SpMV gain $2.9\text{--}4.1\times$ from either backend even at half-bandwidth 3, because each complex non-zero carries four real multiplies and thus enough work to amortize the padded lanes.

One caveat on the `ts` and `qs` rows. The AVX-512 headers `avx2/_bncavx_{ts,qs}_avx512.h` honour the build-time switches `USE_TS_BF` / `USE_QS_BF` and select the branch-free addition and multiplication kernels, while the AVX2 headers `avx2/_bncavx_{ts,qs}.h` do not implement them and always use the branchy `addq/mult` kernels (for `dd` and `ds` the asymmetry runs the other way). The two kernels agree bit-for-bit on the operands tested, so this does not change results, but the branches in the AVX2 path are why the `ts/qs` AVX-512 speedups reach $28\text{--}30\times$ where the AVX2 ones stay near $4\times$: much of that gap comes from the kernel selected, not from the vector width alone. Implementing the branch-free float-based kernels for AVX2 (and the `dd/ds` ones for AVX-512) would close it.

The complex types were measured the same way with `bench/mds/mds_bench_cplx`. Their gains are more modest for the dense kernels — each complex product is four real products plus two additions, so there is more shuffling per useful FLOP — but, as noted above, their SpMV is the one sparse case where SIMD helps:

type	AXPY		GEMV		GEMM		SpMV	
	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512	AVX2	AVX-512
<code>cd</code>	1.16	1.37	1.08	1.08	1.00	1.13	1.89	1.15
<code>cdd</code>	2.15	3.20	2.79	2.89	1.84	2.03	3.57	4.14
<code>ctd</code>	3.50	4.90	3.05	3.28	2.35	2.80	3.17	2.86
<code>cqd</code>	3.98	6.08	3.18	4.06	2.38	3.14	3.19	3.18

Both sweeps are reproduced by `bench/mds/run_x86.sh`, which leaves the raw per-size measurements in `bench/mds/results_x86.csv` and `results_x86_cplx.csv`. Individual entries move by a few per cent between runs — the host uses the `schedutil` governor with turbo enabled — and a handful of the `float/double` points swing more than that when the working set crosses a cache level, which is why the tables report a geometric mean over four sizes rather than a single number.

1.4 Mathematical notation

Here, we define $\mathbb{F}_{bS}$ and $\mathbb{F}_{bL}$ as sets of the S - and L -bit mantissas of floating-point numbers, respectively. For instance, $\mathbb{F}_{b24}$ and $\mathbb{F}_{b53}$ refer to sets of IEEE754-1985 binary32 and binary64 floating-point numbers, whereas $\mathbb{F}_{b106}$, $\mathbb{F}_{b159}$, and $\mathbb{F}_{b212}$ represent examples of DD, TD, and QD precision floating-point numbers, respectively. Although any mantissa length can be selected in MPFR arithmetic, the set of MPFR numbers is expressed as $\mathbb{F}_{bM}$, which is primarily defined as M -bit using the `mpfr_set_default_prec` function.

Moreover, we use $(\mathbf{x})_i (= x_i)$ as the i -th element of the n -dimensional vector $\mathbf{x} = [x_i]_{i=1,2,\dots,n} \in \mathbb{R}^n$, and $(A)_{ij} (= a_{ij})$ as the (i, j) -th element of $A = [a_{ij}]_{i=1,2,\dots,m,j=1,2,\dots,n} \in \mathbb{R}^{m \times n}$.

1.5 Why do we require a multiple-precision floating-point arithmetic library?

The following is a brief explanation regarding the need for multiple-precision calculations with a mantissa exceeding binary64. First, users may seek to use floating-point arithmetic to obtain a value $F(x)$ from an input value x represented by a floating-point number. Ultimately, $F(x)$ must maintain at least U significant digits. Intuitively, in numerical computations, the initial error due to rounding is assumed to be included in the input value. Suppose the number of mantissa digits for the floating-point arithmetic to be $L(> U)$. Then, the number of significant digits of $F(x)$ is $U-R$, where R denotes the digits lost in the process of computing $F(x)$. Furthermore, it is assumed that the algorithm cannot be modified to ensure accuracy with an L -digit computation.

Unless the calculation algorithm for $F(x)$ is modified, the error propagation minimally changes irrespective of the number of mantissa digits. Therefore, we can increase the number of mantissa digits, slightly exceeding R with some slack of α digits, to $L+R+\alpha$ digits. If the initial error can be significantly minimized, $F(x)$ with U or more significant digits can be subsequently obtained. This process illustrates how multiple-precision calculations are used to ensure accuracy.

In contrast, the multi-fold calculation method proposed by Rump and Ogita [9] [10] prevents an increase in the initial error by suppressing the rounding error generated by arithmetic operations to the lower digits using error-free transformation techniques. The computational manner of the error-free transformation process is almost the same as that of multiple-precision calculations. Moreover, in this process, floating-point arithmetic with L or more digits is not used. Accordingly, there is no need to perform renormalization procedures. This offers the advantage of reducing computational complexity compared to that of a multi-component approach that uses error-free transformation techniques.

A conceptual diagram of the two aforementioned approaches is presented in Fig. 1.1.

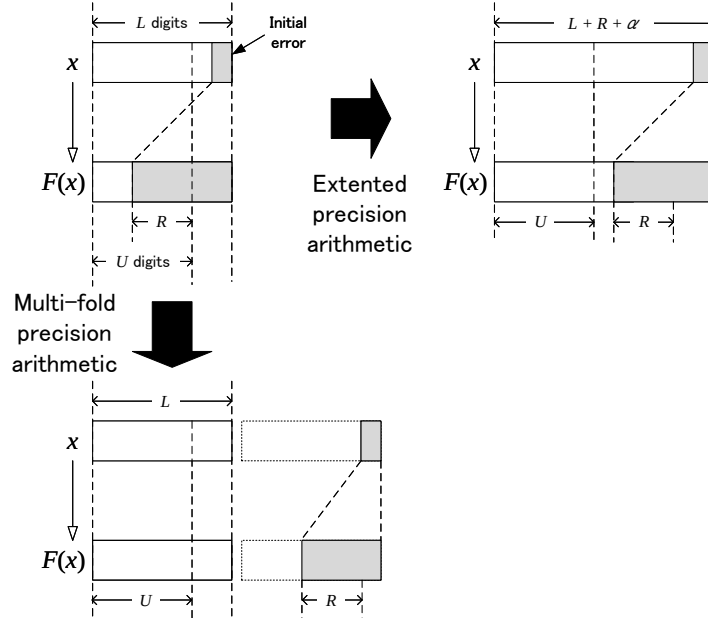


Fig. 1.1 Extended precision and multi-fold precision arithmetics

Currently, not all algorithms used in numerical computation can be applied using multi-fold arithmetic, especially with nonlinear calculations. Therefore, it is necessary to employ a combination of multi-fold and multiple-precision calculations, where the basic linear calculation part, such as explicit extrapolation process solving initial value problems of ordinary differential equations, uses multi-fold calculations to

reduce computational time.

The Ozaki scheme, a matrix multiplication algorithm that we incorporated into our library, is a technique based on the multi-fold calculation approach. By leveraging the speed of existing binary32 and binary64 xGEMM algorithms, it significantly optimizes multiple-precision matrix multiplication under certain conditions, as revealed in our benchmark and previous studies[7][11]. We are confident that the acceleration of multiple-precision linear computation libraries will remain an important theme in guaranteeing the accuracy of broader numerical computation algorithms, following the development trend of multiple-precision numerical algorithms.

1.6 Algorithms and performance of optimized matrix multiplication

We focused on the optimization of multiple-precision matrix multiplication, starting with MPI support for arbitrary-precision numerical calculations[4]. Subsequently, owing to current popularity of multi-core CPUs, we accelerated multiple-precision matrix multiplication via parallelization support using OpenMP. As far as multiple-precision calculations are concerned, no other such results have been achieved by incorporating divide-and-conquer methods, such as the Strassen and Winograd algorithms. Although limited to multi-component methods, our proposed method is the only approach that supports AVX2 and has been converted to OpenMP for achieving higher speeds. However, the current code does not improve performance beyond eight threads for OpenMP acceleration, and a drastic reformulation is necessary to further improve parallelization performance. Algorithm 1 is the Strassen matrix multiplication.

Algorithm 1 Strassen algorithm for matrix multiplication

Input: $A = [A_{ij}]_{i,j=1,2} \in \mathbb{R}^{m \times l}$, $A_{ij} \in \mathbb{R}^{m/2 \times l/2}$, $B = [B_{ij}]_{i,j=1,2} \in \mathbb{R}^{l \times n}$, $B_{ij} \in \mathbb{R}^{l/2 \times n/2}$
Output: $C := \text{Strassen}(A, B) = AB \in \mathbb{R}^{m \times n}$
if $m < m_0$ && $n < n_0$ **then**
 $C := AB$
end if
 $P_1 := \text{Strassen}(A_{11} + A_{22}, B_{11} + B_{22})$
 $P_2 := \text{Strassen}(A_{21} + A_{22}, B_{11})$
 $P_3 := \text{Strassen}(A_{11}, B_{12} - B_{22})$
 $P_4 := \text{Strassen}(A_{22}, B_{21} - B_{11})$
 $P_5 := \text{Strassen}(A_{11} + A_{12}, B_{22})$
 $P_6 := \text{Strassen}(A_{21} - A_{11}, B_{11} + B_{12})$
 $P_7 := \text{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})$
 $C_{11} := P_1 + P_4 - P_5 + P_7$; $C_{12} := P_3 + P_5$
 $C_{21} := P_2 + P_4$; $C_{22} := P_1 + P_3 - P_2 + P_6$
 $C := [C_{ij}]_{i,j=1,2}$

The usefulness of the Ozaki scheme is also clear at multiple-precision levels owing to the success of Mukunoki et al. in accelerating the float128 precision matrix multiplication[7]. The float128 arithmetic supported by GCC features TD to QD precision performance for addition and multiplication and is likewise expected to be sufficient for this precision range.

The Ozaki scheme is an algorithm that aims to simultaneously accelerate performance and improve accuracy by dividing matrices into more matrices with elements represented by shorter digits. Similarly to the “Split” method used in the error-free transformation technique, this approach leverages on the speed of optimized short-precision matrix multiplication (xGEMM) functions. For a given matrix $A \in \mathbb{R}^{m \times l}$ and $B \in \mathbb{R}^{l \times n}$, to obtain a matrix product $C := AB \in \mathbb{R}^{m \times n}$ of long L -bit precision, A and B are divided using the Ozaki scheme, where $D \in \mathbb{N}$ is the maximum number of divisions of short S -bit precision matrices ($S \ll L$), as depicted in Algorithm 2. The S -bit arithmetic is used for calculations where no particular description is given, and the L -bit arithmetic is used only where high-precision operations are required.

Fig. 1.2 illustrates an example of the Ozaki scheme when A and $B \in \mathbb{R}^{3 \times 3}$ are divided into three short-digit matrices. The most important feature of the Ozaki scheme is that the matrices A and B are

Algorithm 2 Ozaki scheme for multiple-precision matrix multiplication

Input: $A \in \mathbb{F}_{bL}^{m \times l}, B \in \mathbb{F}_{bL}^{l \times n}$
Output: $C \in \mathbb{F}_{bL}^{m \times n}$
 $A^{(S)} := A, B^{(S)} := B : A^{(S)} \in \mathbb{F}_{bS}^{m \times l}, B^{(S)} \in \mathbb{F}_{bS}^{l \times n}$
 $\mathbf{e} := [1 \ 1 \ \dots \ 1]^T \in \mathbb{F}_{bS}^l$
 $\alpha := 1$
while $\alpha < D$ **do**
 $\mu_A := [\max_{1 \leq p \leq l} |(A^{(S)})_{ip}|]_{i=1,2,\dots,m} \in \mathbb{F}_{bS}^m$
 $\mu_B := [\max_{1 \leq q \leq l} |(B^{(S)})_{qj}|]_{j=1,2,\dots,n} \in \mathbb{F}_{bS}^n$
 $\tau_A := [2^{\lceil \log_2((\mu_A)_i) \rceil + \lceil (S + \log_2(l))/2 \rceil}]_{i=1,2,\dots,m} \in \mathbb{F}_{bS}^m$
 $\tau_B := [2^{\lceil \log_2((\mu_B)_j) \rceil + \lceil (S + \log_2(l))/2 \rceil}]_{j=1,2,\dots,n} \in \mathbb{F}_{bS}^n$
 $S_A := \tau_A \mathbf{e}^T$
 $S_B := \mathbf{e} \tau_B^T$
 $A_\alpha := (A^{(S)} + S_A) - S_A : A_\alpha \in \mathbb{F}_{bS}^{m \times l}$
 $B_\alpha := (B^{(S)} + S_B) - S_B : B_\alpha \in \mathbb{F}_{bS}^{l \times n}$
 $A := A - A_\alpha, B := B - B_\alpha : L\text{-bit FP arithmetic}$
 $A^{(S)} := A, B^{(S)} := B$
 $\alpha := \alpha + 1$
end while
 $A_D := A^{(S)}, B_D := B^{(S)}$
 $C := O$
for $\alpha = 1, 2, \dots, D$ **do**
 for $\beta = 1, 2, \dots, D - \alpha + 1$ **do**
 $C_{\alpha\beta} := A_\alpha B_\beta$
 end for
 $C := C + \sum_{\beta=1}^{D-\alpha+1} C_{\alpha\beta} : L\text{-bit FP arithmetic}$
end for

divided into A_1, A_2, A_3, B_1, B_2 , and B_3 to fit within a short precision, thereby avoiding rounding errors in fast, low-precision matrix multiplication. An error-free matrix product $C_{ij} := A_i B_j (i, j = 1, 2, 3)$ can be obtained as a result, and a highly accurate matrix product C can be obtained using a multiple-precision matrix addition operation for $C := \sum_{i,j} C_{ij}$. Although the number of divisions of A and B is finite, it is difficult to determine the minimum number of divisions required to guarantee a certain accuracy threshold. As a result, benchmark tests must be performed to determine if the algorithm can be executed faster than other multiplication algorithms.

1.7 Software layer of BNCmatmul

We have already used QD[1] as multi-component-type fixed precision C++ class library, CAMPARY[3] as multi-component-type arbitrary precision library, and MPFR[8] baased on GNU MP[2]'s MPN kernel. MPLAPACK/MPBLAS[6] is greeding these all reliable MP libraries and providing the coresponding APIs of LAPACK/BLAS[5]. Our BNCmatmul developed aims to get better performance than MPBLAS as ATLAS, OpenBLAS and Intel Math Kernel, and it has the following features:

- Core functions and macros are wriiten in the traditional way of ANSI C, not C++.
- Supporting native multi-component-type DD, TD, and QD precision arithmetic, and also accelelating vector and matrix computation like BLAS with AVX2 and AVX-512 on x86-64, and NEON and SVE2 on AArch64. As of 0.24 the AVX-512 backend covers every kernel the AVX2 backend does and is verified bit-identical to the scalar one (Sec. 1.3).
- Direct use of MPFR functions to avoid overhead due to mpreal class library.
- Supporting partly shared-memory-type parallelization based on OpenMP.
- Implementing three algorithmtms of matrix multiplication: simple triple-loop, blocking (tiling), and Strassen or Winograd algorithm. Ozaki scheme can be used as trial optimization.

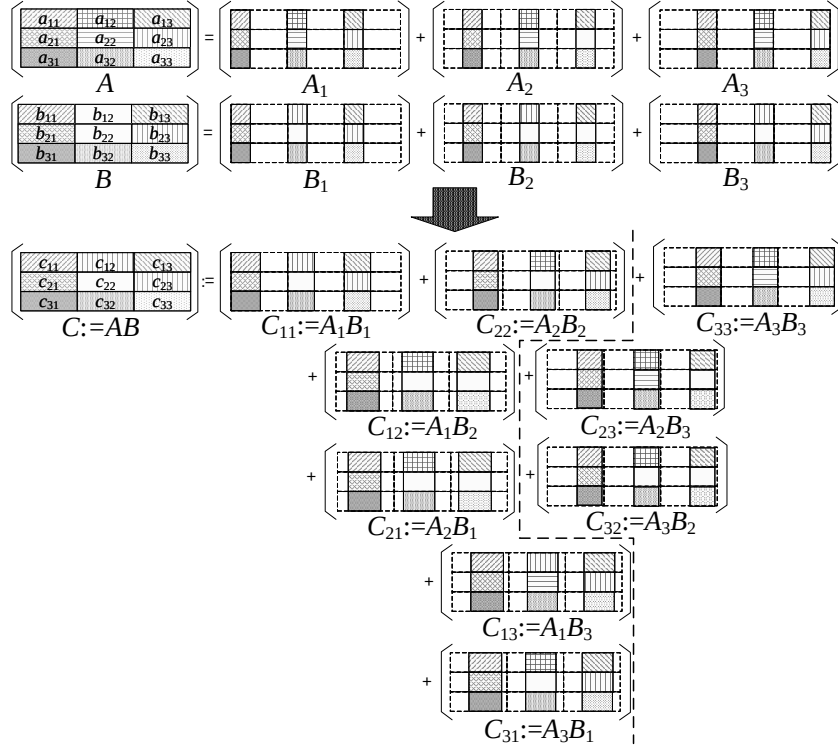


Fig. 1.2 Matrix multiplication based on Ozaki scheme when the matrices are divided into three components

The software layer of BNCmatmul except parallelization is shown as Fig. 1.3.

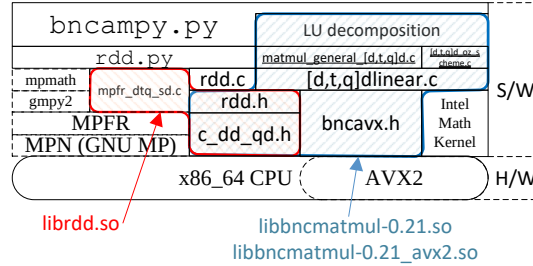


Fig. 1.3 Software layer of BNCmatmul library

Basic DD, TD, and QD precision arithmetic is defined entirely inside `rdd.h`: the `c_dd_*/c_td_*/c_qd_*` C inline functions form its implementation layer, and the `rdd_*/RDD_*` macros and inline functions built on top of them are the interface used throughout the library (up to version 0.23 the inline layer lived in a separate header, `c_dd_qd.h`, which has been absorbed into `rdd.h`). To use the Python environment, `rdd.c`, which is based on `rdd.h`, is compiled as `librdd.so`. The DD, TD, and QD classes on Python use the functions defined in `librdd.so`.

BNCmatmul, which is also built on `rdd.h`, includes block and Strassen matrix multiplications that provide more efficient performance than simple triple-loop matrix multiplication. These have been accelerated using the SIMDized functions defined under `include/avx2/` — `_bncavx_*.h` for AVX2 (`__m256d/``__m256`) and `_bncavx*_avx512.h` for AVX-512 (`__m512d/``__m512`), both reached through the umbrella header `avx2/bncavx.h` — and by their counterparts under `include/neon/` and `include/sve2/`

on AArch64. The LU decomposition has also been accelerated in the same way.

These functions that are defined in BNCmatmul (Fig. 1.3) can be used from libbncmatmul-0.xx.a/.so without any SIMDization techniques, and from libbncmatmul-0.xx_avx2, _avx512, _neon, and _sve2 with the SIMDization of AVX2, AVX-512, Arm NEON and Armv9-A SVE2, respectively. As these libraries have the same function names, so users do not need to modify their codes when selecting the BNCmatmul library — the SIMD backend is chosen at link time only.

1.8 History of Version and Todo list

■History of Version

Version 0.24: 2026-08-15 Certified branch-free FMA and FMA-based elementary functions, ported from dtq-0.0.3 in plain ANSI C:

1. Branch-free fused multiply-add kernels `bnc_{dw,tw,qw}fma[f]` with machine-proved error bounds (`bncfma.h`), including SIMD versions for NEON, SVE2, and — newly — AVX2/AVX-512 (`avx2/_bncavx_fma.h`); the NEON and AVX2 kernels are verified bit-identical to the scalar certified ones (Sec. 2.6)
2. FMA-fused elementary functions `exp`, `expm1`, `log`, `log10`, `sin`, `cos`, `sincos` for DD/TD/QD (DS/TS/QS by delegation), replacing the previous empty stubs and missing implementations of `c_dd_sin`, `c_td_*`, `c_qd_*`, etc.; accuracy validated against MPFR, average error below 1ϵ of each type (Sec. 2.7)
3. Elementwise array functions `bnc_{dd,td,qd}_{exp,log,sin,cos}_array` over limb-planar arrays, blocked by the SIMD width of the build and bitwise identical to the scalar functions (Sec. 2.7)
4. Bug fixes: `const_dd_pi` held $\pi/16$, `const_td_pi/const_qd_pi` held $\pi/1024$, single-based π constants were zero, and `c_dd_nint` was an empty stub
5. div/sqrt-safe FMA variants `bnc_{dw,tw,qw}fma[f]_safe` and the FMA-driven divisions `bnc_{dd,td,qd,ds,ts,qs}_div_fma` (dtq fma_div ports; opt-in default switch `BNC_USE_FMA_DIV`); the double-based divisions run $1.2\times/2.8\times/2.2\times$ (DD/TD/QD) faster at the same accuracy (Sec. 2.7.4)
6. AVX-512 backend brought to parity with AVX2: 411 of the 426 AVX2 constructs in the compiled sources now have a matching AVX-512 arm, the `avx512` build needs no ISA beyond AVX-512F + FMA, and the serial, AVX2 and AVX-512 builds are verified bit-identical on the element-wise and sparse kernels (Sec. 1.3)
7. DLL-ready exported function API `rdd_*/rtd_*/rqd_*/rds_*/rts_*/rqs_*` (99 symbols, `src/rdtq_func.c` + `include/rdtq_func.h`) for Python/ctypes, with the `librdtq` build target and the `python/test_rdtq.py` driver (Sec. 2.8)

Version 0.23: 2026-03-16 BNCmatmul accelerated on Arm Neon CPUs has been released.

Version 0.22: 2024-05-31 Some functions have been appended:

1. Complex arithmetic and linear computation
2. Miscellaneous functions (accessares, elementary and random functions)
3. Sparse matrix and SpMV

Version 0.21: 2023-05-31 Secondly opening sources including DD, TD, QD and MPFR precision real BLAS functions at <https://github.com/tkouya/bncmatmul/blob/main/bncmatmul-0.21.tar.bz2>.

Version 0.20: 2016-06-08 Firstly opening sources at <https://na-inet.jp/na/bnc/bncmatmul-0.2.tar.bz2>, which provides only arbitrary matrix multiplication based on MPFR.

■Todo list

1. Showing more sample sources in this manual using BNCmatmul and MPLAPACK/BLAS
2. Completing descriptions of our library usage and how to apply various situations with it.

Chapter 2

Basic datatypes and arithmetic

In this chapter, we present and describe all basic data types and functions defined in BNCmatmul, and show how to use them in order to construct your programs including multiple-precision computing.

As shown in Chapter 1, all basic datatype and functions are written in ANSI C, not C++ way.

2.1 The `c_dd_*` arithmetic layer (`rdd.h`)

Up to version 0.23 the macros and inline functions in this section were provided by a separate header, `c_dd_qd.h` (and `c_ds_qs.h` for the single-based family). As of 0.24 both headers have been absorbed into `rdd.h` and `rds.h` respectively, which are now the single arithmetic headers of the multi-component families; including `rdd.h` provides everything listed below.

- (Macro) `DFMA(a, b, c)` returns $a \times b + c$ in double precision
- (Macro) `SFMA(a, b, c)` returns $a \times b + c$ in single precision
- (Macro) `QD_FMA(a, b, c)` the same as `DFMA(a, b, c)`
- (Macro) `QD_FMS(a, b, c)` returns $a \times b - c$ in double precision
- (Macro) `DDSIZE` is 2, the number of binary64(double) variables in DD
- (Macro) `TDSIZE` is 3, the number of binary64 variables in TD
- (Macro) `QDSIZE` is 4, the number of binary64 variables in QD

2.1.1 DD (double-double): 106-bit ($53\text{bits} \times 2$) precision floating-point number

- (Macro) `DD_HI(a)` is `a[0]`.
- (Macro) `DD_LOW(a)` is `a[1]`.
- (Macro) `DD_TRUE` is 1UL.
- (Macro) `TD_TRUE`
- (Macro) `QD_TRUE`
- (Macro) `DD_FALSE` is 0UL.
- (Macro) `TD_FALSE`
- (Macro) `QD_FALSE`
- (Macro) `DD_ISNAN(a)` checks if a includes NAN.
- (Macro) `TD_ISNAN(a)`
- (Macro) `QD_ISNAN(a)`
- (Macro) `DD_ISINF(a)` checks if a includes INF.
- (Macro) `TD_ISINF(a)`
- (Macro) `QD_ISINF(a)`
- (Macro) `DD_ISZERO(a)` checks if a is zero.
- (Macro) `TD_ISZERO(a)`
- (Macro) `QD_ISZERO(a)`
- (Macro) `DD_ISONE(a)` checks if a is one.
- (Macro) `TD_ISONE(a)`

(Macro) QD_ISONE(a)
 (Macro) DD_ISNEGATIVE(a) checks if $a < 0$.
 (Macro) TD_ISNEGATIVE(a)
 (Macro) QD_ISNEGATIVE(a)
 (Macro) DD_NAN is FP_NAN.
 (Macro) TD_NAN
 (Macro) QD_NAN

2.2 Error-free transformation

- `double quick_two_sum(double a, double b, double * err) ...` Computes $fl(a + b)$ and $err(a + b)$. Assumes $|a| \geq |b|$.
- `double quick_two_diff(double a, double b, double * err) ...` Computes $fl(a - b)$ and $err(a - b)$. Assumes $|a| \geq |b|$.
- `double two_sum(double a, double b, double * err) ...` Computes $fl(a + b)$ and $err(a + b)$.
- `double two_diff(double a, double b, double * err) ...` Computes $fl(a - b)$ and $err(a - b)$.
- `void split(double a, double *hi, double *lo) ...` divide a to hi and lo without error.
- `double two_prod(double a, double b, double * err) ...` Computes $fl(ab)$ and $err(ab)$.
- `double two_sqr(double a, double * err) ...` Computes $fl(a^2)$ and $err(a^2)$. Faster than the above method.

2.3 Double-double precision arithmetic

- `dd_bool dd_is_zero(const double * x) ...` Check if $*x$ is zero.
- `dd_bool dd_is_one(const double * x) ...` Check if $*x$ is one.
- `void c_dd_set(const double * x, double *ret) ...` $ret := x$
- `void c_dd_set_dd_d(const double x, double * ret) ...` $ret := (double)x$
- `void c_dd_set0(double * ret) ...` $ret := 0$
- `void c_dd_setnan(double * ret) ...` $ret := NaN$
- `void c_dd_neg(const double * a, double * b) ...` $b := -a$
- `dd_bool dd_is_positive(const double * x) ...` $*x > 0$?
- `dd_bool dd_is_negative(const double * x) ...` $*x < 0$?
- `void c_dd_comp(const double * a, const double * b, int * result) ...` Compare with a and b , and store 0 if $a == b$, +1 else if $a > b$, -1 else if $a < b$.
- `void c_dd_comp_dd_d(const double * a, double b, int * result)`
- `void c_dd_comp_d_dd(double a, const double * b, int * result)`
- `void c_dd_pi(double * a)`
- `void c_d_add(double a, double b, double * c) ...` double-double := double + double
- `void c_dd_add(const double * a, const double * b, double * c) ...` $c := a + b$
- `void c_dd_add_sloppy(const double * a, const double * b, double * c)`
- `void c_dd_add_dd_d(const double * a, double b, double * c)`
- `void c_dd_add_d_dd(double a, const double * b, double * c)`
- `void c_d_sub(double a, double b, double * c) ...` double-double = double - double
- `void c_dd_sub(const double * a, const double * b, double * c) ...` $c := a - b$
- `void c_dd_sub_sloppy(const double * a, const double * b, double * c)`
- `void c_dd_sub_dd_d(const double * a, double b, double * c)`
- `void c_dd_sub_d_dd(double a, const double * b, double * c)`
- `void c_d_mul(double a, double b, double * c) ...` double-double := double * double
- `void c_dd_mul(const double * a, const double * b, double * c)`
- `void c_dd_mul_dd_d(const double * a, double b, double * c)`
- `void c_dd_mul_d_dd(double a, const double * b, double * c)`

- void c_d_div(double a, double b, double * c) ... $c := a/b$
- void c_dd_div(const double * a, const double * b, double * c) ... $c := a/b$
- void c_dd_sloppy_div(double * a, double * b, double * c)
- void c_dd_div_dd_d(const double * a, double b, double * c)
- void c_dd_div_d_dd(double a, const double * b, double * c)
- void c_dd_copy(const double * a, double * b) $b := a$
- void c_dd_copy_d(double a, double * b)
- void c_d_sqr(double a, double * b) ... $b := a^2$
- void c_dd_sqr_d(double a, double * ret)
- void c_dd_sqr(const double * a, double * b)
- void c_dd_sqrt(const double * a, double * b) ... $b := \sqrt{a}$
- void c_dd_abs(const double * a, double * b) ... $b := |a|$
- void c_dd_floor(const double * a, double * b) ... $b := \text{floor}(a)$
- void c_dd_ceil(const double * a, double * b) ... $b := \text{ceil}(a)$

2.4 Quadruple-double precision arithmetic

- void c_qd_copy(const double * a, double * b) ... $b := a$
- void c_qd_copy_dd(const double * a, double * b)
- void c_qd_copy_d(double a, double * b)
- void quick_renorm(double * c0, double * c1, double * c2, double * c3, double * c4) ... Renormalize c0 ...
- void renorm(double * c0, double * c1, double * c2, double * c3)
- void renorm4(double * c0, double * c1, double * c2, double * c3, double * c4)
- void three_sum(double * a, double * b, double * c)
- void three_sum2(double * a, double * b, double * c)
- void c_qd_add_qd_dd(const double * a, const double * b, double * c) ... $c := a + b$
- void c_qd_add(const double * a, const double * b, double * c)
- void c_qd_add_sloppy(const double * a, const double * b, double * c) ... not so accurate but fast addition $c := a + b$
- void c_td_addq(const double * a, const double * b, double * c) ... triple double addition based on quad-double way
- void c_qd_selfadd(const double * a, double * b) ... $b := b + a$
- void c_qd_selfadd_dd(const double * a, double * b)
- void c_qd_selfadd_d(double a, double * b)
- void c_qd_neg(const double * a, double * b) ... $b := -a$
- void c_qd_neg_dd(const double * a, double * b)
- void c_qd_neg_d(const double a, double * b)
- void c_qd_sub(const double * a, const double * b, double * c) ... $c := a - b$
- void c_qd_sub_qd_dd(const double * a, const double * b, double * c)
- void c_qd_sub_dd_qd(const double * a, const double * b, double * c)
- void c_qd_sub_qd_d(const double * a, double b, double * c)
- void c_qd_sub_d_qd(double a, const double * b, double * c)
- void c_qd_selfsub(const double * a, double * b) ... $b := b + (-a) = b - a$
- void c_qd_selfsub_dd(const double * a, double * b)
- void c_qd_selfsub_d(double a, double * b)
- void c_qd_mul(const double * a, const double * b, double * c) ... $c := a \times b$
- void c_qd_mul_qd_dd(const double * a, const double * b, double * c)
- void c_qd_mul_dd_qd(const double * a, const double * b, double * c)
- void c_qd_mul_qd_d(const double * a, double b, double * c)
- void c_qd_mul_d_qd(double a, const double * b, double * c)
- void c_qd_mul_sloppy(const double * a, const double * b, double * c)

- void c_qd_sqr(const double * a, double * c) ... $c = a^2$
- void c_qd_selfmul(const double * a, double * b) ... $b := a * b$
- void c_qd_selfmul_dd(const double * a, double * b)
- void c_qd_selfmul_d(double a, double * b)
- void c_qd_div_accurate(const double * a, const double * b, double * c) ... $c := a/b$
- void c_qd_div_sloppy(const double * a, const double * b, double * c) ... not so accurate but fast division (default)

(Macro) USE_QD_DIV_ACCURATE() ... use accurate qd division if true

- void c_qd_div_qd_dd(const double * a, const double * b, double * c)
- void c_qd_div_qd_d(const double * a, double b, double * c)
- void c_qd_div_d_qd(double a, const double * b, double * c)
- void c_qd_selfdiv(const double * a, double * b) ... $b := b/a$
- void c_qd_selfdiv_dd(const double * a, double * b)
- void c_qd_selfdiv_d(double a, double * b)
- void c_qd_mul_pwr2(const double * a, double b, double * c) ... $c := (a[0]*b, a[1]*b, a[2]*b, a[3]*b)$
- void c_qd_set(const double * x, double * qdval) ... $qdval := c$
- void c_qd_set0double * qdval() ... $qdval := 0$
- void c_qd_sqrt(const double * a, double * b) ... $b := \sqrt{a}$
- void c_qd_abs(const double * a, double * b) ... $b := |a|$

(Macro) nint(a) ... round(a) as integer

- void c_qd_nint(const double * a, double * b)
- void c_qd_floor(const double * a, double * b) ... $b := \text{floor}(a)$
- void c_qd_ceil(const double * a, double * b) ... $b := \text{ceil}(a)$
- void c_qd_comp(const double * a, const double * b, int * result) ... return -1 if $a < b$, 0 if $a == b$, and +1 if $a > b$.
- void c_qd_comp_qd_d(const double * a, double b, int * result)
- void c_qd_comp_d_qd(double a, const double * b, int * result)

2.5 Triple-double precision arithmetic

- void c_td_copy(const double * a, double * b) ... $b := a$
- void c_qd_copy_td(const double * a, double * c)
- void c_td_copy_qd(const double * a, double * b)
- void c_td_copy_dd(const double * a, double * b)
- void c_td_copy_d(double a, double * b)
- void vec_sum(double * e, const double * x, int n) ... $e[n] := \text{vec_sum}(x[n])$
- void vseb(double * y, int ny, const double * e, int ne) ... $y[n] := \text{vec_sum_err_branch}(vseb)(k)(e[n])$
- void c_to_td(double * r, double a, double b, double c) ... $r[3] := \text{to_td}(a, b, c)$
- void merge(double * c, double * a, int na, double * b, int nb) ... Merge $a[na]$ and $b[nb]$ into $c[na + nb]$
- void c_td_add(double * a, double * b, double * c) ... $c := a + b$
- void c_td_add_td_d(double * a, double b, double * c)
- void c_td_neg(const double * a, double * c) ... $c := -a$
- //
- void c_td_sub(double * c, double * a, double * b) ... $c := a - b$
- void c_td_sub(double * a, double * b, double * c)
- void c_td_subq(const double * a, const double * b, double * c)
- void c_td_sub_d_td(double a, double * b, double * c) //
- void c_td_sub_td_d(double * c, double * a, double b)
- void c_td_sub_td_d(double * a, double b, double * c)
- void c_td_mul_accurate(double * a, double * b, double * c) ... $c := a \times b$
- void c_td_mul_sloppy(double * a, double * b, double * c) ... not so accurate but fast $c := a \times b$

- (Macro) `USE_TD_MUL_ACCURATE()` ... Use accurate td multiplication if true
 - `void c_td_mul_dd_td_sloppy(double * a, double * b, double * c)`
 - `void c_td_mul_dd_td_accurate(double * a, double * b, double * c)`
- (Macro) `USE_TD_MUL_DD_TD_ACCURATE()` ... Use accurate td-dd multiplication if true
 - `void c_td_mul_d_td(const double a, const double * b, double * c)`
 - `void c_td_mul_td_d(const double * a, const double b, double * c)`
 - `void c_td_abs(const double * a, double * b) ... b := |a|`
 - `void c_td_comp(const double * a, const double * b, int * result) ... return -1 if  $a < b$ , 0 if  $a == b$ , and +1 if  $a > b$ .`
 - `void c_td_comp_td_d(const double * a, double b, int * result)`
 - `void c_td_comp_d_td(double a, const double * b, int * result)`
 - `void c_td_2mtw_dd_td(double a[TDSIZE], double b[TDSIZE], double c[TDSIZE])`
- (Macro) `ONE_P_2DBL_EPS()` ... $1 + 2 \times \text{DBL_EPSILON} = (1.00000000000000044e + 00)$
- (Macro) `ONE_M_2DBL_EPS()` ... $1 - 2 \times \text{DBL_EPSILON} = (9.99999999999999556e - 01)$
 - `void c_td_reciprocal(double * a, double * c)`
 - `void c_td_divt(double * a, double * b, double * c)`
- (Macro) `c_td_div()` ... the same as `c_td_divtq`
 - `void c_td_divq(const double * a, const double * b, double * c) ... td division based on quad-double division.`
 - `void c_td_div_td_d(double * a, double b, double * c)`
 - `void c_td_sqrt(double * a, double * c) ...  $c := \sqrt{a}$`
 - `void c_td_sqrt_d(double a, double * c)`
 - `void c_td_sqr(double * a, double * c) ...  $c := a^2$`

2.6 Branch-free fused multiply-add (`bncfma.h`)

Version 0.24 provides certified branch-free fused multiply-add (FMA) kernels $z := x \times y + c$ for DD/T-D/QD (double-based) and DS/TS/QS (single-based) arithmetic, ported operation-by-operation from the reference implementation of T. Kouya, “Performance evaluation of branch-free fused multiply-add algorithms for multi-component-type multiple-precision floating-point arithmetic” (arXiv:2607.11391). Each kernel matches an FPANVerifier-certified netlist and carries a machine-proved error bound ($u = 2^{-p}$):

	flops	certified error bound
DW (DD/DS)	17	$ z - (xy + c) \leq 34u^2(xy + c)$
TW (TD/TS)	66	$ z - (xy + c) \leq 184u^3(xy + c)$
QW (QD/QS)	146	$ z - (xy + c) \leq 812u^4(xy + c)$

- `void bnc_dwfmaw(double z[2], const double x[2], const double y[2], const double c[2]) ...  $z := x \times y + c$  (DD)`
- `void bnc_twfmaw(double z[3], const double x[3], const double y[3], const double c[3]) ...  $z := x \times y + c$  (TD)`
- `void bnc_qwfmaw(double z[4], const double x[4], const double y[4], const double c[4]) ...  $z := x \times y + c$  (QD)`
- `void bnc_dwfmawf(float z[2], const float x[2], const float y[2], const float c[2]) ...  $z := x \times y + c$  (DS)`
- `void bnc_twfmawf(float z[3], const float x[3], const float y[3], const float c[3]) ...  $z := x \times y + c$  (TS)`
- `void bnc_qwfmawf(float z[4], const float x[4], const float y[4], const float c[4]) ...  $z := x \times y + c$  (QS)`

The scalar kernels live in `bncfma_d.h` / `bncfma_f.h` (umbrella header `bncfma.h`). SIMD-vectorized versions with identical semantics are provided for every backend: `neon/_bncneon_fma.h` (`_bncneon_{dw,tw,qw}fma[f]`), `float64x2_t/float32x4_t`, `sve2/_bncsve2_fma.h`, and — new in

0.24 — `avx2/_bncavx_fma.h` with both AVX2 (`_bncavx2_{dw,tw,qw}fma[f]`, `__m256d/_m256`) and AVX-512 (`_bncavx512_{dw,tw,qw}fma[f]`, `__m512d/_m512`) variants. The NEON and AVX2 kernels have been verified bit-identical to the scalar certified kernels over random operands; the AVX-512 build is covered at the kernel level instead, by the whole-library bit-identity check of Sec. 1.3. All of them require `-ffp-contract=off` (set by every BNCmatmul build): if the compiler contracts $a \times b + c$ on its own, the error-free transformations inside stop being error-free.

■ **div/sqrt-safe variants** The certified bounds above are machine-proved for operands that are normalized (non-overlapping) expansions. The Newton iterations of division and square root feed residuals that are *not* non-overlapping, so no FastTwoSum (`quick_two_sum`) precondition can be asserted for any gate — a FastTwoSum whose precondition fails does not even satisfy $s + e = a + b$. For such operands the *safe* variants (ports of dtq’s `dw_fma_safe` / `tw_fma_safe` / `qw_fma_safe`, with a plain scalar multiplicand) demote every FastTwoSum to a full TwoSum (DW: 20 flops); a safe variant is never cheaper, and never less accurate, than the standard one. They are consumed by the FMA-driven divisions (Sec. 2.7).

- `void bnc_dwfma_safe(double z[2], const double x[2], double y, const double c[2]) ...  $z := x \times y + c$` (DD, scalar y)
- `void bnc_twfma_safe(double z[3], const double x[3], double y, const double c[3]) ...` (TD)
- `void bnc_qwfma_safe(double z[4], const double x[4], double y, const double c[4]) ...` (QD)
- `void bnc_dwfmaf_safe(float z[2], const float x[2], float y, const float c[2]) ...` (DS)
- `void bnc_twfmaf_safe(float z[3], const float x[3], float y, const float c[3]) ...` (TS)
- `void bnc_qwfmaf_safe(float z[4], const float x[4], float y, const float c[4]) ...` (QS)

Defining `BNC_USE_NEW_FMA` at library build time switches `rdd_fma/rtd_fma/rqd_fma` (and the `rds/rts/rqs` counterparts) plus the AXPY / dot product / GEMV / GEMM / polynomial-evaluation call sites in the serial, NEON and SVE2 paths to these kernels; without the macro the library is bit-for-bit unchanged.

2.7 FMA-based elementary functions (`bncelem.h`)

Version 0.24 ports the FMA-fused elementary functions of dtq-0.0.3 to plain ANSI C: proper argument reduction ($\ln 2$ reduction with repeated squaring or two-level exp tables; $2\pi \rightarrow \pi/2 \rightarrow \pi/16$ or $\pi/1024$ reduction with 256-entry tables for trigonometric functions) followed by Taylor kernels whose terms are each accumulated with one certified branch-free FMA (Sec. 2.6) instead of a separate multiply and add. The coefficient tables (`bncelem_tables.h`) are extracted mechanically from the dtq-0.0.3 sources; the TD tables reproduce dtq’s `to_td_real()` folding of the QD tables exactly. Accuracy has been validated against MPFR (512-bit) references: the average relative error is below 1ε of each type ($\varepsilon_{DD} = 2^{-104}$, $\varepsilon_{TD} = 2^{-157}$, $\varepsilon_{QD} = 2^{-209}$), equivalent to dtq itself.

2.7.1 Scalar functions

For each double-based type the following functions are provided (`include/bncelem_dd.h` / `_td.h` / `_qd.h`; umbrella header `bncelem.h`). The prefix is `bnc_dd_`, `bnc_td_` or `bnc_qd_` and the limb count K is 2, 3 or 4:

- `void bnc_dd_exp(const double * x, double * ret) ...  $ret := \exp(x)$`
- `void bnc_dd_expm1(const double * x, double * ret) ...  $ret := \exp(x) - 1$` (accurate for small $|x|$)
- `void bnc_dd_log(const double * x, double * ret) ...  $ret := \log_e(x)$` (one Newton step on exp)
- `void bnc_dd_log10(const double * x, double * ret) ...  $ret := \log_{10}(x)$`
- `void bnc_dd_sin(const double * x, double * ret) ...  $ret := \sin(x)$`
- `void bnc_dd_cos(const double * x, double * ret) ...  $ret := \cos(x)$`
- `void bnc_dd_sincos(const double * x, double * sin_x, double * cos_x) ...` computes $\sin(x)$ and $\cos(x)$ simultaneously
- `void bnc_dd_nint(const double * x, double * ret) ...` nearest integer

The same set exists as `bnc_td_*` and `bnc_qd_*` with 3- and 4-limb arrays. The single-based types delegate to the double-based implementations exactly as in `dtq (bncelem_f.h)`: `bnc_ds_*` $\rightarrow$ DD, `bnc_ts_*` $\rightarrow$ TD, `bnc_qs_*` $\rightarrow$ QD, with exact limb conversions in both directions.

2.7.2 Wiring into the `c_dd_*` / `c_td_*` / `c_qd_*` interfaces

Up to version 0.23 most of the elementary functions declared in `c_dd_qd.h` and `c_ds_qs.h` were *empty stubs* whose bodies were disabled by `#if 0` (`c_dd_sin`, `c_dd_cos`, `c_dd_tan`, `c_dd_sincos`, `c_dd_nint`, all `c_qd_*` and `c_qs_*/c_ds_*` elementary functions), and the `c_td_*/c_ts_*` elementary functions referenced from `rdd.h/rds.h` did not exist at all. As of 0.24 every one of them delegates to the corresponding `bnc_*` implementation. The previous working implementations are kept for comparison purposes under the names `c_dd_exp_orig` (plain Taylor series without argument reduction), `c_dd_log_orig` and `c_dd_log10_orig`; the new `c_dd_exp` is about $10\times$ faster than `c_dd_exp_orig` and `c_dd_log` about $5.7\times$ faster than `c_dd_log_orig` (Arm Cortex-X925, gcc 13.3).

Version 0.24 also fixes wrong constants that the stubs concealed: `const_dd_pi` held $\pi/16$, `const_td_pi` and `const_qd_pi` held $\pi/1024$, and the single-based `const_ds_pi/const_ts_pi/const_qs_pi` were all zero; `c_dd_pi()` etc. now return correctly rounded limb values of π .

2.7.3 Elementwise array functions (`bncelem_vector.h`)

For applying an elementary function to every element of a vector, `src/elem_vector.c` provides array versions operating on limb-planar (SoA) arrays — the same layout as `ddvector/tdvector/qdvector`, so `vec->element` can be passed directly. The array is walked in blocks of `_BNC_D_WIDTH` elements, which is 8 in an AVX-512 build, 4 in an AVX2 build, 2 under NEON/SVE2 and 1 otherwise, so the limb gather/scatter around each element vectorizes and the staging buffers stay in registers. The per-element kernels themselves remain the scalar `bnc_dd_*/bnc_td_*/bnc_qd_*` functions: unlike the $+$, $-$, $\times$, $\div$ error-free-transformation kernels, their argument reduction, table lookups and Newton corrections are branchy and have no lane-uniform SIMD form. The results are therefore bitwise identical to calling the scalar function on each element, which `python/test_rdtq.py` checks.

- `void bnc_dd_exp_array(long n, double * const ret[2], double * const x[2]) ...  $ret_i := \exp(x_i)$ ,  $i = 0, \dots, n-1$`
- `void bnc_dd_log_array(long n, double * const ret[2], double * const x[2])`
- `void bnc_dd_sin_array(long n, double * const ret[2], double * const x[2])`
- `void bnc_dd_cos_array(long n, double * const ret[2], double * const x[2])`
- `void bnc_td_exp_array(long n, double * const ret[3], double * const x[3])` (likewise `log/sin/cos`)
- `void bnc_qd_exp_array(long n, double * const ret[4], double * const x[4])` (likewise `log/sin/cos`)

2.7.4 FMA-driven division

The divisions of `dtq-0.0.3 (fma_div)` are also ported: the same long-division correction sequence as the classic accurate division, but each residual $r \leftarrow r - q \cdot b$ is one fused safe FMA (Sec. 2.6) instead of a multiply followed by a subtraction, which removes one renormalization per step. Measured against MPFR (512-bit) references the accuracy class is unchanged ($\leq 0.5\varepsilon$ for DD/TD/QD, $\leq 2.4\varepsilon$ for DS/TS/QS) while the double-based divisions run $1.2\times$ (DD), $2.8\times$ (TD) and $2.2\times$ (QD) faster than `c_dd_div / c_td_divq / c_qd_div_sloppy` on Arm Cortex-X925 (see `bench/fma/test_fma_safe.c` and `bench/fma/results/test_fma_safe.txt`).

- `void bnc_dd_div_fma(const double * a, const double * b, double * ret) ...  $ret := a/b$  (DD)`
- `void bnc_td_div_fma(const double * a, const double * b, double * ret) ... (TD)`
- `void bnc_qd_div_fma(const double * a, const double * b, double * ret) ... (QD)`
- `void bnc_ds_div_fma(const float * a, const float * b, float * ret) ... (DS; likewise bnc_ts_div_fma / bnc_qs_div_fma)`

Defining `BNC_USE_FMA_DIV` at build time reroutes the default division entry points `c_dd_div`, `c_td_div`, `c_qd_div`, `c_ds_div`, `c_ts_div`, `c_qs_div` (and the `RTD_DIV` / `RTS_DIV` macros of `rdd.h` / `rds.h`) to these functions; the previous DD/DS bodies remain available as `c_dd_div_orig` / `c_ds_div_orig`. Without the macro the library is bit-for-bit unchanged. Note that `dtq` itself calls the safe FMA only in its double-based `fma_div`; the BNCmatmul float-based ports use the safe variants as well, following the rationale `dtq` documents for the double-based case.

2.7.5 Accuracy and performance benchmarks

Accuracy and performance benchmarks (against MPFR references) are provided in `bench/elem/`; the measured results of the porting session are stored under `bench/elem/results/`.

2.8 Exported function API for DLL/Python (`rdtq_func.h`)

The `rdd_*` / `rtd_*` / `rqd_*` (and `rds_*` / `rts_*` / `rqs_*`) interfaces of `rdd.h` / `rds.h` are macros and static-inline functions, which exist only inside a C translation unit and export no symbols. For shared-library (DLL) consumers such as Python/ctypes, `src/rdtq_func.c` compiles real external symbols that alias the `dtq-0.0.3` ports one-to-one (99 functions; prototypes in `include/rdtq_func.h`, with a `BNC_API` decoration macro prepared for Windows `__declspec(dllexport)`). The argument order follows the `rdd.h` convention: result first.

For each type $XX \in \{\text{rdd}, \text{rtd}, \text{rqd}\}$ (double limbs) and $\{\text{rds}, \text{rts}, \text{rqs}\}$ (float limbs) the following are exported:

- `void rdd_fma(double * ret, const double * a, const double * b, const double * c) ...  $ret := a \times b + c$` (certified branch-free FMA)
- `void rdd_fma_safe(double * ret, const double * a, double b, const double * c) ...  $div/\sqrt{\text{safe}}$` FMA (scalar b)
- `void rdd_div_fma(double * ret, const double * a, const double * b) ...  $ret := a/b$`
- `void rdd_exp(double * ret, const double * x) ...` likewise `expm1`, `log`, `log10`, `sin`, `cos`, `tan`
- `void rdd_sincos(double * sin_ret, double * cos_ret, const double * x)`
- `void rdd_nint(double * ret, const double * x)` (double-based types only)

The SIMD-vectorized array functions `bnc_XX_yy_array` (Sec. 2.7) are already real symbols and equally callable from Python. A shared library for Python is built by the legacy targets `make -C src -f Makefile.legacy librdtq` (or `librdtq_neon`), producing `python/librdtq.so` from `rdtq_func.c` + `elem_vector.c`; `python/test_rdtq.py` is a ctypes test driver covering all six precision families and the array functions. Note for C users: do not include `rdtq_func.h` together with `rdd.h` / `rds.h` unless `USE_RDD_FUNCTIONS` etc. are defined first, since the lowercase macro aliases there would mangle the prototypes (the header enforces this with `#error`).

2.9 `rdd.h`

```
// ddfloat, tdfloat, qdfloat
typedef struct { double val[DDSIZE]; } ddfloat; // 53 * 2 = 106
typedef struct { double val[TDSIZE]; } tdfloat; // 53 * 3 = 159
typedef struct { double val[QDSIZE]; } qdfloat; // 53 * 4 = 212
```

(Macro) `SET0_DD(val) ...  $val := 0$   $val[0] = (\text{double})0.0$ ;  $val[1] = (\text{double})0.0$ ;`

(Macro) `SET0_TD(val)`

(Macro) `SET0_QD(val)`

- `void rdd_out_str_base(FILE * fp, int base, int length, double val[DDSIZE]) ...` DD print(no appending CR)
- `int rdd_cmp(double a[DDSIZE], double b[DDSIZE]) ...` return -1 if $a < b$, 0 if $a == b$, and -1 if $a < b$.

- `int rdd_cmp_d(double a[DDSIZE], double b)`
- `void rdd_sqrt_d(double ret[DDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rdd_fma(double ret[DDSIZE], double a[DDSIZE], double b[DDSIZE], double c[DDSIZE]) ...`
 $ret := a \times b + c$
- `void rdd_pow(double ret[DDSIZE], double base[DDSIZE], double power[DDSIZE]) ... ret :=`
 $base^{power}$
- `void rqd_out_str_base(FILE *fp, int base, int length, double val[QDSIZE]) ... print(no append-`
`ing CR)`
- `int rqd_cmp(double a[QDSIZE], double b[QDSIZE]) ... return -1 if  $a < b$ , 0 if  $a == b$ , and -1 if`
 $a < b$.
- `int rqd_cmp_d(double a[QDSIZE], double b)`
- `void rqd_sqrt_d(double ret[QDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rqd_fma(double ret[QDSIZE], double a[QDSIZE], double b[QDSIZE], double c[QDSIZE]) ...`
 $ret := a \times b + c$
- `void rqd_pow(double ret[QDSIZE], double base[QDSIZE], double power[QDSIZE]) ...`
 $ret := base^{power}$
- `void rtd_out_str_base(FILE *fp, int base, int length, double val[TDSIZE])`
- `int rtd_cmp(double a[TDSIZE], double b[TDSIZE]) ... return -1 if  $a < b$ , 0 if  $a == b$ , and -1 if`
 $a < b$.
- `int rtd_cmp_d(double a[TDSIZE], double b)`
- `void rtd_sqrt_d(double ret[TDSIZE], double a) ... return  $\sqrt{ret}$`
- `void rtd_fma(double ret[TDSIZE], double a[TDSIZE], double b[TDSIZE], double c[TDSIZE]) ...`
 $ret := a \times b + c$
- `void rtd_pow(double ret[TDSIZE], double base[TDSIZE], double power[TDSIZE]) ...`
 $ret := base^{power}$

(Macro) `set0_dd(val) val := 0`

(Macro) `rdd_set0(val)`

(Macro) `rdd_add(ret, a, b) ret :=  $a + b$`

(Macro) `rdd_sub(ret, a, b) ret :=  $a - b$`

(Macro) `rdd_mul(ret, a, b) ret :=  $a \times b$`

(Macro) `rdd_div(ret, a, b) ret :=  $a/b$`

(Macro) `rdd_sqrt(ret, a) ret :=  $\sqrt{a}$`

(Macro) `rdd_sqrt_d(ret, a)`

(Macro) `rdd_sqrt_ui(ret, a)`

(Macro) `rdd_get_d(a)`

(Macro) `rdd_set_d(ret, d) ret :=  $d$`

(Macro) `rdd_set_ui(ret, d)`

(Macro) `rdd_set(ret, d)`

(Macro) `rdd_neg(ret, a) ret :=  $-a$`

(Macro) `rdd_abs(ret, a) ret :=  $|a|$`

(Macro) `rdd_cmp_ui(a, b) return -1 if  $a > b$ , 0 if  $a == b$ , and -1 if  $a < b$`

(Macro) `rdd_ui_div(ret, a, b) ret :=  $a/b$`

(Macro) `rdd_ui_sub(ret, a, b) ret :=  $a - b$`

(Macro) `rdd_div_d(ret, a, b) ret :=  $a/b$`

(Macro) `rdd_add_d(ret, a, b) ret :=  $a + b$`

(Macro) `rdd_sub_d(ret, a, b) ret :=  $a - b$`

(Macro) `rdd_mul_d(ret, a, b) ret :=  $a \times b$`

(Macro) `rdd_div_ui(ret, a, b) ret :=  $a/b$`

(Macro) `rdd_add_ui(ret, a, b) ret :=  $a + b$`

(Macro) `rdd_sub_ui(ret, a, b) ret :=  $a - b$`

(Macro) `rdd_mul_ui(ret, a, b) ret :=  $a \times b$`

(Macro) `set0_td(val) SET0_TD(val)`

```

(Macro) rtd_set0(val) SET0_TD(val)
(Macro) rtd_add(ret, a, b) ret :=  $a + b$ 
(Macro) rtd_addt(ret, a, b)
(Macro) rtd_addq(ret, a, b)
(Macro) rtd_sub(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_subt(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_subq(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_mul(ret, a, b) ret :=  $a \times b$ 
(Macro) rtd_divt(ret, a, b) ret :=  $a/b$ 
(Macro) rtd_divtq(ret, a, b)
(Macro) rtd_divq(ret, a, b)
(Macro) rtd_div(ret, a, b)
(Macro) rtd_sqrt(ret, a) ret :=  $\sqrt{a}$ 
(Macro) rtd_sqrt_d(ret, a)
(Macro) rtd_sqrt_ui(ret, a)
(Macro) rtd_get_d(a) return (double)a
(Macro) rtd_set_d(ret, d) ret :=  $d$ 
(Macro) rtd_set_ui(ret, d)
(Macro) rtd_set(ret, d)
(Macro) rtd_neg(ret, a) ret :=  $-a$ 
(Macro) rtd_abs(ret, a) ret :=  $|a|$ 
(Macro) rtd_cmp_ui(a, b) return  $-1$  if  $a > b$ ,  $0$  if  $a == b$ , and  $-1$  if  $a < b$ 
(Macro) rtd_ui_div(ret, a, b) ret :=  $a/b$ 
(Macro) rtd_ui_sub(ret, a, b) ret :=  $a - b$ 
(Macro) rtd_div_d(ret, a, b) ... c :=  $a/b$ 
(Macro) rtd_add_d(ret, a, b) ... c :=  $a + b$ 
(Macro) rtd_sub_d(ret, a, b) ... c :=  $a - b$ 
(Macro) rtd_mul_d(ret, a, b) ... c :=  $a \times b$ 
(Macro) rtd_div_ui(ret, a, b) ... c :=  $a/b$ 
(Macro) rtd_add_ui(ret, a, b) ... c :=  $a + b$ 
(Macro) rtd_sub_ui(ret, a, b) ... c :=  $a - b$ 
(Macro) rtd_mul_ui(ret, a, b) ... c :=  $a \times b$ 
(Macro) set0_qd(val) ... val :=  $0$ 
(Macro) rqd_set0(val)
(Macro) rqd_add(ret, a, b) ... c :=  $a + b$ 
(Macro) rqd_sub(ret, a, b) ... c :=  $a - b$ 
(Macro) rqd_mul(ret, a, b) ... c :=  $a \times b$ 
(Macro) rqd_div(ret, a, b) ... c :=  $a/b$ 
(Macro) rqd_sqrt(ret, a) ... ret :=  $\sqrt{a}$ 
(Macro) rqd_sqrt_d(ret, a) ret :=  $\sqrt{a}$ 
(Macro) rqd_sqrt_ui(ret, a)
(Macro) rqd_get_d(a) return (double)a
(Macro) rqd_set_d(ret, d) ret :=  $d$ 
(Macro) rqd_set_ui(ret, d)
(Macro) rqd_set(ret, d)
(Macro) rqd_neg(ret, a) ret :=  $-a$ 
(Macro) rqd_abs(ret, a) ret :=  $|a|$ 
(Macro) rqd_cmp_ui(a, b) return  $-1$  if  $a > b$ ,  $0$  if  $a == b$ , and  $-1$  if  $a < b$ 
(Macro) rqd_ui_div(ret, a, b) ret :=  $a/b$ 
(Macro) rqd_ui_sub(ret, a, b) ret :=  $a - b$ 
(Macro) rqd_div_d(ret, a, b) ... c :=  $a/b$ 
(Macro) rqd_add_d(ret, a, b) ... c :=  $a + b$ 
(Macro) rqd_sub_d(ret, a, b) ... c :=  $a - b$ 

```

(Macro) `rqd_mul_d(ret, a, b) ... c := a × b`
(Macro) `rqd_div_ui(ret, a, b) ... c := a/b`
(Macro) `rqd_add_ui(ret, a, b) ... c := a + b`
(Macro) `rqd_sub_ui(ret, a, b) ... c := a - b`
(Macro) `rqd_mul_ui(ret, a, b) ... c := a × b`

2.10 AVX2

2.10.1 float

- `__m256 _bncavx2_fabsf(__m256 val8) ... val8 := max val8, 0 - val8`
- `__m256 _bncavx2_fneg(__m256 a) ... return -a`
- `void _bncavx2_ffma(float ret[], float a[], float b[], float c[], int dim) ... ret := a × b + c`
- `void _bncavx2_fmulp(float ret[], float a[], float b[], int dim) ... ret := a × b`

2.10.2 double

- `__m256d _bncavx2_fabs(__m256d val4) ... val4 := max val4, -val4`
- `__m256d _bncavx2_dneg(__m256d a) ... -a`
- `void _bncavx2_dfma(double ret[], double a[], double b[], double c[], int dim) ... ret := a × b + c`
- `void _bncavx2_dmul(double ret[], double a[], double b[], int dim) ... ret := a × b`
- `void _bncavx2_ddiv(double ret[], double a[], double b[], int dim) ... ret := a/b`
- `void _bncavx2_dadd(double ret[], double a[], double b[], int dim) ... ret := a + b`
- `void _bncavx2_dsub(double ret[], double a[], double b[], int dim) ... ret := a - b`
- `double _bncavx2_ddotp(double a[], double b[], int dim) ... ret :=  $\sum_{i=0}^{\text{dim}-1} a[i] \times b[i]$`

2.10.3 Error-free transformation

- `__m256d _bncavx2_dquick_two_sum(__m256d a, __m256d b, __m256d * err) ... Computes fl(a + b) and err(a + b). Assume |a| ≥ |b|.`
- `__m256d _bncavx2_dquick_two_diff(__m256d a, __m256d b, __m256d * err) ... Computes fl(a - b) and err(a - b). Assume |a| ≥ |b|.`
- `__m256d _bncavx2_dtwo_sum(__m256d a, __m256d b, __m256d * err) ... Computes fl(a + b) and err(a + b).`
- `__m256d _bncavx2_dtwo_diff(__m256d a, __m256d b, __m256d * err) ... Computes fl(a - b) and err(a - b).`
- `__m256d _bncavx2_dtwo_prod(__m256d a, __m256d b, __m256d * err) ... Computes fl(a × b) and err(a × b).`

2.10.4 Double-double precision arithmetic

- `void _bncavx2_set0_dd(__m256d) ret[DDSIZE] ... ret := 0`
- (Macro) `_bncavx2_rdd_set0(ret) ... is the same as _bncavx2_set0_dd(ret)`
- `void _bncavx2_get_dd_m256d_i(ddfloat *ret, __m256d ret4[DDSIZE], int avx_index) ... ret := ret4[avx_index]`
- `void _bncavx2_rdd_sum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := ret4[0] + ret4[1] + ret4[2] + ret4[3]`
- `void _bncavx2_rdd_abssum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|`
- `void _bncavx2_rdd_absmax256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret := max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)`

- `void _bncavx2_rdd_norm256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) ... ret :=  $\|ret4[0]^2 + ret4[1]^2 + ret4[2]^2 + ret4[3]^2\|_2$`
- `void _bncavx2_rdd_add(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret :=  $a + b$`
- `void _bncavx2_rdd_sub(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret :=  $a - b$`
- `void _bncavx2_rdd_mul(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret :=  $a \times b$`
- `void _bncavx2_rdd_div(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) ... ret :=  $a/b$`
- `void _bncavx2_rdd_abs(__m256d ret[DDSIZE], __m256d a[DDSIZE]) ... ret :=  $|a|$`

2.10.5 Triple-double precision arithmetic

- `void _bncavx2_set0_td(__m256d ret[TDSIZE]) ... ret := 0`
- `void _bncavx2_get_td_m256d_i(tdfloat * ret, __m256d ret4[TDSIZE], int avx_index) ... ret := ret4[avx_index]`
- `void _bncavx2_rtd_sum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) ... ret :=  $ret4[0] + ret4[1] + ret4[2]$`
- `void _bncavx2_rtd_abs(__m256d ret[TDSIZE], __m256d a[TDSIZE]) ... ret :=  $|a|$`
- `void _bncavx2_rtd_abssum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) ... ret :=  $|ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|$`
- `void _bncavx2_rtd_absmax256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) ... ret :=  $\max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)$`
- `void _bncavx2_rtd_norm256d(double ret[TDSIZE], __m256d ret4[TDSIZE])`
- `void _bncavx2_vec_sum(__m256d e[], const __m256d x[], int n) ... e :=  $\text{vec\_sum}(x)$`
- `void _bncavx2_vseb(__m256d y[], int ny, const __m256d e[], int ne) ... y :=  $\text{vec\_sum\_err\_branch}(\text{vseb}(k))(e)$`
- `void _bncavx2_merge(__m256d c[], __m256d a[], int na, __m256d b[], int nb) ... Merge a[na] & b[nb] into c[na + nb]`

(Macro) `_bncavx2_rtd_add() _bncavx2_rtd_addq`

- `void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret :=  $a + b$`
- `void _bncavx2_rtd_addt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])`
- `void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret :=  $a \times b$`
- `void _bncavx2_rtd_mul(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])`
- `void _bncavx2_rtd_neg(__m256d c[TDSIZE], __m256d a[TDSIZE]) ... c :=  $-a$`
- `void _bncavx2_rtd_sub(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... c :=  $a - b$`
- `void _bncavx2_rtd_subq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])`
- `void _bncavx2_rtd_divq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... c :=  $a/b$`
- `void _bncavx2_to_td(__m256d r[TDSIZE], __m256d a, __m256d b, __m256d c) ... r :=  $\text{TD}(a, b, c)$`
- `void _bncavx2_rtd_mul_d(__m256d c[TDSIZE], __m256d a, __m256d b[TDSIZE]) ... c :=  $a \times b$`
- `void _bncavx2_rtd_mul_dd(__m256d c[TDSIZE], __m256d a[DDSIZE], __m256d b[TDSIZE]) ... c :=  $a \times b$`

(Macro) `_bncavx2_rtd_div()... _bncavx2_rtd_divtq`

- `void _bncavx2_rtd_divt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... c :=  $a/b$`
- `void _bncavx2_rtd_divtq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])`

2.10.6 Quadruple-double precision arithmetic

- `void _bncavx2_set0_qd(__m256d ret[QDSIZE]) ... ret := 0`
- `void _bncavx2_get_qd_m256d_i(qdfloat * ret, __m256d ret4[QDSIZE], int avx_index) ret := ret4[avx_index]`
- `void _bncavx2_rqd_sum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := mmval[0] + ... + mmval[7]`
`void _bncavx2_rqd_abs__m256d ret[QDSIZE], __m256d a[QDSIZE] ... ret := |a|`
- `void _bncavx2_rqd_abssum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|`
- `void _bncavx2_rqd_absmax256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)`
- `void _bncavx2_rqd_norm256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) ... ret := ||ret4[0]2 + ret4[1]2 + ret4[2]2 + ret4[3]2||2`
- `void _bncavx2_renorm(__m256d * c0, __m256d * c1, __m256d * c2, __m256d * c3) ... renorm(double *c0, double *c1, double *c2, double *c3)`
- `void _bncavx2_renorm4(__m256d * c0, __m256d * c1, __m256d * c2, __m256d * c3, __m256d * c4) ... renorm4(double *c0, double *c1, double *c2, double *c3, double *c4)`
- `void _bncavx2_three_sum(__m256d * a, __m256d * b, __m256d * c) ... three_sum(double *a, double *b, double *c)`
- `void _bncavx2_three_sum2(__m256d * a, __m256d * b, __m256d * c) ... void three_sum2(double *a, double *b, double *c)`
- `void _bncavx2_rqd_add(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a + b`
- `void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret := a + b`
- `void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... ret := a × b`
- `void _bncavx2_rqd_mul(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a × b`
- `void _bncavx2_rqd_sub(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... ret := a - b`
- `void _bncavx2_rqd_mul_d(__m256d c[QDSIZE], const __m256d a[QDSIZE], __m256d b) ... ret := a × b`
- `void _bncavx2_rtd_divq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) ... c := a/b in the manner of quad-double division`
- `void _bncavx2_rqd_div(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) ... c := a/b`

2.11 Miscellaneous functions

The following functions are defined in `bnc_common.h`, which are coded in `bnc_common_func.c` and `algebraic_eq.c`.

2.11.1 common functions

- `void bnc_print_env_all(void) ...` Print all environmental variables of `bncmatmul` library.
- `double mylog2(double x) ...` Return $\log_2(x)$.
- `double matmul_gflops(double comp_sec, int dim) ...` Return giga flops when calculating a `dim`-dimensional dense square matrix product from `comp_sec` (computational time in second).
- `int byte_double_sqmat(int dim) ...` Return the number of bytes to store a `dim`-dimensional square

matrix.

- `void set_bnc_default_prec(unsigned long precision)`
- `void set_bnc_default_prec_decimal(unsigned long precision) ...` Set the default precision in bit or in decimal digit.
- `mpf_rnd_t get_bnc_default_rounding_mode(void) ...` Return the current rounding mode.
- `void set_bnc_rounding_mode(mpf_rnd_t rounding_mode) ...` Set `rounding_mode` as default rounding mode.
- `unsigned long int get_bnc_default_prec(void) ...` Return the current default precision in bit.
- `void mpf_fma(mpf_t rop, mpf_t op1, mpf_t op2, mpf_t op3) ...` $rop := op1 \times op2 + op3$
- `void mpf_log10(mpf_t ret, mpf_t x) ...` $\log_{10}(x)$
- `void mpf_exp(mpf_t ret, mpf_t x) ...` $\exp(x)$
- `void mpf_log(mpf_t ret, mpf_t x) ...` $\log_e(x)$
- `void mpf_sin(mpf_t ret, mpf_t x) ...` $\sin(x)$
- `void mpf_cos(mpf_t ret, mpf_t x) ...` $\cos(x)$
- `void mpf_tan(mpf_t ret, mpf_t x) ...` $\tan(x)$
- `void mpf_atan(mpf_t ret, mpf_t x) ...` $\arctan(x)$

2.11.2 Random functions

- `void rdd_srand(unsigned long seed)`
- `void rtd_srand(unsigned long seed)`
- `void rqd_srand(unsigned long seed)`
- `void mpf_srand(unsigned long seed) ...` Set random seed and initialize the current random state for each precision.
- `void rdd_urand(double ret[DDSIZE])`
- `void rtd_urand(double ret[TDSIZE])`
- `void rqd_urand(double ret[QDSIZE])`
- `void mpf_urand(mpf_t ret) ...` Return a random number on $[0, 1]$.
- `void rdd_nrand(double ret[DDSIZE])`
- `void rtd_nrand(double ret[TDSIZE])`
- `void rqd_nrand(double ret[QDSIZE])`
- `void mpf_nrand(mpf_t ret) ...` Return a random number following standard normal distribution $[-1, 1]$.

2.11.3 Solver for algebraic equation

- `int dquadratic_eq(double ans_re[2], double ans_im[2], double coef[3])`
- `int mpfquadratic_eq(mpf_t ans_re[2], mpf_t ans_im[2], mpf_t coef[3]) ...` Solve $\text{coef}[2] \cdot x^2 + \text{coef}[1] \cdot x + \text{coef}[0] = 0$ and store the roots as $x = \text{ans_re}[0] + \text{ans_im}[0] \cdot i$ and $\text{ans_re}[1] + \text{ans_im}[1] \cdot i$

2.12 Tool functions for benchmarking

There some useful functions for benchmarking in BNCmatmul.

2.12.1 Time estimation

- `float fget_sec(int flag)`
- `double get_sec(int flag) ...` Return the current real, user, system time as double prec. in second if `flag` $\neq 0$, and the sum of the current user and system time in second if `flag` = 0.
- `double get_secv(void)`
- `float fget_secv(void) ...` The same as `[f]get_sec(0)`.

- `double get_real_sec(int flag)`
- `float fget_real_sec(int flag) ...` Return the current `tv_sec` (in second) and `tv_usec` (in micro-second) as double prec. in second if `flag ≠ 0`, and the sum of `tv_sec + tv_usec/106`.
- `double get_real_secv(void)`
- `float fget_real_secv(void) ...` The same as `[f]get_real_sec(0)`.

2.12.2 Relative error of vector and matrix

■ Relative error of vector

- `void relerr3_dvector(double *max_relerr, double *min_relerr, double *norm_relerr, DVector vec, DVector vec_true, int kind_of_norm)`
- `void relerr3_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)`
- `void relerr3_tdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVector vec_true, int kind_of_norm)`
- `void relerr3_tdvector_qdvec(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, QDVector vec_true, int kind_of_norm)`
- `void relerr3_qdvector(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm)`
- `void relerr3_cddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDVector vec, CDDVector vec_true, int kind_of_norm)`
- `void relerr3_ctdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE], double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDVector vec, CTDVector vec_true, int kind_of_norm)`
- `void relerr3_cqdvector(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDVector vec, CQDVector vec_true, int kind_of_norm)`
- `void relerr3_mpfvector(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector vec, MPFVector vec_true, int kind_of_norm)`
- `void relerr3_cmpfvector(mpf_t max_abs_relerr, mpf_t min_abs_relerr, mpf_t max_real_relerr, mpf_t min_real_relerr, mpf_t max_image_relerr, mpf_t min_image_relerr, mpf_t norm_relerr, CMPFVector vec, CMPFVector vec_true, int kind_of_norm)`

■ Relative error of matrix

`kind_of_norm = 0` $\|A\|_\infty$
`kind_of_norm = 1` $\|A\|_1$
`kind_of_norm = 2` $\|A\|_F$

- `void relerr3_dmatrix(double *max_relerr, double *min_relerr, double *norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm)`
- `void relerr3_ddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)`
- `void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)`
- `void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)`
- `void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)`

- `void relerr3_cddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDMatrix mat, CDDMatrix mat_true, int kind_of_norm)`
- `void relerr3_ctdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE], double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDMatrix mat, CTDMatrix mat_true, int kind_of_norm)`
- `void relerr3_cqdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDMatrix mat, CQDMatrix mat_true, int kind_of_norm)`
- `void relerr3_cmpfmatrix(mpf_t max_abs_relerr, mpf_t min_abs_relerr, mpf_t max_real_relerr, mpf_t min_real_relerr, mpf_t max_image_relerr, mpf_t min_image_relerr, mpf_t norm_relerr, CMPFMatrix mat, CMPFMatrix mat_true, int kind_of_norm)`

Chapter 3

Basic linear computation

3.1 Datatypes of D, DD, TD, QD and MPFR vector and matrix

Vector datatypes of DD, TD, QD and MPFR precision are shown as follows:

DDVector, TDVector, QDVector

```
typedef struct
{
    long int dim; // dim <= real_dim
    long int real_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE]; // [TDSIZE] and [QDSIZE] used as TDVector and
    ↪ QDVector
} ddvector;

typedef *DDVector; // DDVector, TDVector, QDVector are pointers.
```

MPFVector

```
typedef struct{
    unsigned long int prec; // default precision of element
    mpf_t *element;
    long int dim;
    long int real_dim;
} mpfvector;

typedef mpfvector *MPFVector; // MPFVector is a pointer.
```

Matrix datatypes of DD, TD, QD and MPFR precision are shown as follows:

DDMatrix, TDMatrix, QDMatrix

```
// DD matrix
typedef struct{
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE];
} ddmatrix;

typedef *DDMatrix;
```

MPFMatrix

```

typedef struct{
    unsigned long int prec;
    mpf_t *element;
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim;
    void *element_block; // mantissa block
} mpfmatrix;

typedef mpfmatrix *MPFMatrix;

```

3.2 Vector arithmetic

- `ddfloat get_ddvector_i_ddfloat(DDVector vec, long int index)` ... Get index-th element of `vec` as `ddfloat` datatype.
- `tdfloat get_tdvector_i_tdfloat(TDVector vec, long int index)`
- `qdfloat get_qdvector_i_qdfloat(QDVector vec, long int index)`
- (Macro) `GET_DDVECTOR_I(vec, index)` ... Get index-th element as pointer to `ddfloat`.
- (Macro) `get_ddvector_i(vec, index)`
- (Macro) `GET_TDVECTOR_I(vec, index)` ... Get index-th element as pointer to `tdfloat`.
- (Macro) `get_tdvector_i(vec, index)`
- (Macro) `GET_QDVECTOR_I(vec, index)` ... Get index-th element as pointer to `qdfloat`.
- (Macro) `get_qdvector_i(vec, index)`
- (Macro) `GET_MPFVECTOR_I(vec, index)` ... Get index-th element as pointer to `mpf_t` (`mpfr_t`).
- (Macro) `get_mpfvector_i(vec, index)`
 - `void set_ddvector_i(DDVector vec, long int index, double * val)` ... Store `val` to index-th element of `vec`.
- (Macro) `SET_DDVECTOR_I(vec, index, val)`
 - `void set_tdvector_i(TDVector vec, long int index, double * val)`
- (Macro) `SET_TDVECTOR_I(vec, index, val)`
 - `void set_qdvector_i(QDVector vec, long int index, double * val)`
- (Macro) `SET_QDVECTOR_I(vec, index, val)`
 - `void set_qdvector_i(QDVector vec, long int index, double * val)`
- (Macro) `SET_QDVECTOR_I_D(vec, index, val)`
 - `void set_ddvector_i_d(DDVector vec, long int index, double val)` ... Store `val` to index-th element of `vec` as `double`.
- (Macro) `SET_DDVECTOR_I_D(vec, index, val)`
 - `void set_tdvector_i_d(TDVector vec, long int index, double val)`
- (Macro) `SET_TDVECTOR_I_D(vec, index, val)`
 - `void set_qdvector_i_d(QDVector vec, long int index, double val)`
- (Macro) `SET_QDVECTOR_I_D(vec, index, val)`
 - `void set0_ddvector_i(DDVector vec, long int index)` ... Store zero to index-th element of `vec`.
- (Macro) `SET0_DDVECTOR_I(vec, index)`
 - `void set0_tdvector_i(TDVector vec, long int index)`
- (Macro) `SET0_TDVECTOR_I(vec, index)`
 - `void set0_qdvector_i(QDVector vec, long int index)`
- (Macro) `SET0_QDVECTOR_I(vec, index)`
 - `DDVector init_ddvector(in dimension)` ... Allocate and initialize a vector and set it as zero vector.
 - `TDVector init_tdvector(in dimension)`
 - `QDVector init_qdvector(in dimension)`
 - `MPFVector init_mpfvector(in dimension)` ... Allocate and initialize a `MPFVector` in default

- precision in bits with `bnc_set_default_prec` and set it as zero vector.
- `MPFVector init2_mpfvector(int dimension, unsigned long prec) ...` Initialize a `MPFVector` in `prec` bits and set it as zero vector.
 - `void free_ddvector(DDVector vec) ...` Free `vec`.
 - `void free_tdvector(TDVector vec)`
 - `void free_qdvector(QDVector vec)`
 - `void free_mpfvector(MPFVector vec)`
 - `void set_ddfloat_ddvec(ddfloat ret[], int ret_dim, DDVector vec) ...` Convert `vec` to `ddfloat` array.
 - `void set_tdfloat_tdvec(tdfloat ret[], int ret_dim, TDVector vec) ...` Convert `vec` to `tdfloat` array.
 - `void set_qdfloat_qdvec(qdfloat ret[], int ret_dim, QDVector vec) ...` Convert `vec` to `qdfloat` array.
 - `void set_ddvector_ddfloat(DDVector ret, ddfloat array[], int array_dim) ...` Convert `ddfloat` array to `DDVector` `ret`.
 - `void set_tdvector_tdfloat(TDVector ret, tdfloat array[], int array_dim) ...` Convert `tdfloat` array to `TDVector` `ret`.
 - `void set_qdvector_qdfloat(QDVector ret, qdfloat array[], int array_dim) ...` Convert `qdfloat` array to `QDVector` `ret`.
 - `void print_ddvector(DDVector vec) ...` Print `vec`.
 - `void print_tdvector(TDVector vec)`
 - `void print_qdvector(QDVector vec)`
 - `void print_mpfvector(MPFVector vec)`
 - `void set0_ddvector(DDVector vec) ...` Set `vec` as zero vector.
 - `void set0_tdvector(TDVector vec)`
 - `void set0_qdvector(QDVector vec)`
 - `void set0_mpfvector(MPFVector vec)`
 - `void set_ddvector_i_str(DDVector vec, long int index, const char * str) ...` Set `*str` expressed in decimal to `index`-th element of `vec`.
 - `void set_tdvector_i_str(TDVector vec, long int index, const char * str)`
 - `void set_qdvector_i_str(QDVector vec, long int index, const char * str)`
 - `void set_mpfvector_i_str(MPFVector vec, long int index, const char * str)`
 - `void add_ddvector(DDVector c, DDVector a, DDVector b) ...` $c := a + b$
 - `void add_tdvector(TDVector c, TDVector a, TDVector b)`
 - `void add_qdvector(QDVector c, QDVector a, QDVector b)`
 - `void add_mpfvector(MPFVector c, MPFVector a, MPFVector b)`
 - `void add2_ddvector(DDVector c, DDVector a) ...` $c := c + a$
 - `void add2_tdvector(TDVector c, TDVector a)`
 - `void add2_qdvector(TDVector c, QDVector a)`
 - `void add2_mpfvector(MPFVector c, MPFVector a)`
 - `void sub_ddvector(DDVector c, DDVector a, DDVector b) ...` $c := a - b$
 - `void sub_tdvector(TDVector c, TDVector a, TDVector b)`
 - `void sub_qdvector(QDVector c, QDVector a, QDVector b)`
 - `void sub_mpfvector(MPFVector c, MPFVector a, MPFVector b)`
 - `void sub2_ddvector(DDVector c, DDVector a) ...` $c -= a$
 - `void sub2_tdvector(TDVector c, TDVector a)`
 - `void sub2_qdvector(QDVector c, QDVector a)`
 - `void sub2_mpfvector(MPFVector c, MPFVector a)`
 - `void cmul_ddvector(DDVector c, double val [DDSIZE], DDVector a) ...` $c := val \times a$
 - `void cmul_tdvector(TDVector c, double val [TDSIZE], TDVector a)`
 - `void cmul_qdvector(QDVector c, double val [QDSIZE], QDVector a)`
 - `void cmul_mpfvector(MPFVector c, mpf_t val, MPFVector a)`
 - `void cmul2_ddvector(DDVector c, double val [DDSIZE]) ...` $c := val \times c$

- void cmul2_ddvector(TDVector c, double val[TDSIZE])
- void cmul2_qdvector(QDVector c, double val[QDSIZE])
- void cmul2_mpfvector(MPFVector c, mpf_t val)
- void add_cmul_ddvector(DDVector c, DDVector a, double val[DDSIZE], DDVector b) $\cdots c := a + val * b$
- void add_cmul_tdvector(TDVector c, TDVector a, double val[TDSIZE], TDVector b)
- void add_cmul_qdvector(QDVector c, QDVector a, double val[QDSIZE], QDVector b)
- void add_cmul_mpfvector(MPFVector c, MPFVector a, mpf_t val, MPFVector b)
- void ip_ddvector(double ret[DDSIZE], DDVector a, DDVector b) $\cdots$ Calculate the dot product of a and b
- void ip_tdvector(double ret[TDSIZE], TDVector a, TDVector b)
- void ip_qdvector(double ret[QDSIZE], QDVector a, QDVector b)
- void ip_mpfvector(mpf_t ret, MPFVector a, MPFVector b)
- void neg_ddvector(DDVector c, DDVector a) $\cdots c := -a$
- void neg_tdvector(TDVector c, TDVector a)
- void neg_qdvector(QDVector c, QDVector a)
- void neg_mpfvector(MPFVector c, MPFVector a)
- void norm1_ddvector(double ret[DDSIZE], DDVector a) $\cdots \|a\|_1$
- void norm1_tdvector(double ret[TDSIZE], TDVector a)
- void norm1_qdvector(double ret[QDSIZE], QDVector a)
- void norm1_mpfvector(mpf_t ret, MPFVector a)
- void normi_ddvector(double ret[DDSIZE], DDVector a) $\cdots \|a\|_\infty$
- void normi_tdvector(double ret[TDSIZE], TDVector a)
- void normi_qdvector(double ret[QDSIZE], QDVector a)
- void normi_mpfvector(mpf_t ret, MPFVector a)
- void norm2_ddvector(double ret[DDSIZE], DDVector vec) $\cdots \|a\|_2$
- void norm2_tdvector(double ret[TDSIZE], TDVector vec)
- void norm2_qdvector(double ret[QDSIZE], QDVector vec)
- void norm2_mpfvector(mpf_t ret, MPFVector vec)

3.3 Matrix arithmetic

- ddfloat get_ddmatrix_ij_ddfloat(DDMatrix mat, long int i, long int j) $\cdots$ Get the (i, j) -th element of mat.
- (Macro) GET_DDMATRIX_IJ(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))
- (Macro) get_ddmatrix_ij(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))
- tdfloat get_tdmatrix_ij_tdfloat(TDMatrix mat, long int i, long int j)
- (Macro) GET_TDMATRIX_IJ(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))
- (Macro) get_tdmatrix_ij(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))
- qdfloat get_qdmatrix_ij_qdfloat(QDMatrix mat, long int i, long int j)
- (Macro) GET_QDMATRIX_IJ(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))
- (Macro) get_qdmatrix_ij(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))
- (Macro) GET_MPFMATRIX_IJ(mat, i, j) ((get_mpfmatrix_ij((mat), (i), (j))))
- mpf_ptr get_mpfmatrix_ij(MPFMatrix mat, long int i, long int j)
- void set_ddmatrix_ij(DDMatrix mat, long int i, long int j, double * val)
- (Macro) SET_DDMATRIX_IJ(mat, i, j, val) set_ddmatrix_ij((mat), (i), (j), (val))
- void set_tdmatrix_ij(TDMatrix mat, long int i, long int j, double * val)
- (Macro) SET_TDMATRIX_IJ(mat, i, j, val) set_tdmatrix_ij((mat), (i), (j), (val))
- void set_qdmatrix_ij(QDMatrix mat, long int i, long int j, double * val)
- (Macro) SET_QDMATRIX_IJ(mat, i, j, val) set_qdmatrix_ij((mat), (i), (j), (val))
- void set_mpfmatrix_ij(MPFMatrix mat, long int i, long int j, mpf_t val)
- (Macro) SET_MPFMATRIX_IJ(mat, i, j, val) set_mpfmatrix_ij((mat), (i), (j), (val))

- void set_ddmatrix_ij_d(DDMatrix mat, long int i, long int j, double val)
- (Macro) SET_DDMATRIX_IJ_D(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (val))
- (Macro) SET_DDMATRIX_IJ_UI(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))
- (Macro) set_ddmatrix_ij_ui(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))
 - void set0_ddmatrix_ij(DDMatrix mat, long int i, long int j)
- (Macro) SET0_DDMATRIX_IJ(mat, i, j) set0_ddmatrix_ij((mat), (i), (j))
 - void set0_ddmatrix(DDMatrix mat) ... set mat as zero matrix
 - void set0_tdmatrix(TDMatrix mat)
 - void set0_qdmatrix(QDMatrix mat)
 - void set0_mpfmatrix(MPFMatrix mat)
 - DDMatrix init_ddmatrix(long int row_dim, long int col_dim) ... Initialize DDMatrix.
 - TDMatrix init_tdmatrix(long int row_dim, long int col_dim) ... Initialize TDMatrix.
 - QDMatrix init_qdmatrix(long int row_dim, long int col_dim) ... Initialize QDMatrix.
 - MPFMatrix init_mpfmatrix(long int row_dim, long int col_dim) ... Initialize MPFMatrix.
 - MPFMatrix init2_mpfmatrix(long int row_dim, long int col_dim, unsigned long prec) ... Initialize MPFMatrix as prec-bit precision.
 - void free_ddmatrix(DDMatrix mat) ... Free mat
 - void free_tdmatrix(TDMatrix mat)
 - void free_qdmatrix(QDMatrix mat)
 - void free_mpfmatrix(MPFMatrix mat)
 - void print_ddmatrix(DDMatrix mat) ... Print mat.
 - void print_tdmatrix(TDMatrix mat)
 - void print_qdmatrix(QDMatrix mat)
 - void print_mpfmatrix(MPFMatrix mat)
 - void set_ddfloat_ddmat(ddfloat ret[], int ret_dim, DDMatrix mat) ... Convert DDMatrix mat to ddfloat array.
 - void set_tdfloat_tdmatrix(tdfloat ret[], int ret_dim, DDMatrix mat) ... Convert TDMatrix mat to tdfloat array.
 - void set_qdfloat_qdmat(qdfloat ret[], int ret_dim, DDMatrix mat) ... Convert QDMatrix mat to qdfloat array.
 - void set_ddmatrix_ddfloat(DDMatrix ret, ddfloat array[], int array_dim) ... Convert ddfloat array to DDMatrix.
 - void set_tdmatrix_tdfloat(TDMatrix ret, tdfloat array[], int array_dim) ... Convert tdfloat array to TDMatrix.
 - void set_qdmatrix_qdfloat(QDMatrix ret, qdfloat array[], int array_dim) ... Convert qdfloat array to QDMatrix
 - void mul_ddmatrix(DDMatrix ret, DDMatrix a, DDMatrix b) ... Simple matrix multiplication.
 - void mul_tdmatrix(TDMatrix ret, TDMatrix a, TDMatrix b)
 - void mul_qdmatrix(QDMatrix ret, QDMatrix a, QDMatrix b)
 - void mul_mpfmatrix(MPFMatrix ret, MPFMatrix a, MPFMatrix b)
 - void normf_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... Get the value of Frobenius norm of mat.
 - void normf_tdmatrix(double ret[TDSIZE], TDMatrix mat)
 - void normf_qdmatrix(double ret[QDSIZE], QDMatrix mat)
 - void normf_mpfmatrix(mpf_t ret, MPFMatrix mat)
 - void print_normf_ddmatrix(const char * str, DDMatrix mat) ... print the Frobenius norm of mat.
 - void normi_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... Infinity norm of matrix.
 - void normi_tdmatrix(double ret[TDSIZE], TDMatrix mat)
 - void normi_qdmatrix(double ret[QDSIZE], QDMatrix mat)
 - void normi_mpfmatrix(mpf_t ret, TDMatrix mat)
 - void norm1_ddmatrix(double ret[DDSIZE], DDMatrix mat) ... 1-norm of matrix.
 - void norm1_tdmatrix(double ret[TDSIZE], TDMatrix mat)

- void norm1_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void norm1_mpfmatrix(mpf_t ret, MPFMatrix mat)
- void add_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) ... $c := a + b$
- void add_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void add_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void add_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void sub_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) ... $c := a - b$
- void sub_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void sub_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void sub_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void cmul_ddmatrix(DDMatrix c, double sc[DDSIZE], DDMatrix a) ... $c := sc \times a$
- void cmul_tdmatrix(TDMatrix c, double sc[TDSIZE], TDMatrix a)
- void cmul_qdmatrix(QDMatrix c, double sc[QDSIZE], QDMatrix a)
- void cmul_mpfmatrix(MPFMatrix c, mpf_t sc, MPFMatrix a)
- void transpose_ddmatrix(DDMatrix c, DDMatrix a) ... $c := a^T$
- void transpose_tdmatrix(TDMatrix c, TDMatrix a)
- void transpose_qdmatrix(QDMatrix c, QDMatrix a)
- void transpose_mpfmatrix(MPFMatrix c, MPFMatrix a)
- void subst_ddmatrix(DDMatrix c, DDMatrix a) ... $c := a$
- void subst_tdmatrix(TDMatrix c, TDMatrix a)
- void subst_qdmatrix(QDMatrix c, QDMatrix a)
- void subst_mpfmatrix(MPFMatrix c, MPFMatrix a)
- void setI_ddmatrix(DDMatrix c) ... $c := I$
- void setI_tdmatrix(TDMatrix c)
- void setI_qdmatrix(QDMatrix c)
- void setI_mpfmatrix(MPFMatrix c)
- void mul_ddmatrix_ddvec(DDVector v, DDMatrix a, DDVector vb) ... $v := a * vb$
- void mul_tdmatrix_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrix_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrix_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void mul_ddmatrixt_ddvec(DDVector v, DDMatrix a, DDVector vb) ... $v := a^T * vb$
- void mul_tdmatrixt_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrixt_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrixt_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void inv_ddmatrix(DDMatrix a) ... $a := a^{-1}$ (only for square matrix)
- void inv_tdmatrix(TDMatrix a)
- void inv_qdmatrix(QDMatrix a)
- void inv_mpfmatrix(MPFMatrix a)
- void subst_mpfvector_ddvec(MPFVector c, DDVector a) ... $c := (mpf)a$
- void subst_ddvector_mpfvec(DDVector c, MPFVector MPFVector a) ... $c := (dd)a$
- void subst_mpfmatrix_ddmat(MPFMatrix c, DDMatrix a) ... $c := (mpf)a$
- void subst_ddmatrix_mpfmat(DDMatrix c, MPFMatrix a) ... $c := (dd)a$
- void relerr_ddvector_mpfvec(double relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) ... Norm relative error of vector.
- void relerr_element_ddvector_mpf(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) ... Elementwise relative errors of vector.
/* c := (dd)a */
- void subst_ddvector_dvec(DDVector c, DVector sa) ... $c := (dd)a$
/* c := (d)a */
- void subst_dvector_ddvec(DVector c, DDVector a) ... $c := (double)a$
/* c := (dd)a */

- void subst_ddmatrix_dmat(DDMatrix c, DMatrix a) ... $c := (dd)a$
/* c := (d)a */
- void subst_dmatrix_ddmat(DMatrix c, DDMatrix a) ... $c := (double)a$
- void relerr_ddvector(double relerr[DDSIZE], DDVector approx_vec, DDVector true_vec, int norm_type) ... Norm relative error of vector.
- void relerr_element_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector approx_vec, DDVector true_vec, int norm_type) ... Elementwise relative errors of vector.
- void row_swap_ddmatrix(DDMatrix mat, long int row_index0, long int row_index1, long int col_start, long int col_end) ... Exchange a(row_index0, col_start:col_end) to a(row_index1, col_start:col_end)

3.4 File I/O with matrix and vector

- void fread_ddmatrix(FILE * fp, DDMatrix mat) ... Read matrix elements from fp and store theme in mat.
- void fread_tdmatrix(FILE * fp, TDMatrix mat)
- void fread_qdmatrix(FILE * fp, QDMatrix mat)
- void fread_mpfmatrix(FILE * fp, MPFMatrix mat)
- void fread_ddmatrix_fname(const char * fname, DDMatrix mat) ... Read matrix elements from fname and store theme in mat.
- void fread_tdmatrix_fname(const char * fname, TDMatrix mat)
- void fread_qdmatrix_fname(const char * fname, QDMatrix mat)
- void fread_mpfmatrix_fname(const char * fname, MPFMatrix mat)
- void fwrite_ddmatrix(FILE * fp, DDMatrix mat) ... Write matrix elements of mat to fp.
- void fwrite_tdmatrix(FILE * fp, TDMatrix mat)
- void fwrite_qdmatrix(FILE * fp, QDMatrix mat)
- void fwrite_mpfmatrix(FILE * fp, MPFMatrix mat)
- void fwrite_ddmatrix_fname(const char * fname, DDMatrix mat) ... Write matrix elements of mat to fname.
- void fwrite_tdmatrix_fname(const char * fname, TDMatrix mat)
- void fwrite_qdmatrix_fname(const char * fname, QDMatrix mat)
- void fwrite_mpfmatrix_fname(const char * fname, MPFMatrix mat)
- void fread_ddvector(FILE * fp, DDVector vec) ... Read vector elements from fp and store theme in vec.
- void fread_tdvector(FILE * fp, TDVector vec)
- void fread_qdvector(FILE * fp, QDVector vec)
- void fread_mpfvector(FILE * fp, MPFVector vec)
- void fread_ddvector_fname(const char * fname, DDVector vec) ... Read vector elements from fname and store theme in vec.
- void fread_tdvector_fname(const char * fname, TDVector vec)
- void fread_qdvector_fname(const char * fname, QDVector vec)
- void fread_mpfvector_fname(const char * fname, MPFVector vec)
- void fwrite_ddvector(FILE * fp, DDVector vec) ... Write vector elements of vec to fp.
- void fwrite_tdvector(FILE * fp, TDVector vec)
- void fwrite_qdvector(FILE * fp, QDVector vec)
- void fwrite_mpfvector(FILE * fp, MPFVector vec)
- void fwrite_ddvector_fname(const char * fname, DDVector vec) ... Write vector elements of vec to fname.
- void fwrite_tdvector_fname(const char * fname, TDVector vec)
- void fwrite_qdvector_fname(const char * fname, QDVector vec)
- void fwrite_mpfvector_fname(const char * fname, MPFVector vec)

- void read_test_linear_eq_dd(DDMatrix A, DDVector true_x, DDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b) ... Read the coefficient matrix A from fname_A, the true_x from fname_true_x, and the vector b from fname_b.
- void read_test_linear_eq_td(TDMatrix A, TDVector true_x, TDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_qd(QDMatrix A, QDVector true_x, QDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_mpf(MPFMatrix A, MPFVector true_x, MPFVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)

3.5 Generating test matrices

The following functions provide various test matrices for benchmark tests.

- void hilbert_ddmatrix(DDMatrix a, long int dim) ... Hilbert matrix.
- void hilbert_tdmatrix(TDMatrix a, long int dim)
- void hilbert_qdmatrix(QDMatrix a, long int dim)
- void hilbert_mpfmatrix(MPFMatrix a, long int dim)
- void lotkin_ddmatrix(DDMatrix a, long int dim) ... Lotkin matrix.
- void lotkin_tdmatrix(TDMatrix a, long int dim)
- void lotkin_qdmatrix(QDMatrix a, long int dim)
- void lotkin_mpfmatrix(MPFMatrix a, long int dim)
- void frank_ddmatrix(DDMatrix a, long int dim) ... Symmetric Frank matrix.
- void frank_tdmatrix(TDMatrix a, long int dim)
- void frank_qdmatrix(QDMatrix a, long int dim)
- void frank_mpfmatrix(MPFMatrix a, long int dim)
- void tridiag_ddmatrix(DDMatrix a, DDVector low_subdiag, DDVector diag, DDVector up_subdiag, long int dim) ... Triagonal matrix.
- void tridiag_tdmatrix(DDMatrix a, TDVector low_subdiag, TDVector diag, TDVector up_subdiag, long int dim)
- void tridiag_qdmatrix(DDMatrix a, QDVector low_subdiag, QDVector diag, QDVector up_subdiag, long int dim)
- void tridiag_mpfmatrix(DDMatrix a, MPFVector low_subdiag, MPFVector diag, MPFVector up_subdiag, long int dim)
- void int_sym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ... Integer Symmetrix Random Matrix.
- void int_sym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim)
- void int_sym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim)
- void int_sym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim)
- void int_unsym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ... Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim) ... Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim) ... Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim) ... Integer Unsymmetrix Random Matrix.
- void diag_ddmatrix(DDMatrix mat, DDVector diag, long int dim) ... Real Diagonal Matrix.
- void diag_tdmatrix(TDMatrix mat, TDVector diag, long int dim)
- void diag_qdmatrix(QDMatrix mat, QDVector diag, long int dim)
- void diag_mpfmatrix(MPFMatrix mat, MPFVector diag, long int dim)
- void toeplitz_ddmatrix(DDMatrix mat, double gamma_param[DDSIZE], long int dim) ... Toeplitz

matrix.

- void toeplitz_tdmatrix(TDMatrix mat, double gamma_param[TDSIZE], long int dim)
- void toeplitz_qdmatrix(QDMatrix mat, double gamma_param[QDSIZE], long int dim)
- void toeplitz_mpfmatrix(MPFMatrix mat, mpf_t gamma_param, long int dim)
- void pascal_ddmatrix(DDMatrix ret, long int dim) ... Pascal matrix.
- void pascal_tdmatrix(TDMatrix ret, long int dim)
- void pascal_qdmatrix(QDMatrix ret, long int dim)
- void pascal_mpfmatrix(MPFMatrix ret, long int dim)
- void im_rand_ddmatrix(DDMatrix ret, unsigned long seed) ... $I - random$
- void im_rand_tdmatrix(TDMatrix ret, unsigned long seed)
- void im_rand_qdmatrix(QDMatrix ret, unsigned long seed)
- void im_rand_mpfmatrix(MPFMatrix ret, unsigned long seed)

3.6 Simple, block, and Strassen matrix multiplication and related functions

- (Macro) mul_ddmatrix_simple(c, a, b) ... mul_ddmatrixc, a, b ... Matrix multiplication based on simple triple-loop way
- (Macro) mul_tdmatrix_simple(c, a, b) ... mul_tdmatrixc, a, b
- (Macro) mul_qdmatrix_simple(c, a, b) ... mul_qdmatrixc, a, b
- void mul_mpfmatrix_simple(MPFMatrix c, MPFMatrix a, MPFMatrix b)
 - void mul_dmatrix_strassen(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Matrix multiplication based on Strassen or Winograd algorithms
 - void mul_ddmatrix_strassen(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
 - void mul_tdmatrix_strassen(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
 - void mul_qdmatrix_strassen(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
 - void mul_mpfmatrix_strassen(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
 - void mul_dmatrix_block(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Block matrix multiplication
 - void mul_ddmatrix_block(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
 - void mul_tdmatrix_block(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
 - void mul_qdmatrix_block(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
 - void mul_mpfmatrix_block(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
 - void mul_dmatrix_strassen_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Strassen's Algorithm (even dimension)
 - void mul_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
 - void mul_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
 - void mul_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
 - void mul_mpfmatrix_strassen_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
 - void mul_dmatrix_winograd_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) ... Winograd Variant of Strassen's Algorithm
 - void mul_ddmatrix_winograd_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
 - void mul_tdmatrix_winograd_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)

- min_dim)
- void mul_qdmatrix_winograd_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_winograd_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void inv_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, long int min_dim) ... Computation of Inverse Matrix by using Strassen's Algorithm
- void inv_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, long int min_dim)
- void inv_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, long int min_dim)

3.6.1 OpenMP

- (Macro) `_bncomp_mul_ddmatrix_simple(c, a, b)` `_bncomp_mul_ddmatrix(c, a, b)` ... Parallelized simple matrix multiplication,
- (Macro) `_bncomp_mul_tdmatrix_simple(c, a, b)` `_bncomp_mul_tdmatrix(c, a, b)`
- (Macro) `_bncomp_mul_qdmatrix_simple(c, a, b)` `_bncomp_mul_qdmatrix(c, a, b)`
- (Macro) `_bncomp_mul_mpfmatrix_simple(c, a, b)` `_bncomp_mul_mpfmatrix(c, a, b)`
- void `_bncomp_mul_ddmatrix_block`(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ... Parallelized block matrix multiplication.
- void `_bncomp_mul_tdmatrix_block`(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_qdmatrix_block`(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_mpfmatrix_block`(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void `_bncomp_mul_ddmatrix_strassen`(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ... Parallelized Strassen and Winograd matrix multiplication (limited performance due to poor coding)
- void `_bncomp_mul_tdmatrix_strassen`(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_qdmatrix_strassen`(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void `_bncomp_mul_mpfmatrix_strassen`(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)

3.7 Getting relative errors of vector and matrix

- void `relerr3_dvector`(double * max_relerr, double * min_relerr, double * norm_relerr, DVector vec, DVector vec_true, int kind_of_norm) ... Relative errors of vec.
- void `relerr3_ddvector`(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)
- void `relerr3_tdvector`(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVector vec_true, int kind_of_norm)
- void `relerr3_qdvector`(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm)
- void `relerr3_mpfvector`(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector vec, MPFVector vec_true, int kind_of_norm)
- void `relerr3_dmatrix`(double * max_relerr, double * min_relerr, double * norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm) ... Relative errors of mat.
- void `relerr3_ddmatrix`(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)

- void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)
- void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)
- void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)

3.8 Ozaki scheme and related functions

These following functions are necessary for Ozaki scheme and multi-fold precision arithmetic.

- void extract_dvector(DVector ret_high_vec, DVector ret_low_vec, DVector org_vec, double num_bits) ... $ret_high + ret_low = org_vec$
- void split_dmatrix(DMatrix ret_high_mat, DMatrix ret_low_mat, DMatrix org_mat) ... split org_mat to ret_high_mat and ret_low_mat in row direction
- void split_dmatrix_t(DMatrix ret_high_mat, DMatrix ret_low_mat, DMatrix org_mat) ... split org_mat to ret_high_mat and ret_low_mat in column direction
- int split_ddvector_dvec(DVector ret_vec[], int num_div, DDVector org_vec) ... Split org_vec to $ret_vec[0] + ret_vec[1] + \dots + ret_vec[num_div - 1]$
- int split_tdvector_dvec(DVector ret_vec[], int num_div, TDVector org_vec)
- void absmax_ddvector(double ret[DDSIZE], long int * max_index, DDVector vec) ... get absolutely maximum of elements in vec.
- void absmax_tdvector(double ret[TDSIZE], long int * max_index, TDVector vec)
- void absmax_qdvector(double ret[QDSIZE], long int * max_index, QDVector vec)
- void absmax_mpfvector(mpf_t ret, long int * max_index, MPFVector vec)
- void add_ddvector_dvec(DDVector c, DDVector a, DVector b) ... $c := a + (double)b$
- void add_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void add_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void add_mpfvector_dvec(MPFVector c, MPFVector a, DVector b) ;
- void sub_ddvector_dvec(DDVector c, DDVector a, DVector b) ... $c := a - (double)b$
- void sub_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void sub_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void sub_mpfvector_dvec(MPFVector c, MPFVector a, DVector b)
- void subst_dvector_qdvec(DVector c, QDVector a) ... $c := (DDVector)a$
- void absmax_row_ddmatrix(double mu[DDSIZE], long int * max_j, long int row_index, DDMatrix mat) ... Return the absolutely maximum element in the max_j-th row.
- void absmax_row_tdmatrix(double mu[TDSIZE], long int * max_j, long int row_index, TDMatrix mat)
- void absmax_row_qdmatrix(double mu[QDSIZE], long int * max_j, long int row_index, QDMatrix mat)
- void absmax_row_mpfmatrix(mpf_t mu, long int * max_j, long int row_index, MPFMatrix mat)
- void add_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) ... $c := a + (DMatrix)b$
- void add_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
- void add_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
- void add_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
- void sub_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) ... $c := a - (DMatrix)b$
- void sub_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
- void sub_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
- void sub_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
- int split_ddmatrix_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat) ... Split org_mat to ret_mat[] in row direction and return real number of divisions
- int split_tdmatrix_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)

- `int split_qdmatrix_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)`
- `int split_mpfmatrix_dmat(DMatrix ret_mat[], int num_div, MPFMatrix org_mat)`
- `void absmax_col_ddmatrix(double mu[DDSIZE], long int * max_i, long int col_index, DDMatrix mat)` ... return absolutely maximum element and its (i, j) -index in each column of mat.
- `void absmax_col_tdmatrix(double mu[TDSIZE], long int * max_i, long int col_index, TDMatrix mat)`
- `void absmax_col_qdmatrix(double mu[QDSIZE], long int * max_i, long int col_index, QDMatrix-mat)`
- `void absmax_col_mpfmatrix(mpf_t mu, long int * max_i, long int col_index, MPFMatrix mat)`
- `int split_ddmatrix_t_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat)` ... Split `org_mat` to `ret_mat[]` in column direction and return real number of divisions.
- `int split_tdmatrix_t_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)`
- `int split_qdvector_dvec(DVector ret_vec[], int num_div, QDVector org_vec)`
- `int split_qdmatrix_t_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)`
- `int split_mpfmatrix_t_dmat(DMatrix ret_mat[], int num_div, MPFMatrix org_mat)`
- `void subst_qdmatrix_dmat(QDMatrix c, DMatrix a)`
- `void subst_dmatrix_qdmat(DMatrix c, QDMatrix a)`
- `void mul_ddmatrix_oz(DDMatrix ret, DDMatrix a, int max_num_div_a, DDMatrix b, int max_num_div_b)` *cdots* Matrix multiplication based on Ozaki scheme
- `void mul_tdmatrix_oz(TDMatrix ret, TDMatrix a, int max_num_div_a, TDMatrix b, int max_num_div_b)`
- `void mul_qdmatrix_oz(QDMatrix ret, QDMatrix a, int max_num_div_a, QDMatrix b, int max_num_div_b)`
- `void mul_mpfmatrix_oz(MPFMatrix ret, MPFMatrix a, int max_num_div_a, MPFMatrix b, int max_num_div_b)`
- `void mul_ddmatrix_ddvec_oz(DDVector ret, DDMatrix a, int max_num_div_a, DDVector vb, int max_num_div_vb)` ... Matrix-Vector multiplication based on Ozaki scheme
- `void mul_tdmatrix_tdvec_oz(TDVector ret, TDMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)`
- `void mul_qdmatrix_qdvec_oz(QDVector ret, QDMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)`
- `void mul_mpfmatrix_dvec_oz(MPFVector ret, MPFMatrix a, int max_num_div_a, MPFVector vb, int max_num_div_vb)`

Chapter 4

LU decomposition

BNCmatmul has various LU decompositions and solvers for the corresponding decomposed linear system of equations.

4.1 List of functions

- `int DLUdecomp(DMatrix A)` ... LU decompose the matrix A without any types of pivoting.
- `int DDLUdecomp(DDMatrix A)`
- `int TDLUdecomp(TDMatrix A)`
- `int QDLUdecomp(QDMatrix A)`
- `int MPFLUdecomp(MPFMatrix A)`
- `int SolveDLS(DVector x, DMatrix LU, DVector b)` ... Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.
- `int SolveDDL(DDVector x, DDMatrix LU, DDVector b)`
- `int SolveTDL(TDVector x, TDMatrix LU, TDVector b)`
- `int SolveQDL(QDVector x, QDMatrix LU, QDVector b)`
- `int SolveMPFLS(MPFVector x, MPFMatrix LU, MPFVector b)`
- `int DLUdecompP(DMatrix A, long int ch[])` ... LU decompose the matrix A with partial pivoting, order of rows is stored in `ch[]`.
- `int DDLUdecompP(DDMatrix A, long int ch[])`
- `int TDLUdecompP(TDMatrix A, long int ch[])`
- `int QDLUdecompP(QDMatrix A, long int ch[])`
- `int MPFLUdecompP(MPFMatrix A, long int ch[])`
- `int SolveDLSP(DVector x, DMatrix LU, DVector b, long int ch[])` ... Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ using `ch[]` as row numbering.
- `int SolveDDLSP(DDVector x, DDMatrix LU, DDVector b, long int ch[])`
- `int SolveTDLSP(TDVector x, TDMatrix LU, TDVector b, long int ch[])`
- `int SolveQDLSP(QDVector x, QDMatrix LU, QDVector b, long int ch[])`
- `int SolveMPFLSP(MPFVector x, MPFMatrix LU, MPFVector b, long int ch[])`
- `int DLUdecompC(DMatrix A, long int row_ch[], long int col_ch[])` ... LU decompose the matrix A with complete pivoting, order of rows is stored in `row_ch[]`, and order of columns in `col_ch[]`.
- `int DDLUdecompC(DDMatrix A, long int row_ch[], long int col_ch[])`
- `int TDLUdecompC(TDMatrix A, long int row_ch[], long int col_ch[])`
- `int QDLUdecompC(QDMatrix A, long int row_ch[], long int col_ch[])`
- `int MPFLUdecompC(MPFMatrix A, long int row_ch[], long int col_ch[])`
- `int SolveDLSC(DVector x, DMatrix LU, DVector b, long int row_ch[], long int col_ch[])` ... Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ using `row_ch[]` and `col_ch[]` as row and column numbering, respectively.
- `int SolveDDLSC(DDVector x, DDMatrix LU, DDVector b, long int row_ch[], long int col_ch[])`
- `int SolveTDLSC(TDVector x, TDMatrix LU, TDVector b, long int row_ch[], long int col_ch[])`
- `int SolveQDLSC(QDVector x, QDMatrix LU, QDVector b, long int row_ch[], long int col_ch[])`
- `int SolveMPFLSC(MPFVector x, MPFMatrix LU, MPFVector b, long int row_ch[], long int col_ch[])`

- `int DLUdecomp_strassen(DMatrix A, long int min_dim)` ... LU decompose, using Strassen matrix multiplication.
- `int DDLUdecomp_strassen(DDMatrix A, long int min_dim)`
- `int TDLUdecomp_strassen(TDMatrix A, long int min_dim)`
- `int QDLUdecomp_strassen(QDMatrix A, long int min_dim)`
- `int MPFLUdecomp_strassen(MPFMatrix A, long int min_dim)`
- `int DLUdecomp_strassenPM(DMatrix A, long int ch[], long int min_dim)` ... LU decompose, using Strassen matrix multiplication, the matrix A with partial pivoting, order of rows is stored in `ch[]`.
- `int DDLUdecomp_strassenPM(DDMatrix A, long int ch[], long int min_dim)`
- `int TDLUdecomp_strassenPM(TDMatrix A, long int ch[], long int min_dim)`
- `int QDLUdecomp_strassenPM(QDMatrix A, long int ch[], long int min_dim)`
- `int MPFLUdecomp_strassenPM(MPFMatrix A, long int ch[], long int min_dim)`
- `int DLUdecomp_oz(DMatrix A, long int min_dim, int max_num_div)` ... LU decompose, using Ozaki scheme setting `max_num_div` as maximum number of divisions.
- `int DDLUdecomp_oz(DDMatrix A, long int min_dim, int max_num_div)`
- `int TDLUdecomp_oz(DMatrix A, long int min_dim, int max_num_div)`
- `int QDLUdecomp_oz(DMatrix A, long int min_dim, int max_num_div)`
- `int MPFLUdecomp_oz(MPFMatrix A, long int min_dim, int max_num_div)`
- `int DLUdecomp_ozPM(DMatrix A, long int ch[], long int min_dim, int max_num_div)` ... LU decompose, using partial pivoting and Ozaki scheme setting `max_num_div` as maximum number of divisions.
- `int DDLUdecomp_ozPM(DDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int TDLUdecomp_ozPM(TDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int QDLUdecomp_ozPM(QDMatrix A, long int ch[], long int min_dim, int max_num_div)`
- `int MPFLUdecomp_ozPM(MPFMatrix A, long int ch[], long int min_dim, int max_num_div)`

Chapter 5

Array, polynomial and solvers of algebraic equations

As application of multiple-precision arithmetic and linear computing described previously, we have introduced accelerated solvers of algebraic equations as presenting in this chapter.

5.1 Data types of Array and polynomial

Polynomials are represented as in equation (5.1):

$$p_n(x) = \sum_{k=0}^n a_k x^k \quad (5.1)$$

The library provides the following polynomial and array types, all declared in `poly.h` (copyright © 2002–2025 Tomonori Kouya; distributed under the GNU LGPL v3 or later).

5.1.1 Polynomial types

Float (FPoly) and float complex (CFPoly)

- `fpoly` — single-precision polynomial; pointer type `FPoly`. Members: `float *coef`, `long int deg`, `long int max_len`.
- `cfpoly` — single-precision complex polynomial; pointer type `CFPoly`. Members: `fcmplx *coef`, `long int deg`, `long int max_len`.

Double (DPoly) and double complex (CDPoly)

- `dpoly` — double-precision polynomial; pointer type `DPoly`. Members: `double *coef`, `long int deg`, `long int max_len`.
- `cdpoly` — double-precision complex polynomial; pointer type `CDPoly`. Members: `double _Complex *coef`, `long int deg`, `long int max_len`.

Double-double (DDPoly) and complex double-double (CDDPoly)

- `ddpoly` — double-double polynomial; pointer type `DDPoly`. Members: `ddfloat *coef`, `ddfloat zero`, `long int deg`, `long int max_len`.
- `cddpoly` — complex double-double polynomial; pointer type `CDDPoly`. Members: `cddfloat *coef`, `cddfloat zero`, `long int deg`, `long int max_len`.

Triple-double (TDPoly) and complex triple-double (CTDPoly)

- `tdpoly` — triple-double polynomial; pointer type `TDPoly`. Members: `tdfloat *coef`, `tdfloat zero`, `long int deg`, `long int max_len`.

- `ctdpoly` — complex triple-double polynomial; pointer type `CTDPoly`. Members: `ctdfloat *coef`, `ctdfloat zero`, `long int deg`, `long int max_len`.

Quadruple-double (`QDPoly`) and complex quadruple-double (`CQDPoly`)

- `qdpoly` — quadruple-double polynomial; pointer type `QDPoly`. Members: `qdfloat *coef`, `qdfloat zero`, `long int deg`, `long int max_len`.
- `cqdpoly` — complex quadruple-double polynomial; pointer type `CQDPoly`. Members: `cqdfloat *coef`, `cqdfloat zero`, `long int deg`, `long int max_len`.

Multiple-precision (`MPFPoly`) and complex multiple-precision (`CMPFPoly`)

Available when `USE_GMP` is defined.

- `mpfpoly` — GMP multiple-precision polynomial; pointer type `MPFPoly`. Members: `unsigned long int prec`, `mpf_t *coef`, `mpf_t zero`, `long int deg`, `long int max_len`.
- `cmpfpoly` — MPC complex multiple-precision polynomial; pointer type `CMPFPoly`. Members: `unsigned long int prec`, `mpc_t *coef`, `mpc_t zero`, `long int deg`, `long int max_len`.
- `_bncold_cmpfpoly` — legacy complex multiple-precision polynomial (old implementation); pointer type `_bncold_CMPFPoly`. Members: `unsigned long int prec`, `mpfcmplx *coef`, `long int deg`, `long int max_len`.

5.1.2 Array types

- `farray` — float array; pointer type `FArray`. Members: `float *array`, `long int size`.
- `cfarray` — float complex array; pointer type `CFArray`. Members: `fcmplx *array`, `long int size`.
- `darray` — double array; pointer type `DArray`. Members: `double *array`, `long int size`.
- `cdarray` — double complex array; pointer type `CDArray`. Members: `dcmplx *array`, `long int size`.
- `mpfarray` — GMP multiple-precision array (requires `USE_GMP`); pointer type `MPFArray`. Members: `unsigned long prec`, `mpf_t *array`, `long int size`.
- `cmpfarray` — MPC complex multiple-precision array (requires `USE_GMP`); pointer type `CMPFArray`. Members: `unsigned long prec`, `mpc_t *array`, `long int size`.
- `_bncold_cmpfarray` — legacy complex multiple-precision array (requires `USE_GMP`); pointer type `_bncold_CMPFArray`. Members: `unsigned long prec`, `mpfcmplx *array`, `long int size`.

5.2 List of functions

5.2.1 Float polynomial functions (`poly.c` — `FPoly`)

- `FPoly init_fpoly(long int max_len)`
- `void free_fpoly(FPoly pol)`
- `float get_fpoly_i(FPoly pol, long int i)` (alias: `gfpi`)
- `void set_fpoly_i(FPoly pol, long int i, float val)` (alias: `sfpi`)
- `long int setdegree_fpoly(FPoly pol)`
- `void print_fpoly(FPoly pol)`
- `void add_fpoly(FPoly ret, FPoly a, FPoly b)`
- `void sub_fpoly(FPoly ret, FPoly a, FPoly b)`
- `void cmul_fpoly(FPoly ret, float c, FPoly a)`
- `void subst_fpoly(FPoly ret, FPoly a)`
- `void set0_fpoly(FPoly pol)`
- `long int max_abscoef_fpoly(FPoly pol)`

- `long int num_nonzero_fpoly(FPoly pol)`
- `void diff_fpoly(FPoly pol)`
- `float eval_fpoly(FPoly pol, float x)`
- `float eval_diff_fpoly(FPoly pol, float x)`
- `void ceval_fpoly(FCmplx ret, FPoly pol, FCmplx x)`
- `void ceval_diff_fpoly(FCmplx ret, FPoly pol, FCmplx x)`

5.2.2 Double polynomial functions (`poly.c` — `DPoly`)

- `DPoly init_dpoly(long int max_len)`
- `void free_dpoly(DPoly pol)`
- `DPoly init_set_dpoly(DPoly org)`
- `double get_dpoly_i(DPoly pol, long int i)` (alias: `gdpi`)
- `void set_dpoly_i(DPoly pol, long int i, double val)` (alias: `sdpi`)
- `long int setdegree_dpoly(DPoly pol)`
- `void print_dpoly(DPoly pol)`
- `void add_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void sub_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void add2_dpoly(DPoly ret, DPoly a)`
- `void sub2_dpoly(DPoly ret, DPoly a)`
- `void mul_dpoly(DPoly ret, DPoly a, DPoly b)`
- `void cmul_dpoly(DPoly ret, double c, DPoly a)`
- `void subst_dpoly(DPoly ret, DPoly a)`
- `void set0_dpoly(DPoly pol)`
- `long int max_abscoef_dpoly(DPoly pol)`
- `long int num_nonzero_dpoly(DPoly pol)`
- `void diff_dpoly(DPoly pol)`
- `double eval_dpoly_horner(DPoly pol, double x)` (also aliased as `eval_dpoly`)
- `double eval_dpoly_estrin(DPoly pol, double x)`
- `double eval_diff_dpoly(DPoly pol, double x)`
- `void div_dpoly(DPoly quo, DPoly rem, DPoly a, DPoly b)`
- `void xpow_mul_dpoly(DPoly ret, long int n, DPoly a)`
- `void ceval_dpoly_horner(DCmplx ret, DPoly pol, DCmplx x)` (also aliased as `ceval_dpoly`)
- `void ceval_dpoly_estrin(DCmplx ret, DPoly pol, DCmplx x)`
- `void ceval_diff_dpoly(DCmplx ret, DPoly pol, DCmplx x)`
- `double _bncavx2_eval_dpoly_estrin(DPoly a, double x)` (AVX2)
- `void _bncavx2_ceval_dpoly_estrin(DCmplx ret, DPoly a, DCmplx x)` (AVX2)

5.2.3 Double complex polynomial functions (`poly.c` — `CDPoly`)

- `CDPoly init_cdpoly(long int max_len)`
- `void free_cdpoly(CDPoly pol)`
- `CDPoly init_set_cdpoly(CDPoly org)`
- `CDPoly init_set_cdpoly_dpoly(DPoly org)`
- `void set_cdpoly_dpoly(CDPoly ret, DPoly src)`
- `double _Complex get_cdpoly_i(CDPoly pol, long int i)` (alias: `gcdpi`)
- `void set_cdpoly_i(CDPoly pol, long int i, double _Complex val)` (alias: `scdpi`)
- `void set_cdpoly_i_d(CDPoly pol, long int i, double val)`
- `long int setdegree_cdpoly(CDPoly pol)`
- `void print_cdpoly(CDPoly pol)`
- `void add_cdpoly(CDPoly ret, CDPoly a, CDPoly b)`
- `void sub_cdpoly(CDPoly ret, CDPoly a, CDPoly b)`

- void add2_cdpoly(CDPoly ret, CDPoly a)
- void sub2_cdpoly(CDPoly ret, CDPoly a)
- void mul_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void mul2_cdpoly(CDPoly ret, CDPoly a)
- void cmul_cdpoly(CDPoly ret, double _Complex c, CDPoly a)
- void subst_cdpoly(CDPoly ret, CDPoly a)
- void set0_cdpoly(CDPoly pol)
- long int max_abscoef_cdpoly(CDPoly pol)
- long int num_nonzero_cdpoly(CDPoly pol)
- void diff_cdpoly(CDPoly pol)
- double _Complex eval_cdpoly_horner(CDPoly a, double _Complex x) (also aliased as eval_cdpoly)
- double _Complex eval_cdpoly_estrin(CDPoly a, double _Complex x)
- double _Complex eval_diff_cdpoly(CDPoly a, double _Complex x)
- void div_cdpoly(CDPoly quo, CDPoly rem, CDPoly a, CDPoly b)
- void xpow_mul_cdpoly(CDPoly ret, long int n, CDPoly a)

5.2.4 Double-double polynomial functions (poly.c — DDPoly, CDDPoly)

- DDPoly init_ddpoly(long int max_len)
- void free_ddpoly(DDPoly pol)
- double * get_ddpoly_i(DDPoly pol, long int i) (alias: gddpi)
- void set_ddpoly_i(DDPoly pol, long int i, double * val) (alias: sddpi)
- void set_ddpoly_i_si(DDPoly pol, long int i, long val)
- void set_ddpoly_i_ui(DDPoly pol, long int i, unsigned long val)
- void set_ddpoly_i_d(DDPoly pol, long int i, double val)
- void set_ddpoly_i_str(DDPoly pol, long int i, const char * str, int base)
- long int setdegree_ddpoly(DDPoly pol)
- void print_ddpoly(DDPoly pol)
- void add_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void sub_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void mul_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void cmul_ddpoly(DDPoly ret, double * c, DDPoly a)
- void subst_ddpoly(DDPoly ret, DDPoly a)
- void set0_ddpoly(DDPoly pol)
- long int max_abscoef_ddpoly(DDPoly pol)
- long int num_nonzero_ddpoly(DDPoly pol)
- void diff_ddpoly(DDPoly pol)
- void eval_ddpoly_horner(double * ret, DDPoly a, double * x) (also aliased as eval_ddpoly)
- void eval_ddpoly_estrin(double * ret, DDPoly a, double * x)
- void eval_diff_ddpoly(double * ret, DDPoly a, double * x)
- void ceval_ddpoly_horner(cddfloat * ret, DDPoly a, cddfloat * x) (also aliased as ceval_ddpoly)
- void ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x)
- void ceval_diff_ddpoly(cddfloat * ret, DDPoly a, cddfloat * x)
- void _bncavx2_eval_ddpoly_estrin(double ret[DDSIZE], DDPoly a, double x[DDSIZE]) (AVX2)
- void _bncavx2_ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x) (AVX2)

Complex double-double polynomial (CDDPoly):

- CDDPoly init_cddpoly(long int max_len)
- void free_cddpoly(CDDPoly pol)
- CDDPoly init_set_cddpoly(CDDPoly org)
- CDDPoly init_set_cddpoly_ddpoly(DDPoly org)

- `cddfloat * get_cddpoly_i(CDDPoly pol, long int i)` (alias: `gcddpi`)
- `void set_cddpoly_i(CDDPoly pol, long int i, cddfloat * val)` (alias: `scddpi`)
- `long int setdegree_cddpoly(CDDPoly pol)`
- `void print_cddpoly(CDDPoly pol)`
- `void add_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)`
- `void sub_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)`
- `void mul_cddpoly(CDDPoly ret, CDDPoly a, CDDPoly b)`
- `void subst_cddpoly(CDDPoly ret, CDDPoly a)`
- `void set0_cddpoly(CDDPoly pol)`
- `long int max_abscoef_cddpoly(CDDPoly pol)`
- `long int num_nonzero_cddpoly(CDDPoly pol)`
- `void diff_cddpoly(CDDPoly pol)`
- `void eval_cddpoly_horner(cddfloat * ret, CDDPoly a, cddfloat * x)` (also aliased as `eval_cddpoly`)
- `void eval_cddpoly_estrin(cddfloat * ret, CDDPoly a, cddfloat * x)`
- `void eval_diff_cddpoly(cddfloat * ret, CDDPoly a, cddfloat * x)`
- `void _bncavx2_eval_cddpoly_estrin(cddfloat * ret, CDDPoly a, cddfloat * x)` (AVX2)

5.2.5 Triple-double polynomial functions (`poly.c` — `TDPoly`)

- `TDPoly init_tdpoly(long int max_len)`
- `void free_tdpoly(TDPoly pol)`
- `tdfloat get_tdpoly_i_float(TDPoly pol, long int i)`
- `double * get_tdpoly_i(TDPoly pol, long int i)` (alias: `gtdpi`)
- `void set_tdpoly_i(TDPoly pol, long int i, double val[TDSIZE])` (alias: `stdpi`)
- `void set_tdpoly_i_si(TDPoly pol, long int i, long val)`
- `void set_tdpoly_i_ui(TDPoly pol, long int i, unsigned long val)`
- `void set_tdpoly_i_d(TDPoly pol, long int i, double val)`
- `void set_tdpoly_i_str(TDPoly pol, long int i, const char * str, int base)`
- `long int setdegree_tdpoly(TDPoly pol)`
- `void print_tdpoly(TDPoly pol)`
- `void add_tdpoly(TDPoly c, TDPoly a, TDPoly b) ...  $c = a + b$`
- `void add2_tdpoly(TDPoly c, TDPoly a) ...  $c += a$`
- `void sub_tdpoly(TDPoly c, TDPoly a, TDPoly b) ...  $c = a - b$`
- `void sub2_tdpoly(TDPoly c, TDPoly a) ...  $c -= a$`
- `void mul_tdpoly(TDPoly c, TDPoly a, TDPoly b) ...  $c = a \times b$`
- `void cmul_tdpoly(TDPoly c, double val[TDSIZE], TDPoly a) ...  $c = val \times a$`
- `void cmul2_tdpoly(TDPoly c, double val[TDSIZE]) ...  $c *= val$`
- `void subst_tdpoly(TDPoly c, TDPoly a) ...  $c := a$`
- `void set0_tdpoly(TDPoly c) ...  $c := 0$`
- `long int num_nonzero_tdpoly(TDPoly c)`
- `long int max_abscoef_tdpoly(TDPoly c)`
- `void diff_tdpoly(TDPoly a) ...  $a := a'(x)$`
- `void eval_tdpoly_horner(double ret[TDSIZE], TDPoly a, double x[TDSIZE])` (also aliased as `eval_tdpoly`)
- `void eval_tdpoly_estrin(double ret[TDSIZE], TDPoly a, double x[TDSIZE])`
- `void eval_diff_tdpoly(double ret[TDSIZE], TDPoly a, double x[TDSIZE])`
- `void ceval_tdpoly_horner(ctdfloat * ret, TDPoly a, ctdfloat * x)` (also aliased as `ceval_tdpoly`)
- `void ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x)`
- `void ceval_diff_tdpoly(ctdfloat * ret, TDPoly a, ctdfloat * x)`
- `void _bncavx2_eval_tdpoly_estrin(double ret[TDSIZE], TDPoly a, double x[TDSIZE])` (AVX2)
- `void _bncavx2_ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x)` (AVX2)

5.2.6 Quadruple-double polynomial functions (poly.c — QDPoly)

- QDPoly init_qdpoly(long int max_len)
- void free_qdpoly(QDPoly pol)
- qdfloat get_qdpoly_i_float(QDPoly pol, long int i)
- double * get_qdpoly_i(QDPoly pol, long int i) (alias: gqdpi)
- void set_qdpoly_i(QDPoly pol, long int i, double val[QDSIZE]) (alias: sqdpi)
- void set_qdpoly_i_si(QDPoly pol, long int i, long val)
- void set_qdpoly_i_ui(QDPoly pol, long int i, unsigned long val)
- void set_qdpoly_i_d(QDPoly pol, long int i, double val)
- void set_qdpoly_i_str(QDPoly pol, long int i, const char * str, int base)
- long int setdegree_qdpoly(QDPoly pol)
- void print_qdpoly(QDPoly pol)
- void add_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\dots c = a + b$
- void add2_qdpoly(QDPoly c, QDPoly a) $\dots c += a$
- void sub_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\dots c = a - b$
- void sub2_qdpoly(QDPoly c, QDPoly a) $\dots c -= a$
- void mul_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\dots c = a \times b$
- void cmul_qdpoly(QDPoly c, double val[QDSIZE], QDPoly a) $\dots c = val \times a$
- void cmul2_qdpoly(QDPoly c, double val[QDSIZE]) $\dots c *= val$
- void subst_qdpoly(QDPoly c, QDPoly a) $\dots c := a$
- void set0_qdpoly(QDPoly c) $\dots c := 0$
- long int num_nonzero_qdpoly(QDPoly c)
- long int max_abscoef_qdpoly(QDPoly c)
- void diff_qdpoly(QDPoly a) $\dots a := a'(x)$
- void eval_qdpoly_horner(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (also aliased as eval_qdpoly)
- void eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void eval_diff_qdpoly(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void ceval_qdpoly_horner(cqdfloat * ret, QDPoly a, cqdfloat * x) (also aliased as ceval_qdpoly)
- void ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_diff_qdpoly(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_hes_qdmatrix(cqdfloat * ret, QDMatrix hes, cqdfloat * x)
- void _bncavx2_eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (AVX2)
- void _bncavx2_ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x) (AVX2)

Conversion functions between polynomial types (requires USE_GMP):

- void set_ddpoly_i_mpf(DDPoly pol, long int i, mpf_t val)
- void set_tdpoly_i_mpf(TDPoly pol, long int i, mpf_t val)
- void set_qdpoly_i_mpf(QDPoly pol, long int i, mpf_t val)
- void set_cddpoly_i_mpf(CDDPoly pol, long int i, mpc_t val)
- DDPoly init_set_ddpoly_mpfpoly(MPFPoly org_pol)
- TDPoly init_set_tdpoly_mpfpoly(MPFPoly org_pol)
- QDPoly init_set_qdpoly_mpfpoly(MPFPoly org_pol)
- CDDPoly init_set_cddpoly_cmpfpoly(CMPFPoly org_pol)

5.2.7 Multiple-precision polynomial functions (poly.c — MPFPoly; requires USE_GMP)

- MPFPoly init_mpfpoly(long int max_len)
- MPFPoly init2_mpfpoly(long int max_len, unsigned long prec)
- void free_mpfpoly(MPFPoly pol)

- `mpf_ptr get_mpfpoly_i(MPFPoly pol, long int i)` (alias: `gmpfpi`)
- `void set_mpfpoly_i(MPFPoly pol, long int i, mpf_t val)` (alias: `smpfpi`)
- `void set_mpfpoly_i_d(MPFPoly pol, long int i, double val)`
- `void set_mpfpoly_i_si(MPFPoly pol, long int i, long val)`
- `void set_mpfpoly_i_ui(MPFPoly pol, long int i, unsigned long val)`
- `void set_mpfpoly_i_str(MPFPoly pol, long int i, const char * str, int base)`
- `void print_mpfpoly(MPFPoly pol)`
- `void print_fdmfpoly(FPoly fp, DPoly dp, MPFPoly mp)`
- `void add_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)`
- `void sub_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)`
- `void add2_mpfpoly(MPFPoly ret, MPFPoly a)`
- `void sub2_mpfpoly(MPFPoly ret, MPFPoly a)`
- `void mul_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)`
- `void cmul_mpfpoly(MPFPoly ret, mpf_t c, MPFPoly a)`
- `void subst_mpfpoly(MPFPoly ret, MPFPoly a)`
- `void set0_mpfpoly(MPFPoly pol)`
- `long int max_abscoef_mpfpoly(MPFPoly pol)`
- `long int num_nonzero_mpfpoly(MPFPoly pol)`
- `long int setdegree_mpfpoly(MPFPoly pol)`
- `void diff_mpfpoly(MPFPoly pol)`
- `void eval_mpfpoly_horner(mpf_t ret, MPFPoly pol, mpf_t x)` (also aliased as `eval_mpfpoly`)
- `void eval_mpfpoly_estrin(mpf_t ret, MPFPoly pol, mpf_t x)`
- `void eval_diff_mpfpoly(mpf_t ret, MPFPoly pol, mpf_t x)`
- `void ceval_mpfpoly_horner(mpc_t ret, MPFPoly pol, mpc_t x)` (also aliased as `ceval_mpfpoly`)
- `void ceval_mpfpoly_estrin(mpc_t ret, MPFPoly pol, mpc_t x)`
- `void ceval_diff_mpfpoly(mpc_t ret, MPFPoly pol, mpc_t x)`
- `void ceval_hes_mpfmatrix(mpc_t ret, MPFMatrix hes, mpc_t x)`

Complex multiple-precision polynomial (CMPFPoly; requires `USE_GMP`):

- `CMPFPoly init_cmpfpoly(long int max_len)`
- `CMPFPoly init2_cmpfpoly(long int max_len, unsigned long prec)`
- `void free_cmpfpoly(CMPFPoly pol)`
- `mpc_ptr get_cmpfpoly_i(CMPFPoly pol, long int i)` (alias: `gcmpfpi`)
- `void set_cmpfpoly_i(CMPFPoly pol, long int i, mpc_t val)` (alias: `scmpfpi`)
- `void set_cmpfpoly_i_cd(CMPFPoly pol, long int i, double _Complex val)`
- `void set_cmpfpoly_i_d(CMPFPoly pol, long int i, double val)`
- `void set_cmpfpoly_i_si_si(CMPFPoly pol, long int i, long re, long im)`
- `void set_cmpfpoly_i_si(CMPFPoly pol, long int i, long val)`
- `void set_cmpfpoly_i_ui(CMPFPoly pol, long int i, unsigned long val)`
- `void set_cmpfpoly_i_str(CMPFPoly pol, long int i, const char * str, int base)`
- `long int setdegree_cmpfpoly(CMPFPoly pol)`
- `void print_cmpfpoly(CMPFPoly pol)`
- `void subst_cmpfpoly_mfpoly(CMPFPoly ret, MPFPoly pol)`
- `CMPFPoly init_set_cmpfpoly_mfpoly(MPFPoly org)`
- `CMPFPoly init_set_cmpfpoly(CMPFPoly org)`
- `void add_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)`
- `void sub_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)`
- `void add2_cmpfpoly(CMPFPoly ret, CMPFPoly a)`
- `void sub2_cmpfpoly(CMPFPoly ret, CMPFPoly a)`
- `void mul_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)`
- `void cmul_cmpfpoly(CMPFPoly ret, mpc_t c, CMPFPoly a)`
- `void subst_cmpfpoly(CMPFPoly ret, CMPFPoly a)`
- `void set0_cmpfpoly(CMPFPoly pol)`

- long int max_abscoef_cmpfpoly(CMPFPoly pol)
- long int num_nonzero_cmpfpoly(CMPFPoly pol)
- void diff_cmpfpoly(CMPFPoly pol)
- void eval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x) (also aliased as eval_cmpfpoly)
- void eval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void eval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_hes_cmpfmatrix(mpc_t ret, CMPFMatrix hes, mpc_t x)

5.2.8 Array functions (array.c)

- FArray init_farray(long int size)
- void free_farray(FArray a)
- float get_farray_i(FArray a, long int i) (alias: gfai)
- void set_farray_i(FArray a, long int i, float val) (alias: sfai)
- void subst_farray(FArray ret, FArray src)
- void print_farray(FArray a)
- CArray init_carray(long int size)
- void free_carray(CArray a)
- FCmplx get_carray_i(CArray a, long int i) (alias: gcgai)
- void set_carray_i(CArray a, long int i, FCmplx val) (alias: scgai)
- void subst_carray(CArray ret, CArray src)
- void print_carray(CArray a)
- DArray init_darray(long int size)
- void free_darray(DArray a)
- double get_darray_i(DArray a, long int i) (alias: gdai)
- void set_darray_i(DArray a, long int i, double val) (alias: sdai)
- void subst_darray(DArray ret, DArray src)
- void print_darray(DArray a)
- CArray init_cdarray(long int size)
- void free_cdarray(CArray a)
- DCmplx get_cdarray_i(CArray a, long int i) (alias: gcdai)
- void set_cdarray_i(CArray a, long int i, DCmplx val) (alias: scdai)
- void subst_cdarray(CArray ret, CArray src)
- void print_cdarray(CArray a)

Multiple-precision array functions (requires USE_GMP):

- MPFArray init_mpfarray(long int size)
- MPFArray init2_mpfarray(long int size, unsigned long prec)
- void free_mpfarray(MPFArray a)
- mpf_ptr get_mpfarray_i(MPFArray a, long int i) (alias: gmpgai)
- void set_mpfarray_i(MPFArray a, long int i, mpf_t val) (alias: smpgai)
- void set_mpfarray_i_d(MPFArray a, long int i, double val)
- void set_mpfarray_i_ui(MPFArray a, long int i, unsigned long val)
- void subst_mpfarray(MPFArray ret, MPFArray src)
- void print_mpfarray(MPFArray a)
- CMPFArray init_cmpfarray(long int size)
- CMPFArray init2_cmpfarray(long int size, unsigned long prec)
- void free_cmpfarray(CMPFArray a)
- mpc_ptr get_cmpfarray_i(CMPFArray a, long int i) (alias: gcmpgai)

- void set_cmpfarray_i(CMPFArray a, long int i, mpc_t val) (alias: scmpfai)
- void set_cmpfarray_i_d(CMPFArray a, long int i, double val)
- void set_cmpfarray_i_ui(CMPFArray a, long int i, unsigned long val)
- void set_cmpfarray_i_real(CMPFArray a, long int i, mpf_t val)
- void subst_cmpfarray(CMPFArray ret, CMPFArray src)
- void print_cmpfarray(CMPFArray a)

5.2.9 DKA root-finding methods (dka.c)

The Durand–Kerner–Aberth (DKA) method simultaneously finds all roots of a polynomial.

- float fdka_center(FPoly pol)
- float fdka_radius(FPoly pol)
- void fdka_init(CFArray x, FPoly pol)
- long int fdka(CFArray ans, CFArray x_init, FPoly pol, long int maxtimes, float aeps, float reps)
- double ddka_center(DPoly pol)
- double __Complex cddka_center(CDPoly pol)
- double ddka_radius(DPoly pol)
- double cddka_radius(CDPoly pol)
- void ddka_init(CDArray x, DPoly pol)
- void cddka_init(double __Complex[] x, CDPoly pol)
- void cddka_init_cdarray(CDArray x, CDPoly pol)
- long int ddka(CDArray ans, CDArray x_init, DPoly pol, long int maxtimes, double aeps, double reps)
- long int cddka(double __Complex[] ans, double __Complex[] x_init, CDPoly pol, long int maxtimes, double aeps, double reps)

Double-double DKA (dd_dka.c):

- void dd_dka_center(double ret[DDSIZE], DDPoly pol)
- void dd_dka_radius(double ret[DDSIZE], DDPoly pol)
- void dd_dka_init(CDDVector x, DDPoly pol)
- long int dd_dka(CDDVector ans, CDDVector x_init, DDPoly pol, long int maxtimes, double * aeps, double * reps)

Triple-double DKA (td_dka.c):

- void td_dka_center(double ret[TDSIZE], TDPoly pol)
- void td_dka_radius(double ret[TDSIZE], TDPoly pol)
- void td_dka_init(CTDVector x, TDPoly pol)
- long int td_dka(CTDVector ans, CTDVector x_init, TDPoly pol, long int maxtimes, double * aeps, double * reps)

Quadruple-double DKA (qd_dka.c):

- void qd_dka_center(double ret[QDSIZE], QDPoly pol)
- void qd_dka_radius(double ret[QDSIZE], QDPoly pol)
- void qd_dka_init(CQDVector x, QDPoly pol)
- long int qd_dka(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double * aeps, double * reps)
- long int qd_dka_mod(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_petckovic(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_aberth(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])

Multiple-precision DKA (`mpf_dka.c`; requires `USE_GMP`):

- `void mpf_dka_center(mpf_t ret, MPFPoly pol)`
- `void mpf_dka_radius(mpf_t ret, MPFPoly pol)`
- `void mpf_dka_ozawa_radius(mpf_t ret, MPFPoly pol) ... Ozawa initial radius:  $r := (p(\text{center})/a_n)^{1/n}$`
- `void mpf_dka_init(CMPFArray x, MPFPoly pol)`
- `void mpf_dka_init2(CMPFArray x_init, MPFPoly pol, void (*) get_radius, void (*) get_center)`
- `long int mpf_dka(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpf_dka_mod(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) ... DKA method, 2nd order`
- `long int mpf_petkovic(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) ... Petkovic method, 3rd order`
- `long int mpf_aberth(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) ... Aberth method, 3rd order`

MPC complex multiple-precision DKA (`mpc_dka.c`; requires `USE_GMP`):

- `void mpc_dka_center(mpc_t ret, CMPFPoly pol)`
- `void mpc_dka_radius(mpf_t ret, CMPFPoly pol)`
- `void mpc_dka_ozawa_radius(mpf_t ret, CMPFPoly pol)`
- `void mpc_dka_init(CMPFArray x, CMPFPoly pol)`
- `void mpc_dka_init2(CMPFArray x_init, CMPFPoly pol, void (*) get_radius, void (*) get_center)`
- `long int mpc_dka(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_dka_mod(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_petkovic(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`
- `long int mpc_aberth(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)`

5.2.10 Hirano method (`hirano.c`, `dd_hirano.c`)

The Hirano method finds a root x satisfying $|p(x)| < \max_j |a_j x^j| \cdot r_\varepsilon + a_\varepsilon$.

- `double _Complex dhorner(double _Complex x, DPoly pol) ... Horner evaluation of  $p(x)$`
- `double _Complex cdhorner(double _Complex x, CDPoly pol)`
- `void dcoef_horner(double _Complex[] ret, double _Complex x, DPoly pol) ... coefficients of  $p(x + d)$`
- `void cdcoef_horner(double _Complex[] ret, double _Complex x, CDPoly pol)`
- `long int absmax_dpoly(double * absmax, double _Complex x, DPoly pol)`
- `long int absmax_cdpoly(double * absmax, double _Complex x, CDPoly pol)`
- `double dget_plus_arg(double _Complex x) ...  $\arg(x) \in [0, 2\pi]$`
- `long int dget_nearest_int(double x)`
- `double _Complex dget_min_branch(double _Complex x, double mu, double _Complex[] coef, long int i_num, long int i_den)`
- `long int dhirano(double _Complex * ret, double _Complex init_x, DPoly pol, double reps, double aeps, long int maxtimes)`
- `long int cdhirano(double _Complex * ret, double _Complex init_x, CDPoly pol, double reps, double aeps, long int maxtimes)`

Multiple-precision Hirano (`hirano.c`; requires `USE_GMP`):

- `void mpf_coef_horner(CMPFArray ret, mpc_t x, MPFPoly pol)`
- `void mpc_coef_horner(CMPFArray ret, mpc_t x, CMPFPoly pol)`
- `long int absmax_mpfpoly(mpf_t absmax, mpc_t x, MPFPoly pol)`
- `long int absmax_cmpfpoly(mpf_t absmax, mpc_t x, CMPFPoly pol)`
- `void mpf_get_plus_arg(mpf_t ret, mpc_t x)`
- `void mpf_get_nearest_int(mpf_t ret, mpf_t x)`
- `void mpf_get_min_branch(mpc_t ret, mpc_t x, mpf_t mu, CMPFArray coef, long int i_num, long int i_den)`
- `long int mpf_hirano(mpc_t ret, mpc_t init_x, MPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int mpc_hirano(mpc_t ret, mpc_t init_x, CMPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `void ono_poly(MPFPoly pol, long int deg)` ... Ono's problem polynomial
- `void wilkinson_poly(MPFPoly ret, long int n)` ... Wilkinson polynomial $(x-1)(x-2)\cdots(x-n)$
- `void deflation_cmpfpoly(CMPFPoly ret, CMPFPoly pol, mpc_t root)` ... deflation $p(x)/(x-r)$

Double-double Hirano (`dd_hirano.c`):

- `void dd_horner(cddfloat * ret, cddfloat * x, DDPoly pol)`
- `void dd_coef_horner(CDDVector ret, cddfloat * x, DDPoly pol)`
- `void cdd_horner(cddfloat * ret, cddfloat * x, CDDPoly pol)`
- `void cdd_coef_horner(CDDVector ret, cddfloat * x, CDDPoly pol)`
- `long int absmax_ddpoly(double absmax[DDSIZE], cddfloat * x, DDPoly pol)`
- `long int absmax_cddpoly(double absmax[DDSIZE], cddfloat * x, CDDPoly pol)`
- `void dd_get_plus_arg(double ret[DDSIZE], cddfloat * x)`
- `void dd_get_nearest_int(double ret[DDSIZE], double x[DDSIZE])`
- `void dd_get_min_branch(cddfloat * ret, cddfloat * x, double mu[DDSIZE], CDDVector coef, long int i_num, long int i_den)`
- `long int dd_hirano(cddfloat * ret, cddfloat * init_x, DDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)`
- `long int cdd_hirano(cddfloat * ret, cddfloat * init_x, CDDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)`
- `void deflation_cddpoly(CDDPoly ret, CDDPoly pol, cddfloat * root)`

5.2.11 File I/O functions (`fread_write_complex.c`; requires `USE_GMP`)

- `void fread_cmpfarray(FILE * fp, CMPFArray array)`
- `void fread_cmpfarray_fname(const char * fname, CMPFArray array)`
- `void fwrite_cmpfarray(FILE * fp, CMPFArray array)`
- `void fwrite_cmpfarray_fname(const char * fname, CMPFArray array)`
- `void fread_mpfpolycoef(FILE * fp, MPFPoly pol, long int maxdeg)`

Chapter 6

Sparse matrix and related functions

This chapter describes the sparse matrix types and associated functions provided by the library. Sparse matrices are stored in Compressed Row Storage (CRS) format. For each matrix type, the library provides real and complex variants at double, double-double (DD), triple-double (TD), quadruple-double (QD), and multiple-precision (MPF, via GMP) precisions.

6.1 Structures of CRS matrix

6.1.1 DRSMMatrix — double-precision sparse matrix

The `drsmatrix` structure stores a real double-precision sparse matrix in CRS format. Its pointer type is `DRSMMatrix`. Key members are:

- `double *element` — array of non-zero elements
- `long int row_dim, col_dim` — row and column dimensions
- `long int **nzero_index` — column indices of non-zero elements
- `long int *nzero_col_dim` — number of non-zero elements in each row
- `long int *nzero_row_dim` — number of non-zero elements in each column
- `long int nzero_total_num` — total number of non-zero elements
- `long int real_nzero_total_num` — SIMD-aligned total; equals $\sum_i \text{real_nzero_col_dim}[i]$
- `long int *real_nzero_col_dim` — SIMD-aligned row non-zero counts (padded to `_BNC_D_WIDTH`)
- `double zero_element` — zero sentinel value

Note that `_BNC_D_WIDTH` (and `_BNC_S_WIDTH` for the float-based types) depends on the SIMD backend the library was built for: 4/8 under AVX2, 8/16 under AVX-512, 2/4 under NEON and SVE2, and 1/1 in the serial build. The padding therefore costs more in a wider build, and for matrices whose rows hold fewer non-zeros than the vector width the padded lanes can dominate: with a half-bandwidth of 3 (7 non-zeros per row) the AVX-512 SpMV is measurably *slower* than the serial one. See Sec. 1.3 for the measurements and for guidance on which library to link against.

Functions for `DRSMMatrix`:

- `DRSMMatrix init_drsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_drsmatrix(DRSMMatrix mat)`
- `void set_nzero_row_dim(DRSMMatrix mat)`
- `void print_drsmatrix(DRSMMatrix mat)`
- `void set_dmatrix_drsmatrix(DMatrix ret, DRSMMatrix spmat) ... Dense := Sparse`
- `DRSMMatrix init_set_drsmatrix_dmatrix(DMatrix org)`
- `double get_drsmatrix_ij(DRSMMatrix mat, long int i, long int j)`
- `void set_drsmatrix_ij(DRSMMatrix mat, long int i, long int j, double val)`
- `DRSMMatrix init_set_drsmatrix(DRSMMatrix org)`
- `void set0_drsmatrix(DRSMMatrix spmat)`
- `int get_vars_drsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int *`

- `nzero_total_num, const char * fname)`
- `int fread_urilinkdat_fname(DRSMMatrix mat, const char * fname)`
- `int mul_drsmatrix_dvec(DVector ret, DRSMMatrix mat, DVector vec)`
- `int mul_drsmatrixt_dvec(DVector ret, DRSMMatrix mat, DVector vec) ... transposed`
- `void mul_drsmatrix_dvec_oz(DVector ret, DRSMMatrix a, int max_num_div_a, DVector vb, int max_num_div_vb) ... Ozaki scheme`
- `void mul_drsmatrixt_dvec_oz(DVector ret, DRSMMatrix a, int max_num_div_a, DVector vb, int max_num_div_vb) ... transposed, Ozaki scheme`
- `double dpower_rsmatrix(DVector evec, DRSMMatrix mat, double reps, double aeaps, long int maxtimes) ... power method`
- `int smul_dvector(DVector ret, double c, DVector vec)`
- `long int absmax_index_dvector(double * absmax, DVector vec)`
- `void subst_drsmatrix(DRSMMatrix ret, DRSMMatrix a) ...  $ret := a$`
- `void add_drsmatrix(DRSMMatrix ret, DRSMMatrix a, DRSMMatrix b) ...  $ret := a + b$`
- `void sub_drsmatrix(DRSMMatrix ret, DRSMMatrix a, DRSMMatrix b) ...  $ret := a - b$`
- `void cmul_drsmatrix(DRSMMatrix ret, double c, DRSMMatrix a) ...  $ret := c \cdot a$`
- `double absmax_row_drsmatrix(long int * max_j, long int row, DRSMMatrix mat)`
- `double absmax_col_drsmatrix(long int * max_i, long int col, DRSMMatrix mat)`
- `double normf_drsmatrix(DRSMMatrix mat) ... Frobenius norm`
- `void split_drsmatrix(DRSMMatrix high, DRSMMatrix low, DRSMMatrix org) ... SplitMat_A`
- `int split_drsmatrix_drsmat(DRSMMatrix[] ret, int num_div, DRSMMatrix org) ... SplitMat_A (multiple parts)`
- `int split_drsmatrix_t_drsmat(DRSMMatrix[] ret, int num_div, DRSMMatrix org) ... SplitMat_B`
- `void iLU0_drsmatrix(DRSMMatrix mat) ... incomplete LU decomposition (iLU0)`
- `void solve_iLU0_drsmatrix(DVector ret, DRSMMatrix ilu, DVector b) ... solve  $(iLU)x = b$`
- `void solve_iLU0t_drsmatrix(DVector ret, DRSMMatrix ilu, DVector b) ... solve  $x^T(iLU) = b^T$`

6.1.2 CDRSMMatrix — complex double-precision sparse matrix

The `cdrsmatrix` structure stores a complex sparse matrix as a pair of real `DRSMMatrix` parts. Its pointer type is `CDRSMMatrix`. Members: `DRSMMatrix re` (real part), `DRSMMatrix im` (imaginary part).

- `CDRSMMatrix init_cdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_cdrsmatrix(CDRSMMatrix mat)`
- `void print_cdrsmatrix(CDRSMMatrix mat)`
- `void set_cdmatrix_cdrsmatrix(CDMatrix ret, CDRSMMatrix spmat)`
- `double normf_cdrsmatrix(CDRSMMatrix mat) ... Frobenius norm`
- `CDRSMMatrix init_set_cdrsmatrix_cdmatrix(CDMatrix org)`
- `CDRSMMatrix init_set_cdrsmatrix(CDRSMMatrix org)`
- `double _Complex get_cdrsmatrix_ij(CDRSMMatrix mat, long int i, long int j)`
- `void set_cdrsmatrix_ij(CDRSMMatrix mat, long int i, long int j, double _Complex val)`
- `int mul_cdrsmatrix_cdvec(CDVector ret, CDRSMMatrix mat, CDVector vec)`
- `int mul_cdrsmatrixt_cdvec(CDVector ret, CDRSMMatrix mat, CDVector vec) ... transposed`
- `int mul_cdrsmatrixs_cdvec(CDVector ret, CDRSMMatrix mat, CDVector vec) ... conjugate-transposed`
- `void iLU0_cdrsmatrix(CDRSMMatrix mat)`
- `void solve_iLU0_cdrsmatrix(CDVector ret, CDRSMMatrix ilu, CDVector b)`
- `void solve_iLU0t_cdrsmatrix(CDVector ret, CDRSMMatrix ilu, CDVector b)`
- `void solve_iLU0s_cdrsmatrix(CDVector ret, CDRSMMatrix ilu, CDVector b)`

6.1.3 DDRSMatix and CDDRSMatix — double-double sparse matrices

The `ddrsmatrix` structure stores a double-double sparse matrix; pointer type `DDRSMatix`. Its elements are stored as double `*element[DDSIZE]`. The complex variant `CDDRSMatix` holds a pair of `DDRSMatix` parts.

- `DDRSMatix init_ddrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_ddrsmatrix(DDRSMatix mat)`
- `void set_nzero_row_dim_dd(DDRSMatix mat)`
- `void print_ddrsmatrix(DDRSMatix mat)`
- `void normf_ddrsmatrix(double ret[DDSIZE], DDRSMatix mat)`
- `DDRSMatix init_set_ddrsmatrix_ddmatrix(DDMatrix org)`
- `void get_ddrsmatrix_ij(double ret[DDSIZE], DDRSMatix mat, long int i, long int j)`
- `void set_ddrsmatrix_ij(DDRSMatix mat, long int i, long int j, double val[DDSIZE])`
- `DDRSMatix init_set_ddrsmatrix(DDRSMatix org)`
- `void set0_ddrsmatrix(DDRSMatix spmat)`
- `void subst_ddrsmatrix(DDRSMatix ret, DDRSMatix a)`
- `DRSMatix init_set_drsmatrix_ddrsmat(DDRSMatix org)`
- `int get_vars_ddrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)`
- `int mul_ddrsmatrix_ddvec(DDVector ret, DDRSMatix mat, DDVector vec)`
- `int mul_ddrsmatrixt_ddvec(DDVector ret, DDRSMatix mat, DDVector vec)`
- `int mul_drsmatrix_ddvec(DDVector ret, DRSMatix mat, DDVector vec)`
- `int mul_drsmatrixt_ddvec(DDVector ret, DRSMatix mat, DDVector vec)`
- `void ddpower_sp(double max_eig[DDSIZE], DDVector evec, DDRSMatix mat, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)`
- `long int absmax_index_ddvector(double ret[DDSIZE], DDVector vec)`
- `void subst_ddrsmatrix_drsmat(DDRSMatix c, DRSMatix a) ... c := (DD)a`
- `void subst_drsmatrix_ddrsmat(DRSMatix c, DDRSMatix a) ... c := (D)a`
- `void absmax_row_ddrsmatrix(double mu[DDSIZE], long int * max_j, long int row, DDRSMatix mat)`
- `void absmax_col_ddrsmatrix(double mu[DDSIZE], long int * max_i, long int col, DDRSMatix mat)`
- `void add_ddrsmatrix_drsmat(DDRSMatix c, DDRSMatix a, DRSMatix b)`
- `void sub_ddrsmatrix_drsmat(DDRSMatix c, DDRSMatix a, DRSMatix b)`
- `int split_ddrsmatrix_drsmat(DRSMatix[] ret, int num_div, DDRSMatix org)`
- `int split_ddrsmatrix_t_drsmat(DRSMatix[] ret, int num_div, DDRSMatix org)`
- `void mul_ddrsmatrix_ddvec_oz(DDVector ret, DDRSMatix a, int max_num_div_a, DDVector vb, int max_num_div_vb)`
- `void mul_ddrsmatrixt_ddvec_oz(DDVector ret, DDRSMatix a, int max_num_div_a, DDVector vb, int max_num_div_vb)`
- `void iLU0_ddrsmatrix(DDRSMatix mat)`
- `void solve_iLU0_ddrsmatrix(DDVector ret, DDRSMatix ilu, DDVector b)`
- `void solve_iLU0t_ddrsmatrix(DDVector ret, DDRSMatix ilu, DDVector b)`
- `void solve_iLU0_drsmatrix_ddvec(DDVector ret, DRSMatix ilu, DDVector b)`
- `void solve_iLU0t_drsmatrix_ddvec(DDVector ret, DRSMatix ilu, DDVector b)`

Complex DD sparse matrix (`CDDRSMatix`):

- `CDDRSMatix init_cddrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)`
- `void free_cddrsmatrix(CDDRSMatix mat)`

- void print_cddrsmatrix(CDDRSMatrix mat)
- void normf_cddrsmatrix(double ret[DDSIZE], CDDRSMatrix mat)
- CDDRSMatrix init_set_cddrsmatrix(CDDRSMatrix org)
- int mul_cddrsmatrix_cddvec(CDDVector ret, CDDRSMatrix mat, CDDVector vec)
- int mul_cddrsmatrixt_cddvec(CDDVector ret, CDDRSMatrix mat, CDDVector vec)
- int mul_cddrsmatrixs_cddvec(CDDVector ret, CDDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrixt_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrixs_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- void iLU0_cddrsmatrix(CDDRSMatrix mat)
- void solve_iLU0_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)

6.1.4 TDRSMatrix and CTDRSMatrix — triple-double sparse matrices

The `tdrsmatrix` structure stores a triple-double sparse matrix; pointer type `TDRSMatrix`. Elements are stored as double `*element[TDSIZE]`.

- TDRSMatrix init_tdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_tdrsmatrix(TDRSMatrix mat)
- void set_nzero_row_dim_td(TDRSMatrix mat)
- void print_tdrsmatrix(TDRSMatrix mat)
- void get_tdrsmatrix_ij(double ret[TDSIZE], TDRSMatrix mat, long int i, long int j)
- void set_tdrsmatrix_ij(TDRSMatrix mat, long int i, long int j, double val[TDSIZE])
- TDRSMatrix init_set_tdrsmatrix(TDRSMatrix org)
- void set0_tdrsmatrix(TDRSMatrix spmat)
- void subst_tdrsmatrix(TDRSMatrix ret, TDRSMatrix a)
- DRSMMatrix init_set_drsmatrix_tdrsmat(TDRSMatrix org)
- TDRSMatrix init_set_tdrsmatrix_tdrsmat(TDRSMatrix org)
- int get_vars_tdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_tdrsmatrix_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_tdrsmatrixt_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_drsmatrix_tdvec(TDVector ret, DRSMMatrix mat, TDVector vec)
- int mul_drsmatrixt_tdvec(TDVector ret, DRSMMatrix mat, TDVector vec)
- void tdpower_sp(double max_eig[TDSIZE], TDVector evec, TDRSMatrix mat, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int absmax_index_tdvector(double ret[TDSIZE], TDVector vec)
- void subst_tdrsmatrix_drsmat(TDRSMatrix c, DRSMMatrix a)
- void subst_drsmatrix_tdrsmat(DRSMMatrix c, TDRSMatrix a)
- void absmax_row_tdrsmatrix(double mu[TDSIZE], long int * max_j, long int row, TDRSMatrix mat)
- void absmax_col_tdrsmatrix(double mu[TDSIZE], long int * max_i, long int col, TDRSMatrix mat)
- void add_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMMatrix b)
- void sub_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMMatrix b)
- int split_tdrsmatrix_drsmat(DRSMMatrix[] ret, int num_div, TDRSMatrix org)
- int split_tdrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, TDRSMatrix org)

- void mul_tdrsmatrix_tdvec_oz(TDVector ret, TDRSMMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void mul_tdrsmatrixt_tdvec_oz(TDVector ret, TDRSMMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void iLU0_tdrsmatrix(TDRSMMatrix mat)
- void solve_iLU0_tdrsmatrix(TDVector ret, TDRSMMatrix ilu, TDVector b)
- void solve_iLU0t_tdrsmatrix(TDVector ret, TDRSMMatrix ilu, TDVector b)
- void solve_iLU0_drsmatrix_tdvec(TDVector ret, DRSMMatrix ilu, TDVector b)
- void solve_iLU0t_drsmatrix_tdvec(TDVector ret, DRSMMatrix ilu, TDVector b)

Complex TD sparse matrix (CTDRSMMatrix):

- CTDRSMMatrix init_ctdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_ctdrsmatrix(CTDRSMMatrix mat)
- void print_ctdrsmatrix(CTDRSMMatrix mat)
- CTDRSMMatrix init_set_ctdrsmatrix(CTDRSMMatrix org)
- int mul_ctdrsmatrix_ctdvec(CTDVector ret, CTDRSMMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixt_ctdvec(CTDVector ret, CTDRSMMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixs_ctdvec(CTDVector ret, CTDRSMMatrix mat, CTDVector vec)
- int mul_cdrsmatrix_ctdvec(CTDVector ret, CDRSMMatrix mat, CTDVector vec)
- int mul_cdrsmatrixt_ctdvec(CTDVector ret, CDRSMMatrix mat, CTDVector vec)
- int mul_cdrsmatrixs_ctdvec(CTDVector ret, CDRSMMatrix mat, CTDVector vec)
- void iLU0_ctdrsmatrix(CTDRSMMatrix mat)
- void solve_iLU0_ctdrsmatrix(CTDVector ret, CTDRSMMatrix ilu, CTDVector b)
- void solve_iLU0t_ctdrsmatrix(CTDVector ret, CTDRSMMatrix ilu, CTDVector b)
- void solve_iLU0s_ctdrsmatrix(CTDVector ret, CTDRSMMatrix ilu, CTDVector b)
- void solve_iLU0_cdrsmatrix_ctdvec(CTDVector ret, CDRSMMatrix ilu, CTDVector b)
- void solve_iLU0t_cdrsmatrix_ctdvec(CTDVector ret, CDRSMMatrix ilu, CTDVector b)
- void solve_iLU0s_cdrsmatrix_ctdvec(CTDVector ret, CDRSMMatrix ilu, CTDVector b)

6.1.5 QDRSMMatrix and CQDRSMMatrix — quadruple-double sparse matrices

The `qdrsmatrix` structure stores a quadruple-double sparse matrix; pointer type `QDRSMMatrix`. Elements are stored as double `*element[QDSIZE]`.

- QDRSMMatrix init_qdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_qdrsmatrix(QDRSMMatrix mat)
- void set_nzero_row_dim_qd(QDRSMMatrix mat)
- void print_qdrsmatrix(QDRSMMatrix mat)
- QDRSMMatrix init_set_qdrsmatrix_qdmatrx(QDMatrix org)
- void get_qdrsmatrix_ij(double ret[QDSIZE], QDRSMMatrix mat, long int i, long int j)
- void set_qdrsmatrix_ij(QDRSMMatrix mat, long int i, long int j, double val[QDSIZE])
- QDRSMMatrix init_set_qdrsmatrix(QDRSMMatrix org)
- void set0_qdrsmatrix(QDRSMMatrix spmat)
- void subst_qdrsmatrix(QDRSMMatrix ret, QDRSMMatrix a)
- DRSMMatrix init_set_drsmatrix_qdrsmat(QDRSMMatrix org)
- int get_vars_qdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_qdrsmatrix_qdvec(QDVector ret, QDRSMMatrix mat, QDVector vec)
- int mul_qdrsmatrixt_qdvec(QDVector ret, QDRSMMatrix mat, QDVector vec)
- int mul_drsmatrix_qdvec(QDVector ret, DRSMMatrix mat, QDVector vec)
- int mul_drsmatrixt_qdvec(QDVector ret, DRSMMatrix mat, QDVector vec)

- void qdpower_sp(double max_eig[QDSIZE], QDVector evec, QDRSMMatrix mat, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int absmax_index_qdvector(double ret[QDSIZE], QDVector vec)
- void subst_qdrsmatrix_drsmat(QDRSMMatrix c, DRSMMatrix a)
- void subst_drsmatrix_qdrsmat(DRSMMatrix c, QDRSMMatrix a)
- void absmax_row_qdrsmatrix(double mu[QDSIZE], long int * max_j, long int row, QDRSMMatrix mat)
- void absmax_col_qdrsmatrix(double mu[QDSIZE], long int * max_i, long int col, QDRSMMatrix mat)
- void add_qdrsmatrix_drsmat(QDRSMMatrix c, QDRSMMatrix a, DRSMMatrix b)
- void sub_qdrsmatrix_drsmat(QDRSMMatrix c, QDRSMMatrix a, DRSMMatrix b)
- int split_qdrsmatrix_drsmat(DRSMMatrix[] ret, int num_div, QDRSMMatrix org)
- int split_qdrsmatrix_t_drsmat(DRSMMatrix[] ret, int num_div, QDRSMMatrix org)
- void mul_qdrsmatrix_qdvec_oz(QDVector ret, QDRSMMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)
- void mul_qdrsmatrixt_qdvec_oz(QDVector ret, QDRSMMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)
- void iLU0_qdrsmatrix(QDRSMMatrix mat)
- void solve_iLU0_qdrsmatrix(QDVector ret, QDRSMMatrix ilu, QDVector b)
- void solve_iLU0t_qdrsmatrix(QDVector ret, QDRSMMatrix ilu, QDVector b)
- void solve_iLU0_drsmatrix_qdvec(QDVector ret, DRSMMatrix ilu, QDVector b)
- void solve_iLU0t_drsmatrix_qdvec(QDVector ret, DRSMMatrix ilu, QDVector b)

Complex QD sparse matrix (CQDRSMMatrix):

- CQDRSMMatrix init_cqdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_cqdrsmatrix(CQDRSMMatrix mat)
- void print_cqdrsmatrix(CQDRSMMatrix mat)
- CQDRSMMatrix init_set_cqdrsmatrix(CQDRSMMatrix org)
- int mul_cqdrsmatrix_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixt_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixs_cqdvec(CQDVector ret, CQDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrixt_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- int mul_cdrsmatrixs_cqdvec(CQDVector ret, CDRSMMatrix mat, CQDVector vec)
- void iLU0_cqdrsmatrix(CQDRSMMatrix mat)
- void solve_iLU0_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0t_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0s_cqdrsmatrix(CQDVector ret, CQDRSMMatrix ilu, CQDVector b)
- void solve_iLU0_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)
- void solve_iLU0t_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)
- void solve_iLU0s_cdrsmatrix_cqdvec(CQDVector ret, CDRSMMatrix ilu, CQDVector b)

6.1.6 MPFRSMMatrix and CMPFRSMMatrix — multiple-precision sparse matrices (requires USE_GMP)

The `mpfrsmatrix` structure stores a GMP multiple-precision sparse matrix; pointer type `MPFRSMMatrix`. Its elements are stored as `mpf_t *element`. The complex variant `CMPFRSMMatrix` uses `mpc_t *element`.

- MPFRSMMatrix init_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- MPFRSMMatrix init2_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int

- nzero_total_num, unsigned long prec)
- void free_mpfrsmatrix(MPFRSMatrix mat)
- void set_nzero_row_dim_mpf(MPFRSMatrix mat)
- void print_mpfrsmatrix(MPFRSMatrix mat)
- void get_mpfrsmatrix_ij(mpf_t ret, MPFRSMatrix mat, long int i, long int j)
- void set_mpfrsmatrix_ij(MPFRSMatrix mat, long int i, long int j, mpf_t val)
- MPFRSMatrix init_set_mpfrsmatrix(MPFRSMatrix org)
- void set0_mpfrsmatrix(MPFRSMatrix spmat)
- void set_mpfmatrix_mpfrsmat(MPFMatrix ret, MPFRSMatrix spmat)
- MPFRSMatrix init_set_mpfrsmatrix_mpfmatrix(MPFMatrix org)
- int get_vars_mpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int fread_urilinkdat_fname_mpf(MPFRSMatrix mat, const char * fname)
- long int absmax_index_mpfvector(mpf_t absmax, MPFVector vec)
- int mul_mpfrsmatrix_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_mpfrsmatrixt_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_drsmatrix_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- int mul_drsmatrixt_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- void mpfpower_rsmatrix(mpf_t max_eig, MPFVector evec, MPFRSMatrix mat, mpf_t reps, mpf_t aeps, long int maxtimes)
- int smul_mpfvector(MPFVector ret, mpf_t c, MPFVector vec)
- void iLU0_mpfrsmatrix(MPFRSMatrix mat)
- void solve_iLU0_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0t_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)
- void solve_iLU0t_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)

Complex MPF sparse matrix (CMPFRSMatrix):

- CMPFRSMatrix init_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- CMPFRSMatrix init2_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num, unsigned long prec)
- void free_cmpfrsmatrix(CMPFRSMatrix mat)
- void set_nzero_row_dim_cmpf(CMPFRSMatrix mat)
- void print_cmpfrsmatrix(CMPFRSMatrix mat)
- void normf_cmpfrsmatrix(mpf_t ret, CMPFRSMatrix mat)
- mpc_ptr get_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j)
- void set_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j, mpc_t val)
- CMPFRSMatrix init_set_cmpfrsmatrix(CMPFRSMatrix org)
- void set_cmpfmatrix_cmpfrsmat(CMPFMatrix ret, CMPFRSMatrix spmat)
- CMPFRSMatrix init_set_cmpfrsmatrix_cmpfmatrix(CMPFMatrix org)
- int get_vars_cmpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_cmpfrsmatrix_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cmpfrsmatrixt_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cmpfrsmatrixs_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mulcdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- int mulcdrsmatrixt_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- int mulcdrsmatrixs_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- void iLU0_cmpfrsmatrix(CMPFRSMatrix mat)
- void solve_iLU0_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)
- void solve_iLU0t_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)
- void solve_iLU0s_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)

- void solve_iLU0_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)
- void solve_iLU0t_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)
- void solve_iLU0s_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)

Conversion functions from MPFRSMatrix to other types (requires USE_GMP):

- DRSMatrix init_set_drsmatrix_mpfrsmatrix(MPFRSMatrix org)
- CDRSMatrix init_set_cdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)
- DDRSMatrix init_set_ddrsmatrix_mpfrsmatrix(MPFRSMatrix org)
- CDDRSMatrix init_set_cddrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)
- TDRSMatrix init_set_tdrsmatrix_mpfrsmatrix(MPFRSMatrix org)
- CTDRSMatrix init_set_ctdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)
- QDRSMatrix init_set_qdrsmatrix_mpfrsmatrix(MPFRSMatrix org)
- CQDRSMatrix init_set_cqdrsmatrix_cmpfrsmatrix(CMPFRSMatrix org)

6.2 Krylov subspace iterative solvers

6.2.1 Conjugate Gradient method (FCG, DCG, DDCG, TDCG, QDCG, MPFCG)

- long int FCG(FVector ans, FMatrix a, FVector b, float reps, float aeps, long int maxtimes)
- long int DCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int bnc_DCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int CDCOCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCOCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

Double-double CG:

- long int DDCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCG_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCOCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

Triple-double CG:

- long int TDCG_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCG_sp_d(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCOCG_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCOCG_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double

- `aeps[TDSIZE], long int maxtimes)`
- `long int CTDCOCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)`

Quadruple-double CG:

- `long int QDCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`
- `long int QDCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`
- `long int QDCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`
- `long int CQDCOCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`
- `long int CQDCOCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`
- `long int CQDCOCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)`

Multiple-precision CG (requires `USE_GMP`):

- `long int MPFCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int MPFCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int MPFCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int CMPFCOCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int CMPFCOCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`
- `long int CMPFCOCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)`

6.2.2 BiCG, CGS, BiCGSTAB, GPBiCG methods (double)

Each solver is provided in sparse (`_sp`), sparse with preconditioning (`_sp_iLU0`), and dense variants.

- `long int DBiCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int DCGS_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int DBiCGSTAB_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int DGPBiCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)`
- `long int DBiCG_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)`
- `long int DCGS_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)`
- `long int DBiCGSTAB_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)`
- `long int DGPBiCG_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)`
- `long int DBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)`

- long int DCGS(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCGSTAB(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DGPBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCGS_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCGSTAB_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDCGS_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCGSTAB_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDGPBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCGS(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCGSTAB(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

6.2.3 BiCG, CGS, BiCGSTAB, GPBiCG methods (double-double)

- long int DDBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCG_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDCGS_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDBiCGSTAB_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDGPBiCG_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDBiCG_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

- double aeps[DDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CDDBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

6.2.4 BiCG, CGS, BiCGSTAB, GPBiCG methods (triple-double)

- long int TDBiCG_sp(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCG_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_iLU0(TDVector ans, TDRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCG_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp_d(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCG_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_d_iLU0(TDVector ans, DRSMMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMMatrix ilu, double * norm2_res_history)
- long int TDBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

- long int CTDBiCG_sp(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCG_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp_d(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_d_iLU0(CTDVector ans, CDRSMMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMMatrix ilu, double * norm2_res_history)
- long int CTDBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

6.2.5 BiCG, CGS, BiCGSTAB, GPBiCG methods (quadruple-double)

- long int QDBiCG_sp(QDVector ans, QDRSMMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS_sp(QDVector ans, QDRSMMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB_sp(QDVector ans, QDRSMMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG_sp(QDVector ans, QDRSMMatrix a, QDVector b, double reps[QDSIZE], double

- [illegible]

- aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDCGS_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCGSTAB_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDGPBiCG_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

6.2.6 BiCG, CGS, BiCGSTAB, GPBiCG methods (multiple-precision; requires USE_GMP)

- long int MPFBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCGSTAB_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCG_sp_d(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp_d(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp_d(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp_d(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_d_iLU0(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_d_iLU0(MPFVector ans, DRSMMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMMatrix ilu, MPFVector norm2_res_history)

- [illegible]

Chapter 7

Single-based and complex precision families

Chapters 2–6 describe the `double`-based real types `D`, `DD`, `TD`, `QD` and the arbitrary precision type `MPF`. `BNCmatmul` also provides two further groups of fixed-precision types that share the very same set of operations: a *single-based* family (built on `float`) and a *complex* family. This chapter summarises their data types; every vector, matrix, multiplication, LU and sparse routine of the previous chapters exists for these types as well, obtained by substituting the type prefix in the function name.

7.1 Single-based multi-component types (DS, TS, QS)

The single-based family stores a number as a non-overlapping sum of two, three or four `float` values (the “multi-component” way applied to `float`). It is declared in `rds.h` and the linear headers `dslinear.h`, `tslinear.h`, `qslinear.h`.

type	components	precision (bits, $\approx$)	decimal digits ($\approx$)
DS (double-single)	<code>float</code> ×2	48	14
TS (triple-single)	<code>float</code> ×3	72	21
QS (quadruple-single)	<code>float</code> ×4	96	28

Table 7.1 Single-based multi-component types. `DSSIZE`= 2, `TSSIZE`= 3, `QSSIZE`= 4.

The element-level arithmetic uses the macro set `rds_*`, `rts_*`, `rqs_*` (analogous to `rdd_*` of Section 2.9), e.g. `rds_add`, `rts_mul`, `rqs_div`, `rqs_sqrt`. The vector and matrix objects and their operations mirror Chapter 3 exactly:

- `DSVector` `init_dsvector`(long int dim) ... and `init_tsvector`, `init_qsvector`, `init_dsmatrix`, ...
- `void` `add_dsvector`(`DSVector` c, `DSVector` a, `DSVector` b) ... vector $\pm$, `cmul_dsvector`, `ip_dsvector`, `norm2_dsvector`, etc.
- `void` `mul_dsmatrix`(`DSMatrix` C, `DSMatrix` A, `DSMatrix` B) ... $C := AB$; also `mul_dsmatrix_block`, `mul_dsmatrix_strassen_even`, `mul_dsmatrix_dsvec`, `mul_dsmatrixt_dsvec`.
- `int` `DSLUDecomp`(`DSMatrix` A) ... LU decomposition; also `DSLUDecompP` (partial pivoting), `SolveD-SLS`, `SolveDSLSP` and the TS/QS variants.

The accessor of an element is the value structure `dsfloat` / `tsfloat` / `qsfloat` (a `float` array of `DSSIZE` etc.); read it with `get_dsmatrix_ij_dsfloat` and store the returned value in a local variable before using its `.val` pointer.

7.2 Complex types (CD and complex multi-component)

A complex number is stored as a separate real part and imaginary part. The native complex `double` type `CD` keeps a C99 `double` `_Complex` array; the complex multi-component types keep the real and imaginary parts as two vectors/matrices of the corresponding real type.

type	real/imag element	header
CD	double _Complex	cdlinear.h, clinear.h
CDD	DD (double-double)	cddlinear.h
CTD	TD (triple-double)	ctdlinear.h
CQD	QD (quadruple-double)	cqdlinear.h
CDS	DS (double-single)	cdslinear.h
CTS	TS (triple-single)	ctslinear.h
CQS	QS (quadruple-single)	cqslinear.h

Table 7.2 Complex types. The multi-component complex element value is `cXXfloat = {val_re[.], val_im[.]}` (see `rcdd.h`, `rcds.h`).

7.2.1 Native complex double (CD)

- `CDVector init_cdvector(long int dim) ...` and `init_cdmatrix`.
- `void set_cdvector_i(CDVector vec, long int index, double _Complex val) ...` and `get_cdvector_i`, `set_cdmatrix_ij`, `get_cdmatrix_ij`.
- `void mul_cdmatrix(CDMatrix C, CDMatrix A, CDMatrix B) ...  $C := AB$` ; also `mul_cdmatrix_cdvec`, `mul_cdmatrixt_cdvec`.
- `int CDLUdecomp(CDMatrix A) ... LU`; also `CDLUdecompP` (partial pivoting) and `CDLUdecompC` (complete pivoting).

7.2.2 Complex multi-component (CDD, CTD, CQD, CDS, CTS, CQS)

The vector/matrix structure holds two parts of the underlying real type:

```
typedef struct { XXVector re; XXVector im; } cXXvector; /* e.g. DDVector re, im */
typedef struct { XXMatrix re; XXMatrix im; } cXXmatrix;
```

- `CDDVector init_cddvector(int dim) ...` and `init_cddmatrix` and the CTD/CQD/CDS/CTS/CQS analogues.
- `void set_cddvector_i(CDDVector vec, long int index, cddfloat* val) ...` element set/get via the `cXXfloat` value (`get_cddvector_i_cddfloat`, `get_cddmatrix_ij_cddfloat`, `set_cddmatrix_ij`).
- `void mul_cddmatrix_4m(CDDMatrix C, CDDMatrix A, CDDMatrix B) ...` complex matrix multiplication using 4 real multiplications ($\text{Re} = A_r B_r - A_i B_i$, $\text{Im} = A_r B_i + A_i B_r$); the 3-multiplication (Karatsuba) variant is `mul_cddmatrix_3m`.
- `void mul_cddmatrix_cdvec_4m(CDDVector v, CDDMatrix A, CDDVector vb) ...` matrix-vector product; also `mul_cddmatrixt_cdvec` ($A^T \mathbf{vb}$).
- `double normf_cddmatrix(CDDMatrix A) ...` Frobenius norm; block/Strassen multiplication is available as `mul_cddmatrix_block_4m`, `mul_cddmatrix_strassen_4m`.

Each of CTD, CQD, CDS, CTS, CQS provides the identical set with its own prefix (e.g. `mul_cqsmatrix_4m`, `init_ctsmatrix`). The sparse-matrix counterparts (CRS storage and SpMV) for CD, CDD, CTD, CQD are described together with the real sparse types in Chapter 6 (`sparse_cd.c`, `sparse_cdd.c`, `sparse_ctd.c`, `sparse_cqd.c`).

7.3 Summary of available families

group	types	where
real, <code>double</code> -based	D, DD, TD, QD, MPF	Chap. 3–6
real, <code>float</code> -based	F, DS, TS, QS	this chapter
complex	CD, CDD, CTD, CQD, CDS, CTS, CQS, CMPF	this chapter, Chap. 6
GPU (CUDA)	<i>g/gc/cg</i> variants of all of the above	Chap. 8

Table 7.3 Precision families provided by BNCmatmul.

Chapter 8

GPU computation with CUDA

This chapter describes the CUDA implementation of BNCmatmul, which executes the basic linear computation, matrix multiplication, LU decomposition and sparse matrix-vector multiplication on an NVIDIA GPU. It is built on top of **gdtq-0.0.2** (the device multi-component floating-point header layer) and, for arbitrary precision, on **mpc_cuda-0.0.1** (a device port of MPFR/MPC). The reference target is an NVIDIA GB10 (**sm_121**, CUDA 13).

All GPU functions are gathered into the static library **libbncmatmul-0.24_cuda.a**. The single script **bench/build_cuda_full.sh** compiles every GPU module into this library and runs the whole verification suite (matrix multiplication, matrix-vector product, LU and SpMV for every precision, plus the CUDA MPFR/MPC matrix multiplication).

8.1 Naming convention and data types

Every GPU type and function carries a leading **g** (“GPU”). The element type follows the same multi-component scheme as the CPU side: a **d**-family based on **double** and an **s**-family based on **float**.

GPU prefix	element type	bits ($\approx$)	CPU counterpart
gf	float (float)	24	FVector / FMatrix
gd	double (double)	53	DVector / DMatrix
gds	double-single (float2)	48	DSVector / DSMatrix
gts	triple-single (float3)	72	TSVector / TSMatrix
gqs	quadruple-single (float4)	96	QSVector / QSMatrix
gdd	double-double (double2)	106	DDVector / DDMatrix
gtd	triple-double (double3)	159	TDVector / TDMatrix
gqd	quadruple-double (double4)	212	QDVector / QDMatrix

Table 8.1 Real-valued GPU element types. **g τ** with $\tau \in \{f, d, ds, ts, qs, dd, td, qd\}$.

For complex numbers two families are provided. The native complex types **gcd** (complex **double**) and **gcf** (complex **float**) store the real and imaginary parts as two separate scalar arrays. The multi-component complex types **cgdd**, **cgtd**, **cgqd**, **cgds**, **cgts**, **cgqs** store the real and imaginary parts as two separate arrays of the corresponding **g τ** element.

Each family defines a vector and a matrix object. For a real type **g τ** :

```
typedef struct { long int dim;          gXX_real *element; } gXXvector;  
typedef struct { long int row_dim, col_dim; gXX_real *element; } gXXmatrix;
```

and for the complex types the single **element** pointer is replaced by a pair **re**, **im**. A pointer to the structure is named **GXXVector** / **GXXMatrix** (upper case), e.g. **GDDVector**, **GCDMatrix**, **CGTSVector**.

Matrices are stored row-major with stride equal to **col_dim** (the GPU side does *not* use the SIMD column padding of the CPU side). Two block/grid arguments **int num_blocks_per_grid** and **int num_threads_per_block** are passed to every kernel-launching function; typical values are 32–128 each.

8.2 Device memory and host-device transfer

In the list below `g $\tau$` stands for any of the real GPU types and `XX` for its CPU counterpart (e.g. `gdd $\leftrightarrow$ DD`).

- `GXXVector init_gXXvector(long int dim) ...` Allocate a vector in host memory.
- `GXXVector init_gXXvector_dev(long int dim) ...` Allocate a vector in device (GPU) memory.
- `void free_gXXvector(GXXVector vec)`
- `void free_gXXvector_dev(GXXVector vec)`
- `GXXMatrix init_gXXmatrix(long int row_dim, long int col_dim)`
- `GXXMatrix init_gXXmatrix_dev(long int row_dim, long int col_dim)`
- `void free_gXXmatrix_dev(GXXMatrix mat)`
- `void subst_gXXvector_dev_XXvec(GXXVector dev, XXVector src) ...` Copy a CPU vector to the GPU (host $\rightarrow$ device).
- `void subst_XXvector_gXXvec_dev(XXVector dst, GXXVector dev) ...` Copy a GPU vector back to the CPU (device $\rightarrow$ host).
- `void subst_gXXmatrix_dev_XXmat(GXXMatrix dev, XXMatrix src)`
- `void subst_XXmatrix_gXXmat_dev(XXMatrix dst, GXXMatrix dev)`
- `void print_gXXvector_dev(GXXVector dev)`
- `void print_gXXmatrix_dev(GXXMatrix dev)`

For the native complex types `gcd/gcf` the CPU counterpart is the complex-double `CDVector/CDMatrix`; the transfer functions are `subst_gcdvector_dev_cdvec` and `subst_cdvector_gcdvec_dev` (and the matrix variants). For the multi-component complex types `cg $\tau$` the CPU counterpart is the complex multi-component `CXXVector/CXXMatrix` (e.g. `CDDMatrix`), with `subst_cgddvector_dev_cddvec` etc.

8.3 Vector and matrix arithmetic

All routines below run on the GPU and take the two launch arguments `int nb $\tau$` (blocks) and `int nt $\tau$` (threads); they are omitted from the argument lists for brevity.

- `void add_gXXvector_dev(GXXVector c, GXXVector a, GXXVector b) ...` $\mathbf{c} := \mathbf{a} + \mathbf{b}$.
- `void sub_gXXvector_dev(GXXVector c, GXXVector a, GXXVector b) ...` $\mathbf{c} := \mathbf{a} - \mathbf{b}$.
- `void cmul_gXXvector_dev(GXXVector c, gXX_real val, GXXVector a) ...` $\mathbf{c} := \text{val} \cdot \mathbf{a}$.
- `void subst_gXXvector_dev(GXXVector c, GXXVector a) ...` $\mathbf{c} := \mathbf{a}$.
- `void set0_gXXvector_dev(GXXVector c) ...` $\mathbf{c} := \mathbf{0}$.
- `void ip_gXXvector_dev(gXX_real* ret_dev, GXXVector a, GXXVector b) ...` inner product $(\mathbf{a}, \mathbf{b})$, result written to device scalar `ret_dev`.
- `void norm2_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...` $\|\mathbf{a}\|_2$.
- `void norm1_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...` $\|\mathbf{a}\|_1$.
- `void normi_gXXvector_dev(gXX_real* ret_dev, GXXVector a) ...` $\|\mathbf{a}\|_\infty$.
- `void add_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...` $C := A + B$.
- `void sub_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...` $C := A - B$.
- `void cmul_gXXmatrix_dev(GXXMatrix C, gXX_real sc, GXXMatrix A) ...` $C := \text{sc} \cdot A$.
- `void transpose_gXXmatrix_dev(GXXMatrix C, GXXMatrix A) ...` $C := A^T$.
- `void subst_gXXmatrix_dev(GXXMatrix C, GXXMatrix A) ...` $C := A$.
- `void set0_gXXmatrix_dev(GXXMatrix C) ...` $C := \mathbf{O}$.
- `void setI_gXXmatrix_dev(GXXMatrix C) ...` $C := \mathbf{I}$.
- `void normf_gXXmatrix_dev(gXX_real* ret_dev, GXXMatrix A) ...` Frobenius norm $\|A\|_F$.

The complex families additionally provide `conjtrans_gcdmatrix_dev` / `conjtrans_cgXXmatrix_dev` ($C := A^H$, conjugate transpose), and the complex scalar multiplication takes the real and imaginary parts as two arguments, e.g. `cmul_gcdvector_devGCDVector c, double val_re, double val_im, GCDVector a`.

8.4 Matrix multiplication and matrix-vector product

- `void mul_gXXmatrix_dev(GXXMatrix C, GXXMatrix A, GXXMatrix B) ...  $C := AB$  (one thread per output row, grid-stride).`
- `void mul_gXXmatrix_gXXvec(GXXVector v, GXXMatrix A, GXXVector vb) ...  $v := Avb$ .`
- `void mul_gXXmatrixt_gXXvec(GXXVector v, GXXMatrix A, GXXVector vb) ...  $v := A^T vb$ .`

For the complex families the same three routines exist, e.g. `mul_gcdmatrix_dev`, `mul_cgddmatrix_dev`, `mul_cgddmatrix_cgddvec` and `mul_cgddmatrixt_cgddvec`; complex multiplication uses the 4-real-multiply formula $\text{Re}(AB) = A_r B_r - A_i B_i$, $\text{Im}(AB) = A_r B_i + A_i B_r$.

A Strassen matrix multiplication for the multi-component types is provided in `src/matmul_strassen_general_g{dd,ts,qs}`.

Verified accuracy of $C := AB$ on the GB10 against the corresponding CPU routine: `gf` and the quadruple-single `cgqs` are bit-identical (0); `gd` $\sim 10^{-16}$, `gcd` $\sim 10^{-19}$, `gdd` $\sim 10^{-31}$, `gds` $\sim 10^{-14}$, `gts` $\sim 10^{-22}$.

8.5 LU decomposition and linear solver on GPU

A right-looking LU decomposition with partial pivoting and a forward/back substitution solver are provided for every real type and for the native complex types. The decomposition and the solve are combined in one driver:

- `int gXX_lu_solve_dev(GXXMatrix A, GXXVector b, GXXVector x, long int ch[], int blocks, int threads) ...` Overwrite A with its LU factors and solve $Ax = b$ into x (all device objects). The pivot row chosen at each step is recorded in `ch[]` (host, length n ; may be NULL). Returns 0 on success.

`XX` is one of `d`, `f`, `dd`, `td`, `qd`, `ds`, `ts`, `qs` (real, e.g. `gqs_lu_solve_dev`), or `cd`, `cf` (native complex, `gcd_lu_solve_dev` / `gcf_lu_solve_dev`). All ten drivers are declared `extern "C"`. Verified maximum relative error against the known solution (dimension 100): `gd` 1.2×10^{-12} , `gf` 9.5×10^{-7} , `gcd` 2.8×10^{-15} , `gdd/gtd/gds` 0, `gts` 7.7×10^{-22} , `gqs` 2.8×10^{-29} .

8.6 Sparse matrix-vector multiplication (SpMV)

The GPU sparse matrix is stored in the standard CSR (Compressed Sparse Row) format, independent of the CPU “DRS” format. The structure holds the dimensions, the number of non-zeros, and three device arrays:

```
typedef struct {
    long int row_dim, col_dim, nnz;
    long int *row_ptr; /* length row_dim + 1 */
    long int *col_idx; /* length nnz */
    gXX_real *val; /* length nnz */
} gXXspmatrix; /* GXXSPMatrix = gXXspmatrix * */
```

- `GXXSPMatrix init_gXXspmatrix_dev(long int row_dim, long int col_dim, long int nnz, const long int* row_ptr, const long int* col_idx, const gXX_real* val) ...` Build a device CSR matrix from host CSR arrays.
- `void free_gXXspmatrix_dev(GXXSPMatrix mat)`
- `void mul_gXXspmatrix_gXXvec(GXXVector ret, GXXSPMatrix A, GXXVector x, int nbg, int ntb) ...` $ret := Ax$ (one thread per row).

The native `gd/gf` SpMV uses the `GDVector/GFVector` dense vectors as above; the multi-component real types (`gdd...gqs`) take raw device `gXX_real *` arrays for x and ret via `mul_gXXspmatrixgXX_real*` y , `GXXSPMatrix A`, `const gXX_real* x`, The native complex `gcd/gcf` and multi-component complex

cg7 SpMV keep the real/imaginary value arrays separate, e.g. `mul_gcdspmatrixdouble* yre, double* yim`, GCDSPMatrix `A`, `const double* xre, const double* xim, ...`. The transpose product $A^T \mathbf{x}$ is obtained either by the native `mul_gdspmatrixt_gdvec` (atomic scatter) or, for the multi-component types, by building the CSR of A^T on the host and reusing the $A\mathbf{x}$ kernel.

8.7 Arbitrary precision matrix multiplication (CUDA MPFR / MPC)

Using the device port `mpc_cuda`, the dot products of a matrix multiplication or a matrix-vector product can be accumulated at an arbitrary compile-time precision (default 1024 bits) on the GPU. The matrices and vectors are passed as plain row-major `double` arrays (the complex variants use separate real/imaginary `double` arrays).

- `int cuda_mul_mpfmatrix(double* C, const double* A, const double* B, int n, int lblocks, int lthreads) ...  $C := AB$  ( $n \times n$ ), each entry accumulated in MPFR at PREC bits.`
- `int cuda_mul_mpfmatrix_vec(double* y, const double* A, const double* x, int n, int lblocks, int lthreads) ...  $y := Ax$ .`
- `int cuda_mul_cmpfmatrix(double* Cr, double* Ci, const double* Ar, const double* Ai, const double* Br, const double* Bi, int n, int lblocks, int lthreads) ... Complex  $C := AB$  accumulated in MPC.`
- `int cuda_mul_cmpfmatrix_vec(double* yr, double* yi, const double* Ar, const double* Ai, const double* xr, const double* xi, int n, int lblocks, int lthreads)`

These routines link the static library `/usr/local/lib/libmpc_cuda.a` and require the device stack-/heap limits to be enlarged (the drivers set `cudaLimitStackSize` and the device heap internally). Verified on the GB10: both the MPFR ($n = 96$) and MPC ($n = 80$) matrix multiplication and matrix-vector product agree with the CPU result to 0 at 1024 bits.

8.8 Building and running the GPU suite

```
sh bench/build_cuda_full.sh
```

compiles every module listed above into `libbncmatmul-0.24_cuda.a` and runs the complete verification. The device-math operators are supplied by the `gdtq` aggregators: `gqd.cu` (`double2/3/4`) for the d-family and `gqs.cu` (`float2/3/4`) for the s-family; the `gdd/gds` objects already carry their operators inline, while `gtd/gqd/gts/gqs` are device-linked against the corresponding aggregator object. The arbitrary precision modules need the `mpc_cuda` headers under `/usr/local/include/mpc_cuda` and the definitions in `/usr/local/share/mpc_cuda/mpfr_cuda_defs.txt`.

Chapter 9

Using BNCmatmul from Python

This chapter shows how to call the DD/TD/QD (and DS/TS/QS) arithmetic, FMA and elementary functions of BNCmatmul from Python via the standard `ctypes` module, using the exported function API of Sec. 2.8 (`src/rdtq_func.c` / `include/rdtq_func.h`) and the elementwise array functions of Sec. 2.7. All code fragments in this chapter are taken from `python/example_rdtq.py`, which is shipped with the library and can be executed as-is; `python/test_rdtq.py` is the corresponding regression driver.

9.1 Building the shared library

The exported symbols live in a small shared library `python/librdtq.so`, built from `rdtq_func.c` (99 scalar functions) and `elem_vector.c` (12 array functions) by the legacy make targets:

```
$ cd bncmatmul-0.24/src
$ make -f Makefile.legacy librdtq      # generic build
$ make -f Makefile.legacy librdtq_neon # Arm NEON build
```

or directly with the compiler:

```
$ gcc -O3 -ffp-contract=off -fPIC -shared -Iinclude \
    src/rdtq_func.c src/elem_vector.c -o python/librdtq.so -lm
```

(`-ffp-contract=off` is mandatory, as everywhere in BNCmatmul.) The full library shared objects `libbncmatmul-0.24*.so` contain the same symbols plus the whole linear-computation API; `librdtq.so` is merely the smallest self-contained subset for scripting use.

9.2 Loading the library and printing full precision

A DD/TD/QD value is an unevaluated sum of 2/3/4 doubles, so the natural `ctypes` representation is a small `c_double` array. Since every limb is an exact binary value, converting each limb to `decimal.Decimal` and summing reproduces the full-precision value exactly — Python's `float` arithmetic would round it back to 53 bits:

```
import ctypes as ct
from decimal import Decimal, getcontext

lib = ct.CDLL("./librdtq.so")

DD = ct.c_double * 2    # double-double   (~32 decimal digits)
TD = ct.c_double * 3    # triple-double   (~48 decimal digits)
QD = ct.c_double * 4    # quad-double    (~63 decimal digits)

getcontext().prec = 70
def to_decimal(limbs):
```

```

    return sum((Decimal(x) for x in limbs), Decimal(0))

x = DD(1.5, 0.0)          # x = 1.5 (high limb, low limb)
r = DD()
lib.rdd_exp(r, x)         # result first, as in rdd.h
print("dd exp(1.5) =", to_decimal(r))
# dd exp(1.5) = 4.48168907033806482260205546011928...

```

Note the argument order: as in the `rdd.h` convention, the result comes first. The float-based families use `ct.c_float * 2/3/4` arrays and the `rds_ / rts_ / rqs_` prefixes.

9.3 Arithmetic, FMA and elementary functions

The next fragment exercises quad-double square root, the certified branch-free FMA (as an exact residual check $\sqrt{2}^2 - 2$), the FMA-driven triple-double division, and the simultaneous sin/cos:

```

# quad-double: sqrt(2) to ~63 digits
two = QD(2.0, 0.0, 0.0, 0.0)
s = QD()
lib.rqd_sqrt(s, two)
print("qd sqrt(2) =", to_decimal(s))
# 1.41421356237309504880168872420969807856967187537694807317667973...

# residual sqrt(2)^2 - 2 via the certified FMA: fma(s, s, -2)
m2 = QD(-2.0, 0.0, 0.0, 0.0)
resid = QD()
lib.rqd_fma(resid, s, s, m2)
print("qd sqrt(2)^2 - 2 =", to_decimal(resid))    # ~0 (qd eps level)

# FMA-driven division: 1/3 in triple-double
a = TD(1.0, 0.0, 0.0); b = TD(3.0, 0.0, 0.0); q = TD()
lib.rtd_div_fma(q, a, b)
print("td 1/3 =", to_decimal(q))
# 0.333333333333333333333333333333333333333333333333333333327...

# simultaneous sin/cos (two results, then the input)
sin_x = DD(); cos_x = DD()
lib.rdd_sincos(sin_x, cos_x, x)

# div/sqrt-safe FMA: the scalar multiplier is a plain c_double
res = DD()
lib.rdd_fma_safe(res, DD(3.0, 0.0), ct.c_double(-0.5), DD(1.5, 0.0))
print("1.5 - 0.5*3 =", to_decimal(res))          # exactly 0

```

Every exported function of Sec. 2.8 is callable in the same style: `rXX_add / sub / mul / div / sqrt`, `rXX_fma / fma_safe / div_fma`, and `rXX_exp / expm1 / log / log10 / sin / cos / sincos / tan / nint`.

9.4 Elementwise array functions with NumPy

The elementwise array functions `bnc_XX_yy_array` (Sec. 2.7) operate on limb-planar (SoA) arrays: limb k of all n elements is one contiguous double array. This maps directly onto separate NumPy arrays, one per limb, passed as a `double*[K]` pointer array — no copying is involved, and one call replaces n round

trips through ctypes:

```
import numpy as np

n = 100000
x_hi = np.linspace(-10.0, 10.0, n)      # high limbs
x_lo = np.zeros(n)                      # low limbs
r_hi = np.empty(n); r_lo = np.empty(n)

def planar(*arrays):
    """double*[k] view of k numpy arrays (limb-planar layout)"""
    ptr = ct.POINTER(ct.c_double) * len(arrays)
    return ptr(*[a.ctypes.data_as(ct.POINTER(ct.c_double))
                  for a in arrays])

lib.bnc_dd_exp_array(ct.c_long(n),
                    planar(r_hi, r_lo), planar(x_hi, x_lo))

err = np.max(np.abs((r_hi + r_lo) / np.exp(x_hi) - 1.0))
print("max rel.err vs numpy =", err)    # ~2.2e-16 (binary64 comparison floor)
```

The same pattern works for `bnc_dd_log_array` / `sin` / `cos` and, with three or four limb arrays, for the `bnc_td_*_array` / `bnc_qd_*_array` functions. The array layout is identical to the library's `ddvector`/`tdvector`/`qdvector` types (`vec`->`element`), so results can be exchanged with C code without conversion.

9.5 Notes and caveats

- Result arguments come first (the `rdd.h` convention); the C-level `c_dd_*` functions use the opposite (input-first) order. Mixing them up produces silently wrong results, so keep to the `rXX_*` API from Python.
- Scalar arguments must be wrapped explicitly (`ct.c_double(...)` / `ct.c_float(...)`), e.g. the scalar multiplier of `rXX_fma_safe`; bare Python floats passed by ctypes default to `int`-like conversion and corrupt the call.
- Inputs are expected as normalized limb tuples. Values constructed as `(x, 0, ...)` from a Python float are always safe; results returned by the library are normalized.
- For full-precision I/O beyond `to_decimal`, the MPFR-based conversion functions of `mpfr_dtq_sd.c` (e.g. `rdd_get_str`) are exported by the full `libbncmatmul-0.24*.so` libraries and by `librdd.so` (see `python/rdd.py`).

Bibliography

- [1] D.H. Bailey. QD. <https://www.davidhbailey.com/dhbsoftware/>.
- [2] T. Granlaud and GMP development team. The GNU Multiple Precision arithmetic library. <https://gmplib.org/>.
- [3] M. Jolders, J.-M. Muller, V. Popescu, and W. Tucker. Campary: Cuda mutiple precision arithmetic library and applications. *5th ICMS*, 2016.
- [4] Tomonori Kouya. BNCpack. <https://na-inet.jp/na/bnc/>.
- [5] LAPACK. <http://www.netlib.org/lapack/>.
- [6] MPLAPACK/MPBLAS. Multiple precision arithmetic LAPACK and BLAS. <https://github.com/nakatamaho/mplapack>.
- [7] Daichi Mukunoki, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura. Accurate matrix multiplication on binary128 format accelerated by ozaki scheme. In *50th International Conference on Parallel Processing*, ICPP 2021, New York, NY, USA, 2021. Association for Computing Machinery.
- [8] MPFR Project. The MPFR library. <https://www.mpfr.org/>.
- [9] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part I: Faithful rounding. *SIAM Journal on Scientific Computing*, Vol. 31, No. 1, pp. 189–224, 2008.
- [10] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part II: Sign, k-fold faithful and rounding to nearest. *SIAM Journal on Scientific Computing*, Vol. 31, No. 2, pp. 1269–1302, 2008.
- [11] Taiga Utsugiri and Tomonori Kouya. Acceleration of multiple precision matrix multiplication using ozaki scheme, 2023.